

LATVIJAS UNIVERSITĀTE
EKSAKTO ZINĀTŅU UN TEHNOLOĢIJU FAKULTĀTE

SENSORU MONITORINGA SISTĒMA
KVALIFIKĀCIJAS DARBS

Darba autors
Markuss Birznieks, mb20075

Darba vadītājs
Dr. sc. comp. Guntis Arnicāns

RĪGA 2025

ANOTĀCIJA

Kvalifikācijas darbā ir aprakstīta sistēma, kas izstrādāta, lai uzraudzītu Raiņa bulvārī 19 izvietotos Aranet sensorus. Sistēma uztver sensoru mērījumus, izmantojot MQTT brokeri, un saglabā tos MongoDB datubāzē. Tīmekļa lietotne ļauj apskatīt sensoru reālā laika mērījumus, tostarp precizēt sensoru atrašanās vietas, kā arī atslēgt sensorus no sistēmas un pārvaldīt paziņojumus. Reālā laika mērījumu apskati nodrošina pielietojot tehnoloģijas MongoDB Change Streams un SignalR. Paziņojumu ģenerēšana brīdina par sensoru mērījumiem, pamatojoties uz lietotāja definētiem sliekšņa noteikumiem. Papildus, datu analīzei tiek nodrošināts datu eksports uz SQL datubāzēm.

Sistēma nav ierobežota tikai Aranet sensoriem – tiek definēts MQTT ziņojumu formāts, kas ļauj apstrādāt un uzglabāt dažādus mērījumus un metadatus elastīgā MongoDB datubāzē. Programmatūras pamatā ir C# un .NET, savukārt tīmekļa lietotne ir izstrādāta TypeScript un Angular, un sistēmas komponentes tiek darbinātas Docker konteineros.

Atslēgas vārdi: sensoru uzraudzība, Aranet, MQTT, C#, .NET, MongoDB

ABSTRACT

SENSOR MONITORING SYSTEM

The qualification paper describes a system developed for monitoring Aranet sensors located at Raiņa bulvāris 19. The system collects sensor measurements using an MQTT broker and stores them in a MongoDB database. A web application enables users to view real-time sensor measurements, specify sensor locations, deactivate sensor data collection in the system, and manage notifications. Real-time measurement display is achieved through MongoDB Change Streams and SignalR technologies. Notifications are generated based on user-defined threshold rules for sensor measurements. Additionally, the system includes functionality for exporting sensor data to SQL databases for further analysis.

The system is not limited to Aranet sensors - the defined MQTT message format allows processing and storing diverse measurements and metadata in a flexible MongoDB database. The software is developed using C# and .NET, but the web application is built with TypeScript and Angular, and the system components are deployed in Docker containers.

Keywords: sensor monitoring, Aranet, MQTT, C#, .NET, MongoDB

Saturs

Ievads	8
Nolūks	8
Darbības sfēra.....	8
Saistība ar citiem dokumentiem	8
Pārskats	8
Apzīmējumu saraksts	9
1 Vispārīgais apskats	10
1.1 Esošā stāvokļa apraksts	10
1.2 Pasūtītājs	10
1.3 Produkta perspektīva	10
1.4 Darījumprasības.....	10
1.5 Sistēmas lietotāji.....	11
1.6 Vispārējie ierobežojumi	12
1.7 Pieņemumi un atkarības	12
2 Programmatūras prasību specifikācija	13
2.1 Konceptuālais datu bāzes apraksts	13
2.2 Funkcionālās prasības.....	15
2.2.1 Sensoru uztveršanas modulis.....	16
2.2.2 Sensoru paneļa modulis	19
2.2.3 Sensoru pārvaldības modulis	22
2.2.4 Datu eksporta modulis	24
2.2.5 Paziņojumu modulis	27
2.3 Nefunkcionālo prasību specifikācija	31
3 Programmatūras projektējuma apraksts	32
Ievads	32
3.1 Datubāzes projektējums.....	35

3.1.1	Loģiskais datubāzes modelis	35
3.1.2	Fiziskais datubāzes modelis.....	36
3.1.3	Kolekcija “sensors”	37
3.1.4	Kolekcija “sensorMeasurements”	37
3.1.5	Kolekcija “sensorMetadata”	38
3.1.6	Kolekcija “notificationRules”.....	38
3.1.7	Kolekcija “notifications”	39
3.1.8	Konfigurācijas modelis Mongo datubāzes savienojumam	40
3.2	Komponente “Sensor.Consumer”	41
3.2.1	Atbalstītie MQTT ziņojumi	42
3.2.2	Konfigurācijas modelis MQTT savienojumam	43
3.2.3	Klase “MqttWorker”	45
3.2.4	Ziņojumu plūsma klasēs “ProcessingService” un “SensorService”	47
3.2.5	Klase “InactiveSensorCache”	48
3.2.6	Klase “InactiveSensorCacheWorker”	48
3.3	Komponente “Sensor.WebApi”	49
3.3.1	Sensoru galapunkti “SensorController”	49
3.3.2	Paziņojumu galapunkti “NotificationsController”	52
3.3.3	Sensoru mērījumu monitorings ar “SensorHub”	54
3.3.4	Klase “SensorHub”	55
3.4	Komponente “Sensor.Dashboard”	56
3.4.1	Komponente “TableComponent”	58
3.4.2	Funkcionalitāte “sensor-list”	59
3.4.3	Funkcionalitāte “sensor-details”	60
3.4.4	Funkcionalitāte “rule-create”	64
3.4.5	Funkcionalitāte “rule-details”	65
3.5	Komponente “Sensor.Courier”	67
3.5.1	SQL datubāzes projektējums	68

3.5.2	Konfigurācijas modelis datu ekstrakcijai	70
3.5.3	Darbības konfigurācija	70
3.5.4	Sensoru datu ekstrakcija, transformācija un ielāde	73
3.6	Komponente “Sensor.Notifications”	75
3.6.1	Klase “MeasurementWatcher”	75
3.6.2	Klase “NotificationService”	76
3.6.3	Dublētu (atkārtotu) paziņojumu novēršana	78
3.6.4	Saistītie datu modeļi	79
3.7	Nefunkcionālo prasību projektējums	81
3.8	Tīmekļa lietotnes saskarņu apraksts	83
3.8.1	Sensoru saraksta skats	83
3.8.2	Sensora skats	84
3.8.3	Paziņojumu saraksta skats	85
3.8.4	Paziņojumu noteikumu saraksta skats	85
3.8.5	Paziņojumu noteikuma skats	86
3.8.6	Jauna paziņojumu noteikuma skats	86
4	Testēšanas dokumentācija	87
4.1	Funkcijas “Pieslēgties MQTT brokerim” testēšana	87
4.2	Funkcijas “Klausīties sensorus” testēšana	88
4.3	Funkcijas “Apstrādāt ziņojumu” testēšana	90
4.4	Funkcijas “Iegūt sensoru sarakstu” testēšana	93
4.5	Funkcijas “Apskatīt sensoru” testēšana	93
4.6	Funkcijas “Sensoru mērījumu monitorings” testēšana	94
4.7	Funkcijas “Rediģēt sensora atrašanās vietu” testēšana	95
4.8	Funkcijas “Atslēgt/pieslēgt sensora uztveršanu” testēšana	95
4.9	Funkcijas “Eksportēt mērījumus” testēšana	96
4.10	Funkcijas “Eksportēt metadatus” testēšana	97
4.11	Funkcijas “Atjaunot parametrus” testēšana	98

4.12	Funkcijas “Pārvaldīt noteikumus” testēšana	99
4.13	Funkcijas “Ģenerēt paziņojumus” testēšana.....	100
5	Projekta organizācija	102
5.1	Konfigurācijas pārvaldība.....	103
5.2	Kvalitātes nodrošināšana	104
5.3	Darbietilpības novērtējums.....	104
	Secinājumi.....	106
	Izmantotā literatūra un avoti	107
	Pielikumi	110
1.	pielikums	110
2.	pielikums	114
3.	pielikums	117
4.	pielikums	120
5.	pielikums	122

IEVADS

Nolūks

Dokumenta nolūks ir aprakstīt sensoru monitoringa sistēmas funkcionalitāti, prasības, projektējumu, testēšanas dokumentāciju, iekļaujot arī projekta organizāciju, konfigurācijas pārvaldību un kvalitātes nodrošināšanu.

Programmatūras prasību specifikācija ir paredzēta projektējuma un sistēmas izstrādei, definējot sistēmas funkcionālās prasības. Programmatūras projektējuma apraksts apraksta sistēmas arhitektūru, komponentes, klases un darbības procesus atbilstoši izstrādātajai programmatūrai.

Darbības sfēra

"Sensoru monitoringa sistēma" ir izstrādāta, lai pārvaldītu Raiņa bulvārī 19 izvietoto Aranet sensoru datus. Sistēma uztver sensoru mērījumus, izmantojot MQTT brokeri, un saglabā tos datubāzē. Tā nodrošina reālā laika datu attēlošanu, paziņojumu funkcionalitāti un datu eksportu uz relāciju datubāzēm analīzes nolūkiem. Sistēmas primārais mērķis ir nodrošināt efektīvu risinājumu izvietoto Aranet sensoru pārvaldībai, vienlaikus nodrošinot iespēju paplašināt tās darbību citu sensoru atbalstam.

Saistība ar citiem dokumentiem

Dokumenta veidošanā ir izmantoti standarti: LVS 68:1996 "Programmatūras prasību specifikācijas ceļvedis" un LVS 72:1996 "Ieteicamā prakse programmatūras projektējuma aprakstīšanai".

Pārskats

Dokuments sastāv no šādām daļām:

1. Vispārīgais apraksts
2. Programmatūras prasību specifikācija
3. Programmatūras projektējuma apraksts
4. Programmatūras testēšanas dokumentācija
5. Projekta organizācija

APZĪMĒJUMU SARAKSTS

.NET / C# - Microsoft izstrādāta platforma lietojumprogrammu veidošanai, un C# ir galvenā programmēšanas valoda .NET ietvaros.

Angular – Angular ir ietvars tīmekļa lietotņu izstrādei, kas izmanto TypeScript.

Aranet – Aranet ir bezvadu sensoru risinājums, ko izmanto gaisa kvalitātes (piemēram, temperatūras, mitruma, CO2) mērīšanai.

Atkarību injekcija (Dependency Injection) – Ļauj pārvaldīt klases atkarības ārpus pašas klases. Nepieciešamās klases tiek “ievadītas” konstruktora parametrā, nevis radīti pašā klasē.

Change Streams – MongoDB iespēja, kas ļauj “klausīties” izmaiņas noteiktā kolekcijā (piemēram, ja tiek pievienoti jauni dati).

Docker – Docker ir konteinerizācijas platforma, kas ļauj vienkāršot programmatūras izstrādi un izvietojumu, apvienojot lietotni ar visām tās atkarībām izolētā vidē (konteinerī).

Docker Compose – Docker Compose ir rīks, kas ļauj definēt un pārvaldīt vairākus Docker konteinerus, izmantojot vienu “compose.yaml” konfigurācijas failu.

Docker konteineris – Docker konteineris ir izolēta vide, kurā darbojas konkrēts serviss. Konteineris satur visu nepieciešamo programmas izpildei – no operētājsistēmas bibliotēkām līdz konkrētiem koda failiem.

MongoDB – MongoDB ir NoSQL datubāze, kas glabā datus JSON dokumentos un nodrošina elastīgas datu struktūras.

Mosquitto – Mosquitto ir atvērtā koda MQTT brokeris, kas īsteno MQTT protokolu.

MQTT (Message Queuing Telemetry Transport) – Viegls ziņojumu protokols, kas paredzēts IoT ierīcēm un sistēmām. Tas balstās uz “publicēšanas-abonēšanas” mehānismu, kur klients publicē datus konkrētā tēmā (topic), bet citi klienti var šos datus saņemt, ja tie ir abonējuši attiecīgo tēmu.

MQTT brokeris – MQTT brokeris ir starpnieks, kas saņem ziņas no publicētājiem (piemēram, sensoriem) un nodod tās tālāk abonentiem.

SignalR – SignalR ir .NET bibliotēka, kas nodrošina reāllaika divvirzienu komunikāciju starp serveri un klientu.

Worker (BackgroundService) – .NET funkcionalitāte ilgi dzīvojošu fona procesu realizēšanai.

1 VISPĀRĪGAIS APSKATS

1.1 Esošā stāvokļa apraksts

Pašreiz Latvijas Universitātēs Raiņa bulvāra 19 ēkā ir izvietoti Aranet [1] sensori, bet to dati netiek vākti, glabāti un padarīti pieejami. Ēkā izvietoti Aranet sensori mēra gaisa kvalitāti kā temperatūru, mitrumu, co2 līmeni, ar iespēju apskatīt šos mērījumus uz sensora ekrāna. Šie sensori ir spējīgi padarīt savus datus pieejamus ar MQTT [2] protokolu izmantojot Raiņa bulvārī 19 izvietoto Aranet bāzes staciju. Mērķis ir izstrādāt sistēmu, kas spētu saņemt sensoru mērījumus no bāzes stacijas, glabāt tos datubāzē un padarīt pieejamus gan reālā laika apskatei, gan datu analīzei, nodrošinot datu eksporta iespējas.

Iepriekš eksistēja sistēma, kas veica sensoru datu vākšanu, taču jau 2 gadus, kopš pārtraukta mākoņpakalpojumu izmantošana, šie dati vairs netiek vākti un izmantoti. Tādēļ tagad ir iespēja izveidot jaunu sistēmu.

1.2 Pasūtītājs

Sistēma aprakstīta un realizēta pēc autora iniciatīvas kvalifikācijas darba ietvaros.

1.3 Produkta perspektīva

“Sensoru monitoringa sistēma” ir neatkarīga, bet modulāra tādā ziņā ka tā var darboties kā neatkarīgs sensoru monitoringa risinājums, bet var arī būt kā pamats lielākā sistēmā. Sistēmas mērķis ir nodrošināt Raiņa bulvārī 19 izvietotajiem Aranet sensoriem monitoringa risinājumu, kas ir spējīgs uztvert datus caur MQTT protokolu no Aranet bāzes stacijas un saglabāt saņemtos sensoru datus datubāzē. Papildus sistēma nodrošinās sensoru mērījumu reālā laika apskati, sensoru pārvaldību, paziņojumus un sensoru datu eksportu uz relāciju datubāzi izolētai analīzei.

1.4 Darījumprasības

Sensoru datu uztveršana:

1. Sistēmai jāspēj uztvert un saglabāt Raiņa bulvāra 19 izvietotos Aranet sensorus.
2. Sistēmai jāspēj pieslēgties MQTT brokerim, lai uztvertu sensoru ziņojumus.

3. Sistēmai jāspēj apstrādāt saņemtos sensoru datus un saglabāt tos datubāzē.
4. Sistēma jāspēj abonēt strukturētas MQTT tēmas, kas ietver sensora identifikatorus un atšķir mērījumu un metadatu ziņojumus
5. Sistēmai jāspēj pieņemt un saglabāt ne tikai zināmos Aranet sensoru ziņojumu laukus.

Sensoru panelis:

6. Iespēja apskatīt ienākošos sensora datus.
7. Iespēja meklēt sensorus sensoru sarakstā.
8. Iespēja apskatīt individuāla sensora reālā laika mērījumus.

Sensoru pārvaldība:

9. Iespēja pieslēgt vai atslēgt konkrēta sensora datu uztveršanu.
10. Iespēja pievienot un atjaunot sensora atrašanās vietu.

Datu eksports:

11. Sistēmai jānodrošina iespēja veikt datu eksportu uz relāciju datubāzi, lai atbalstītu izolētu datu analīzi un pārskatus.
12. Iespēja konfigurēt eksporta partijas lielumu un biežumu.

Paziņojumi:

13. Iespēja definēt paziņojumus sensoru mērījumiem, kas iestatīti pēc sliekšņa noteikumiem (piemēram, ja temperatūra $>30^{\circ}\text{C}$, tad izveidot paziņojumu)
14. Iespēja apskatīt paziņojumu sarakstu sensoru panelī.

1.5 Sistēmas lietotāji

Sistēmas lietotāji (Att. 1) ir administrators un datu analītiķis. Galvenais lietotājs ir tehniski kompetents administrators. Visas darbības, tostarp sistēmas palaišanu, konfigurāciju, pārvaldību un uzraudzību, veiks administrators.

Administrators ir atbildīgs par:

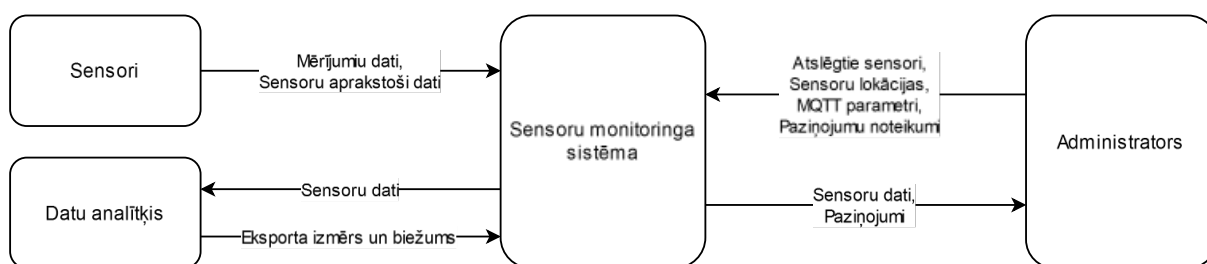
1. Sistēmas konfigurāciju, piemēram, MQTT tēmu precizēšana un ja nepieciešams sistēmas parametru iestatīšanu, kā MQTT brokera savienojumu un datubāzes savienojumu konfigurācija.
2. Sensoru lokācijas precizēšanu.

3. Konkrēta sensora atslēgšanu no sistēmas uztveršanas vai pieslēgšana pēc iepriekšējas atslēgšanas.

Papildus ir datu analītiķis, kas vēlas izgūt datus uz relāciju datubāzi, lai varētu veikt izolētu datu analīzi.

Analītiķa galvenās funkcijas ir:

1. Eksporta moduļa darbināšana.
2. Eksporta partijas izmēra precizēšana, piemēram, cik mērījumus nolasīt no sistēmas datu bāzes un ielādēt relāciju datubāzē vienā piegājienā.
3. Iestatīt eksporta partijas biežumu – aizkave starp partijām.



Att. 1 0. līmeņa datu plūsmas diagramma

1.6 Vispārējie ierobežojumi

Šī sistēma primāri tiek izstrādāta ar nodomu, lai tā spētu strādāt ar Latvijas Universitātes Raiņa bulvārī 19 izvietotajiem Aranet sensoriem un šo sensoru bāzes stacija ir uzstādīta lai sūtītu datus uz MQTT brokeri, tāpēc sistēma tiek veidota ap primāru sensoru datu saņemšanas saskarni - MQTT brokeris.

1.7 Pieņēmumi un atkarības

Pieņēmumi:

1. Aranet bāzes stacija ir saderīga ar MQTT protokolu un sūta sensora datus MQTT brokerim noteiktā formātā [3].
2. Raiņa bulvārī 19 ir izvietoti aptuveni 50 Aranet sensori.
3. Visi sensori nodrošina unikālus identifikatorus, kas tiek izmantoti datubāzē un sistēmas loģikā.
4. Sistēmas darbība ir pilnībā atkarīga no pieejamā MQTT brokera. Ja brokeris nav pieejams, sistēma nespēs uztvert datus.

2 PROGRAMMATŪRAS PRASĪBU SPECIFIKĀCIJA

2.1 Konceptuālais datu bāzes apraksts

Att. 2 ir attēlots datu bāzes konceptuālais modelis. Datubāzes galvenās entitātes ir “Sensori”, “Mērījumu ziņojumi”, un “Metadatu vēsture”. Tā ir veidota, lai aprakstītu kādi dati jā saglabā no Raiņa bulvāra 19 Aranet sensoriem. Entitāte “Mērījumu ziņojumi” un “Metadatu vēsture” nodrošina sensoru datu vēsturisko saglabāšanu.

Entitātes “Mērījumu ziņojumi” vairākvērtīgais atribūts “Mērījumi” parāda, ka datubāzē tiks glabāti mērījumi kā co2, mitrums, spiediens un akumulatora līmenis. Šie ir zināmie sensoru mērījumi. Modelī netiek parādīts tas ka ir jānodrošina iespēja saglabāt dažādus mērījumus, ne tikai zināmos.

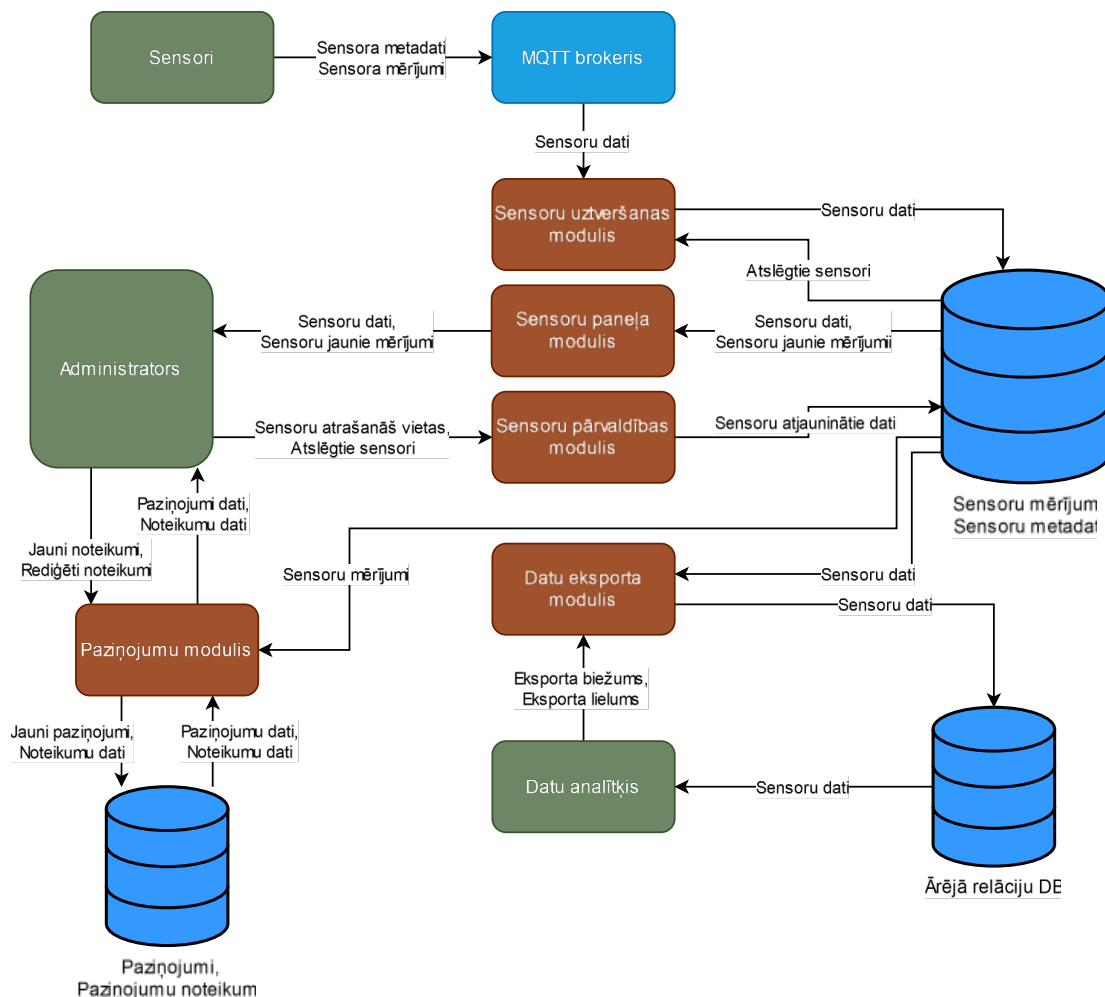
Entitāte “Sensors” ir aktuālie sensori dati, atšķirība no entitātes “Metadatu vēsture”, kas uztur vēsturiskos datus. Sensora atribūti kā “Nosaukums”, “Grupa”, “Produkta numurs” ir dati, kurus sūta pats sensors uz sistēmu, bet atribūti “Lokācija” un “Ir atspējots” ir dati kurus uzstāda administrators caur moduli “Sensoru panelis”. Daudzvērtīgais atribūts “Tēmas” norāda, ka ir nepieciešams saglabāt visas MQTT tēmas no kurām tiek saņemti konkrētā sensora ziņojumi, piemēram, vienam sensoram jā saglabā tēmas “Aranet/SNS12/json/measurements” un “Aranet /SNS12/name”. Sensora atribūti, kuru datu avots ir pats sensors (piemēram, nosaukums) ir balstīti uz dokumentētajiem Aranet sensoru ziņojumiem [3]. Tas pats attiecas uz mērījumiem (piemēram, co2), kas ir balstīti uz dokumentētajiem ziņojumiem [3], bet ir specificēti vēlāmie mērījumi.

2.2 Funkcionālās prasības

Sistēmas funkcionālās prasības tiek sadalītas šādos moduļos:

- Sensoru uztveršanas modulis – sensoru MQTT ziņojumu saņemšana un to datu glabāšana.
- Sensoru paneļa modulis – saņemto sensoru datu apskate.
- Sensoru pārvaldības modulis – lokāciju uzstādīšana un sensoru atslēgšana.
- Datu eksporta modulis – sensoru datu eksportēšana uz relāciju datubāzi.
- Paziņojumu modulis.

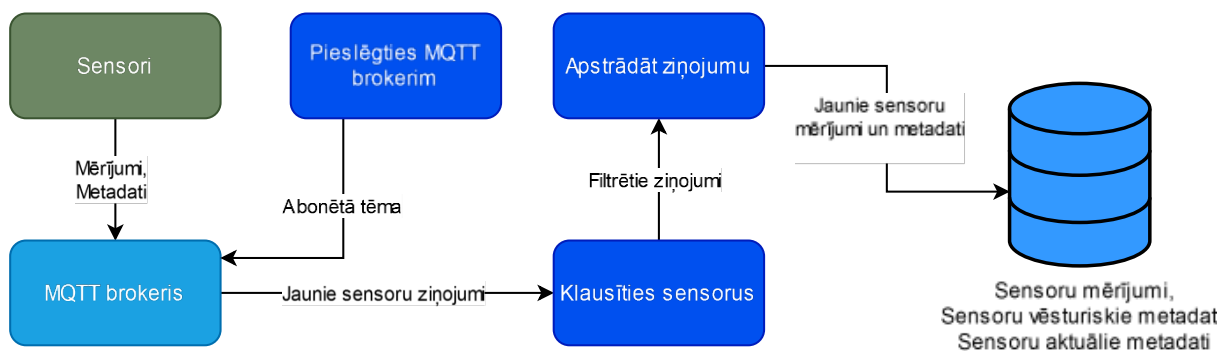
Att. 3 redzamajā diagrammā ir attēlota datu plūsma starp minētajiem moduļiem, datubāzes un lietotājiem.



Att. 3 1. līmeņa datu plūsmu diagramma

2.2.1 Sensoru uztveršanas modulis

Modulis “Sensoru uztveršana” nodrošina sistēmas spēju pieslēgties MQTT brokerim, klausīties konfigurētās tēmas un apstrādāt ienākošos ziņojumus reālā laikā. Tas apstrādā un saglabā Aranet sensoru metadatus un mērījumus. Modulis sastāv no funkcijām “Mqtt.Connect”, “Mqtt.Listen”, “Mqtt.HandleMessage” un to datu plūsmas ir redzamas Att. 4.



Att. 4 Sensoru uztveršanas moduļa 2. līmeņa datu plūsmu diagramma

Pieslēgties MQTT brokerim	
Identifikators	Mqtt.Connect
Mērķis	
Nodrošināt sistēmas savienojumu ar uztādīto MQTT brokeri, lai sāktu datu uztveršanu no konfigurētajām tēmām.	
Ievaddati	
- MQTT brokera parametri: - URL vai IP adrese un ports. - Autentifikācijas parametri (lietotājvārds un parole). - Drošības iestatījumi (piemēram, TLS, autorizācijas sertifikāts). - Citi savienojuma izveidošanai nepieciešamie parametri. - Abonējamās MQTT tēmas, piemēram, “Aranet/#” vai “Araner/123/sensors/+json/measurements”.	
Apstrāde	
- Izveidot savienojumu ar norādīto MQTT brokeri ar norādītajiem parametriem. - Abonē norādīto MQTT tēmu. - Validēt savienojumu, pārliecinoties, ka brokeris ir pieejams un parametri ir pareizi. - Uztur savienojumu ar MQTT brokeri un atjauno savienojumu, ja tas ir pārtrūcis.	
Izvaddati	

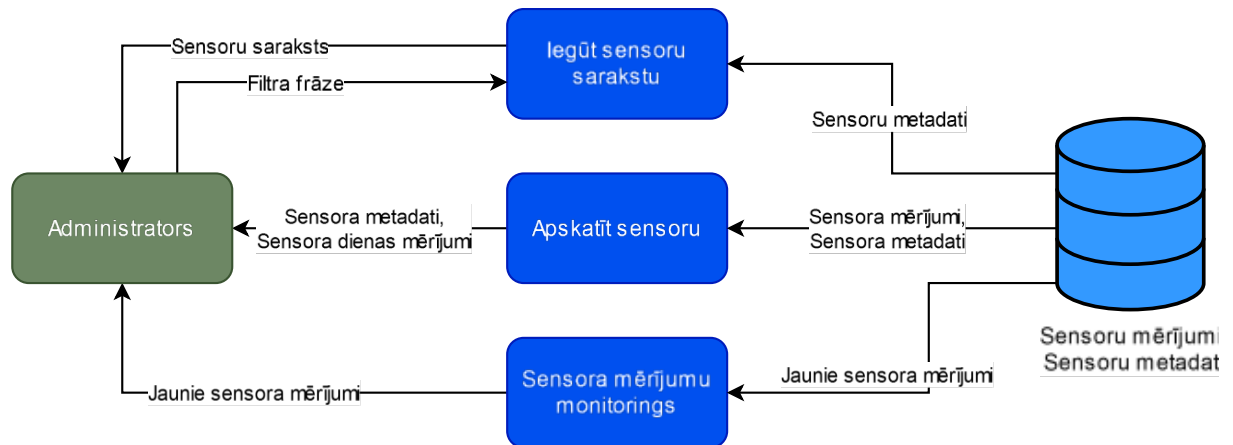
- Veiksmīgi izveidots savienojums ar MQTT brokeri.
- Kļūdas ziņojums ar iemesliem, ja savienojumu nevar izveidot (piemēram, nepareizi parametri vai brokeris nav pieejams).

Klausīties sensorus	
Identifikators	Mqtt.Listen
Mērķis	
Saņemt un filtrēt MQTT ziņojumus, nodrošinot ka tiek saņemti un saglabāti tikai to sensoru dati kuri nav atslēgti no sistēmas.	
Ievaddati	
1. Ienākošie ziņojumi. 2. Atslēgto sensoru saraksts.	
Apstrāde	
1. Saņemot ziņojumu pārbauda vai sensors ir atslēgts. a. Ja tas ir atslēgts, tad ziņojums tiek ignorēts. 2. Saņemto ziņojumu nodod ziņojumu apstrādes funkcijai “Mqtt.HandleMessage”.	
Izvaddati	
• Žurnālē ziņojuma saņemšanas faktu. • Žurnālē radušās kļūdas klausīšanās procesā.	

Apstrādāt ziņojumu	
Identifikators	Mqtt.HandleMessage
Mērķis	
Apstrādāt saņemtos sensora ziņojumus un saglabāt sensoru mērījumus un metadatus.	
Ievaddati	
- Ziņojuma tēma. - Aranet sensori sūta mērījumus uz tēmu “Aranet/<bāzes stacija>/sensors/<sensora id>/json/measurements”. - Un metadatus uz “Aranet/<bāzes stacija>/sensors/<sensora id>/+”, kur simbola “+” vietā būs “name”, “group”, “productNumber”. - Ziņojuma dati - Metadatu ziņojumu saturs ir teksta vērtība, piemēram, tēmas “Aranet/<bāzes stacija>/sensors/<sensora id>/name” ziņojums saturēs vērtību, piemēram, “Gaisa kvalitātes sensors”. - Mērījumu ziņojuma saturs ir JSON formāta, piemēram {“co2”: “200”, “temperature”: “20”, “time”: “1718294233”}.	
Apstrāde	
- Identificēt vai ziņojums satur sensora metadatus vai mērījumus. - Ja ziņojums ir mērījums: - Saglabā mērījumu datubāzē ar laika marķējumu no ziņojuma. - Ja ziņojums ir metadati: - Saglabā datubāzē kā sensora aktuālos datus. - Ja šis ir iepriekš nesaglabāts atribūts vai tā vērtība atšķiras no iepriekš saglabātās, tad pievieno datus sensoru metadatu vēsturē ar ģenerētu laika marķi. - Saglabā tēmu kā šī sensora tēmu.	
Izvaddati	
- Saglabāti mērījumi vai metadati datubāzē. - Kļūdas ziņojums, ja apstrāde vai saglabāšana neizdodas.	

2.2.2 Sensoru paneļa modulis

Att. 5 ir redzamas moduļa “Sensoru panelis” datu plūsmas. Modulis nodrošina iespēju apskatīt saglabāto sensoru datus. Tas sastāv no funkcijām “Sensor.List”, “Sensor.View” un “Sensor.Monitor”.



Att. 5 Sensoru paneļa moduļa 2. līmeņa datu plūsmu diagramma

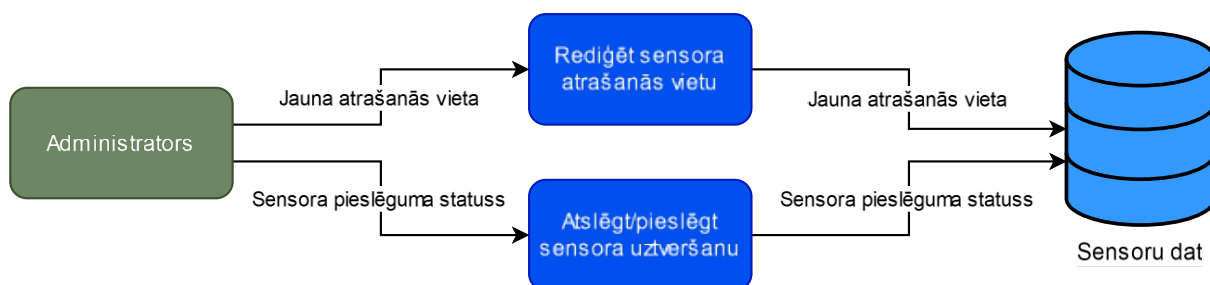
Iegūt sensoru sarakstu	
Identifikators	Sensor.List
Mērķis	
Nodrošināt skatu, kurā var redzēt visu sensoru sarakstu ar iespēju filtrēt.	
Ievaddati	
1. Filtra frāze (neobligāts).	
Apstrāde	
1. Ja ir norādīta filtra frāze, tad: a. Meklē un atgriež sensoru sarakstu, kuriem filtra frāze ir atrodama sensora metadatos. 2. Sistēma atgriež sarakstu ar sensoriem un to metadatiem.	
Izvaddati	
• Sensoru saraksts, kur katram sensora ir ar to saistītie dati: ○ Sensora identifikators ○ Atrašanās vieta ○ Nosaukums ○ Vai ir atslēgts ○ Citi metadati ○ Pēdējie mērījumi	

Apskatīt sensoru	
Identifikators	Sensor.View
Mērķis	
Nodrošināt skatu, kurā ir redzami konkrētā sensora dati (metadati un mērījumi) ar iespēju rediģēt sensora datus – “Atrašanās vieta” un “Ir atslēgts”	
Ievaddati	
1. Sensora ID	
Apstrāde	
1. Sistēma atrod un parāda norādītā sensora metadatus un šīs dienas mērījumus. 2. Mērījumi tiek attēloti līnijdiagrammās, kur katra metrika ir savā diagrammā.	
Izvaddati	
• Pieprasītais sensors ar tā: ○ Datiem kā atrašanās vieta un vai ir atslēgts. ○ Metadatiem un šodienas mērījumiem. • Sensora mērījumi diagrammās.	

Sensora mērījumu monitorings	
Identifikators	Sensor.Monitor
Mērķis	
Nodrošināt “Sensor.View” skatā, mērījumi tiek papildināti ar jaunajiem sistēmā ienākošajiem šī sensora mērījumiem.	
Ievaddati	
1. Sensora ID	
Apstrāde	
1. Sistēma klausās priekš jaunajiem sensora mērījumiem. 2. Jaunos mērījumus apstrādā un pievieno skata diagrammās, neveicot pilnu lapas atsvaidzināšanu.	
Izvaddati	
• Jaunie sensora mērījumi.	

2.2.3 Sensoru pārvaldības modulis

Att. 6 ir redzama moduļa “Sensoru pārvaldība” datu plūsmu diagramma. Sensoru pārvaldības modulis nodrošina iespēju administratoram precizēt sensoru atrašanās vietu ar “Sensor.Edit” un atslēgt sensoru no uztveršanas ar “Sensor.Disable”.



Att. 6 Sensoru pārvaldības moduļa 2. līmeņa datu plūsmu diagramma.

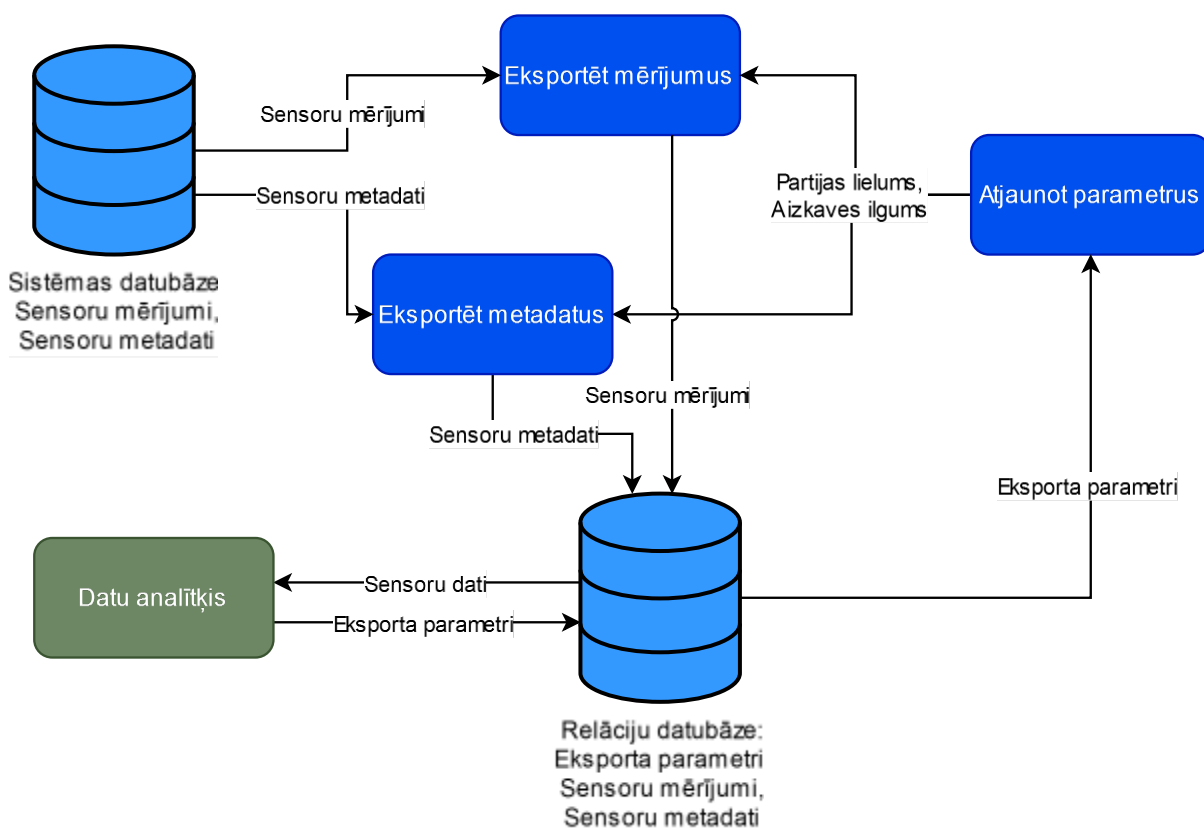
Rediģēt sensora atrašanās vietu	
Identifikators	Sensor.Edit
Mērķis	
Nodrošināt administratoram iespēju rediģēt sensora atrašanās vietu.	
Ievaddati	
1. Sensora ID 2. Jaunā atrašanās vieta	
Apstrāde	
1. Saglabā jauno atrašanās vietu. 2. Nodrošina ka vecās atrašanās vietas tiek saglabātas datubāzē ar laika marķējumu, kas atļauj identificēt kurā laika periodā sensors ir atradies.	
Izvaddati	
- Sensors ar jauno atrašanās vietu. - Vēsturē saglabāta sensora iepriekšējā atrašanās vieta.	

Atslēgt/pieslēgt sensora uztveršanu	
Identifikators	Sensor.Disable
Mērķis	
Nodrošināt iespēju atslēgt iepriekš uztvertu sensoru no sistēmas, lai tas netiktu vairs uztverts, ar iespēju pieslēgt to atpakaļ uztveršanai.	
Ievaddati	
1. Sensora ID 2. Atslēgt vai pieslēgt	
Apstrāde	
1. Ja sensors tiek uztverts, tad nodrošina ka sistēma to turpmāk neuztvers. 2. Ja sensors ir atslēgts, tad ir iespēja to pieslēgt atpakaļ.	
Izvaddati	
• Sensora pieslēguma statuss (atslēgts vai pieslēgts).	

2.2.4 Datu eksporta modulis

Att. 7 ir redzama moduļa “Datu eksports” datu plūsmu diagramma. Modulis nodrošina sistēmas spēju eksportēt sensoru datus no sistēmas datubāzes uz relāciju datubāzi, lai atbalstītu sensoru datu analīzi un pārskatu izolētā vidē.

Moduļa funkcijas ir “Export.Measurements”, “Export.Metadata”, “Export.LoadParameters” un tās kopā nodrošina ka datu eksports notiek periodiski ar partijām, nodrošinot vēsturisku un jaunu mērījumu eksportu. Aizkave starp partijām un to izmērs ir konfigurējams caur datubāzi.



Att. 7 Datu eksporta moduļa 2. līmeņa datu plūsmu diagramma

Eksportēt mērījumus	
Identifikators	Export.Measurements
Mērķis	
Atlasīt sensoru mērījumus no datubāzes un saglabāt tos datubāzē.	
Ievaddati	
1. Pēdējā saglabātā mērījuma datums. 2. Partijas izmērs. 3. Aizkave sekundēs līdz nākamajam eksportam.	
Apstrāde	
1. Atlasa nākamos x sensoru mērījumus pēc pēdējā saglabātā mērījuma datuma, kur x ir iestatītais partijas izmērs. 2. Atrastos mērījumus saglabā mērķa relāciju datubāzē. 3. Gaida x sekundes un atkārt, kur x ir iestatītā aizkave sekundēs.	
Izvaddati	
• Sensoru mērījumu relāciju datubāzē. • Žurnālē veiksmīgi izpildītas partijas faktu. • Žurnālē radušās kļūdas.	

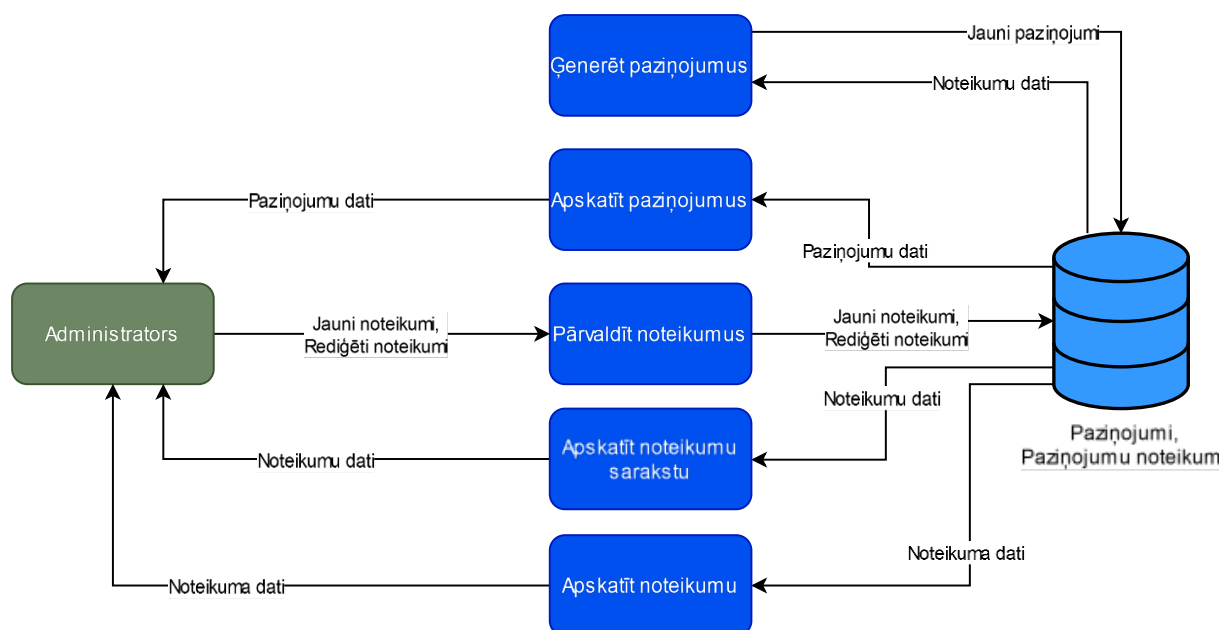
Eksportēt metadatus	
Identifikators	Export.Metadata
Mērķis	
Atlasīt sensoru metadatu vēsturi no datubāzes un saglabāt tos mērķa relāciju datubāzē.	
Ievaddati	
1. Pēdējā saglabātā metadatu datums. 2. Partijas izmērs. 3. Aizkave sekundēs līdz nākamajam eksportam.	
Apstrāde	
1. Atlasa nākamos x sensoru mērījumus pēc pēdējā saglabātā metadatu datuma, kur x ir iestatītais partijas izmērs. 2. Atrastos mērījumus saglabā mērķa relāciju datubāzē. 3. Gaida x sekundes un atkārt, kur x ir iestatītā aizkave sekundēs.	
Izvaddati	
• Sensoru metadati relāciju datubāzē. • Žurnālē veiksmīgi izpildītas partijas faktu. • Žurnālē radušās kļūdas.	

Atjaunot parameterus	
Identifikators	Export.Parameters
Mērķis	
Funkcijas mērķis ir nodrošināt, ka funkcijas “Export.Metadata” un “Export.Measurements” izmantos aktuālos eksporta parametrus (partijas izmērs un aizkave), kurus konfigurē datu analītiķis caur datubāzi.	
Ievaddati	
1. Jaunais partijas izmērs. 2. Jaunā aizkave starp partijām,	
Apstrāde	
1. Pirms sistēma veic eksporta funkciju, nodrošina, ka sistēma izmantos jaunākos parametrus no datubāzes.	
Izvaddati	
• Eksporta parametrs – partijas izmērs. • Eksporta parametrs – aizkave starp partijām.	

2.2.5 Paziņojumu modulis

Att. 8 ir redzama moduļa “Paziņojumi” datu plūsmu diagramma. Modulis nodrošina sistēmas spēju nosūtīt paziņojumus, balstoties uz lietotāja definētiem noteikumiem. Modulis ļauj lietotājiem konfigurēt sliekšņa kritērijus mērījumiem, kas aktivizē paziņojumu ģenerēšanu.

Modulis satur funkcijas “Notifications.Rules.CUD”, “Notifications.Generate”, “Notifications.List”, “Notifications.Rules.List”, “Notifications.Rules.View”.



Att. 8 Paziņojumu moduļa 2. līmeņa datu plūsmu diagramma

Pārvaldīt noteikumus	
Identifikators	Notifications.Rule.CUD
Mērķis	
Ļaut administratoram definēt paziņojumu noteikumus priekš skaitliskiem mērījumiem.	
Ievaddati	
Paziņojumu noteikums, kas nosaka: 1. Interesētā sensora ID (neobligāts – noteikums būs piemērojams visiem sensoriem). 2. Kritērijs, piemēram “Temperatūra > 30°C” vai “Baterija < 10%”. 3. Darbība (izveidot, rediģēt vai dzēst).	
Apstrāde	
1. Ja darbība ir dzēst, tad paziņojumu noteikums tiek dzēsts. 2. Ja darbība ir izveidot, tad tiek izveidots jauns paziņojumu noteikums 3. Ja darbība ir rediģēt, tad tiek atjaunots konkrētais paziņojumu noteikums.	

Ģenerēt paziņojumus	
Identifikators	Notifications.Generate
Mērķis	
Ģenerēt paziņojumus atbilstoši definētajiem noteikumiem.	
Ievaddati	
1. Paziņojumu noteikumi. 2. Ienākošie sensoru mērījumi.	
Apstrāde	
1. Analizēt ienākošos sensoru mērījumus vai tie atbilst kādam noteikumam. 2. Ja mērījums atbilst kādam noteikumam, tad ģenerē jaunu paziņojumu. 3. Nodrošināt, ka netiek sūtīti atkārtoti paziņojumi par to pašu notikumu. a. Piemēram, tiek ģenerēts paziņojums pēc kritērija “co2 > 2000”, jo ienākošajos mērījumos tika novērots “co2” ar vērtību “2050”. Ja ienāk nākamais “co2” mērījums ar vērtību “2100”, nav nepieciešams ģenerēt paziņojumu. b. Paziņojumi, tiek ģenerēti tikai tad, ja pēc kritērija sasniegšanas, kritērijs tiek nesasniegts un tikai tad atkal sasniegts.	
Izvaddati	
• Paziņojumi, kur katram ir: ○ Sensora ID ○ Ziņa, kas satur mērījumu ar pārkāpto kritēriju ○ Paziņojuma laiks ○ Saistītais noteikums	

Apskatīt paziņojumus	
Identifikators	Notifications.List
Mērķis	
Nodrošināt skatu, kurā var redzēt visus paziņojumus secībā sākot ar jaunāko.	
Izvaddati	
• Paziņojumu saraksts sākot ar jaunāko.	

Apskatīt noteikumu sarakstu	
Identifikators	Notifications.Rules.List
Mērķis	
Nodrošināt skatu, kurā var redzēt visu paziņojumu noteikumu sarakstu.	
Ievaddati	
1. Filtra frāze (neobligāts).	
Apstrāde	
1. Tiek attēlots paziņojumu noteikumu saraksts. 2. Ja tiek norādīta filtra frāze, tad noteikumu saraksts satur tos noteikumus, kuru atribūtos ir atrodama filtra frāze.	
Izvaddati	
• Noteikumu saraksts.	

Apskatīt noteikumu	
Identifikators	Notifications.Rules.View
Mērķis	
Nodrošināt skatu, kurā ir redzams konkrētais paziņojumu noteikums ar iespēju rediģēt vai dzēst.	
Ievaddati	
1. Noteikuma ID	
Izvaddati	
• Pieprasītais paziņojumu noteikums ar tā datiem. • Iespēja rediģēt vai dzēst paziņojumu atbilstoši “Notifications.Rules.CUD”. • Ar noteikumu saistītie paziņojumi.	

2.3 Nefunkcionālo prasību specifikācija

1. Sistēma spēj apstrādāt Raiņa bulvāri 19 izvietotos sensorus (~50).
2. Sensoru panelī apskatāmā sensora mērījumi tiek papildināti līdz ko jauns mērījums tiek uztverts (ne ilgāk kā ~1 sekunde).
3. Modulis “Sensoru uztveršana” darbojas neatkarīgi no citiem moduļiem.
4. Sistēmai jānodrošina detalizēta žurnālierakstu veidošana, lai atvieglotu kļūdu diagnosticēšanu un problēmu avota identificēšanu.
5. Administratora saskarne (kuru definē skati moduļos “Sensoru pārvaldība”, “Sensoru panelis”, “Paziņojumi”) jāaizsargā pret nepiederošu personu piekļuvi.

3 PROGRAMMATŪRAS PROJEKTĒJUMA APRAKSTS

Ievads

Sensoru monitoringa sistēma ir veidota kā modulāra arhitektūra, kas sastāv no vairākām savstarpēji saistītām programmatūrām, kuras tiek uzturētas atsevišķos GitHub repozitorijos. Šāda sadalījuma mērķis ir nodrošināt nesaistītu funkcionalitāšu nodalīšanu.

Sistēmas moduļi komponentēs

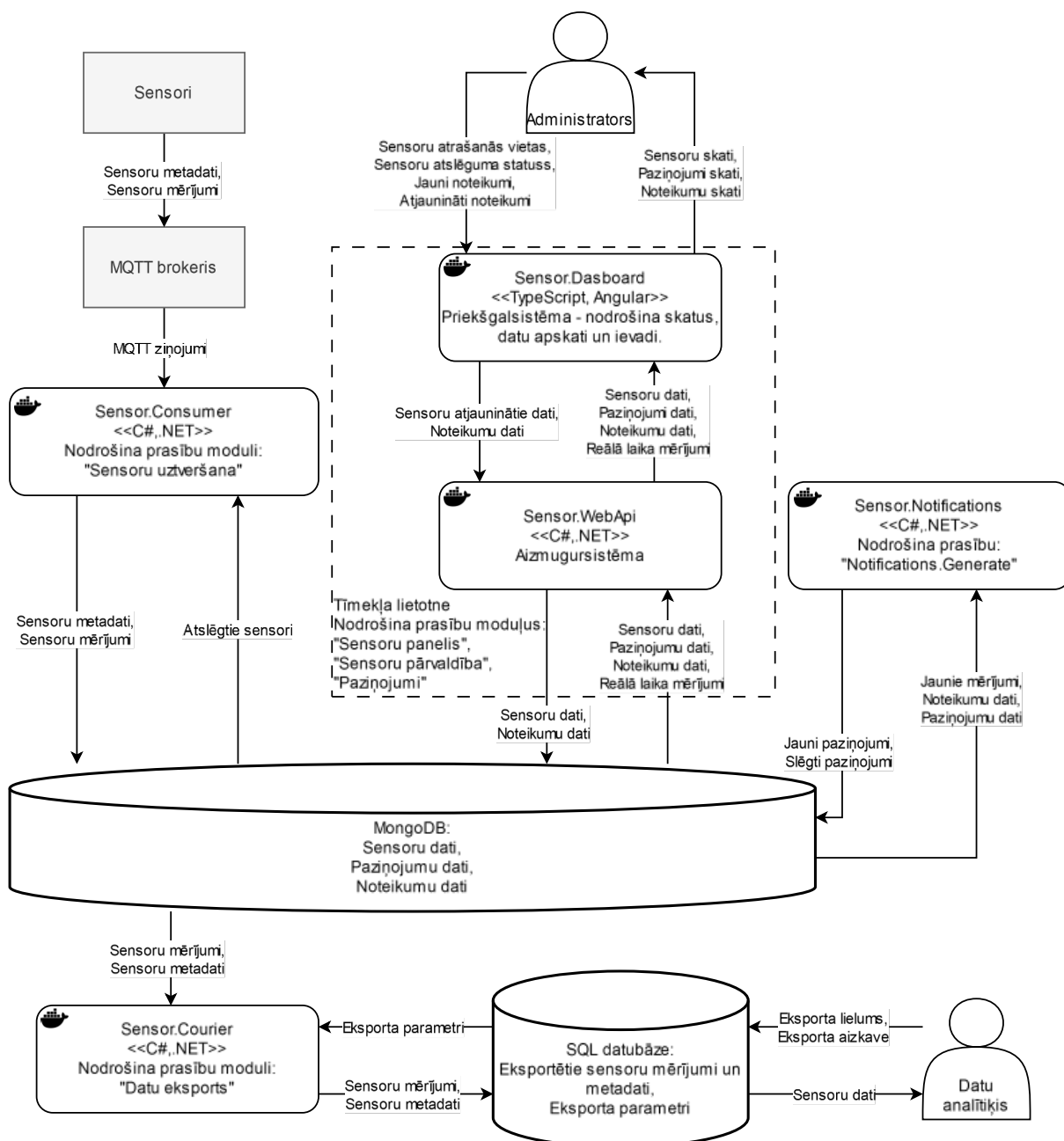
Sistēma tiek dalīta komponentēs (Att. 9 Sensoru monitoringa sistēmas komponentes un datu plūsmas), kur katra komponente ir piegādājama sistēmas daļa, kas atbilst vienam pirmkoda projektam. Katra komponente ir atsevišķi izpildāma kā Docker konteineris.

Prasību moduļi (Att. 3) tiek sadalīti starp projektējuma komponentēm (Att. 9), kur katra komponente tiek izstrādāta C# un .NET, izņemot tīmekļa lapas priekšgalsistēma, kas tiek izstrādāta Angular.

Sistēma sastāv no sekojošajām komponentēm:

1. Sensoru patērētājs (Sensor.Consumer).
 - i. Atbilstošās prasības:
 - i. Sensoru uztveršanas modulis.
 - ii. Realizē savienojumu ar MQTT brokeri un sensoru datu saglabāšanu MongoDB datubāzē.
2. Tīmekļa lapa (Sensor.Dashboard un Sensor.WebApi).
 - i. Atbilstošās prasības:
 - i. Sensoru paneļa modulis.
 - ii. Sensoru pārvaldības modulis.
 - iii. Paziņojumu modulis (izņemot "Notifications.Generate").
 - ii. Sensor.WebApi – aizmugursistēma tīmekļa lapai.
 - iii. Sensor.Dashboard – priekšgalsistēma tīmekļa lapai.
 - iv. Realizē uztverto sensoru datu apskati, paziņojumu pārvaldību, sensoru pārvaldību, ieskaitot reālā laika sensoru mērījumus ar SignalR bibliotēku.
3. Paziņojumu ģenerators (Sensor.Notifications).
 - i. Atbilstošās prasības:
 - i. Paziņojumu modulis: "Notifications.Generate".

- ii. Vēro datubāzē ienākošos sensoru mērījumus, lai ražotu paziņojumus atbilstoši paziņojumu noteikumiem.
4. Datu ekstraktors (Sensor.Courier).
 - i. Atbilstošās prasības:
 - i. Datu eksporta modulis
 - ii. Realizē sensoru datu eksportu no sistēmas MongoDB datubāzes uz SQL relāciju datubāzēm (šobrīd SqlServer un MySql).



Att. 9 Sensoru monitoringa sistēmas komponentes un datu plūsmas

Tehniskā infrastruktūra

Sistēma tiek uzstādīta Latvijas Universitātes serverī, kur katra iepriekš aprakstītā komponente dzīvo savā Docker konteinerī. Sensor.Dashboard dzīvo Ngninx [4] Docker konteinerī. Šī modularitāte atļauj darbināt komponentes neatkarīgi no citas, darbinot tikai atsevišķas komponentes vai pat aizvietojo konkrētas komponentes ar pavisam citiem izstrādātiem projektiem.

Papildus, tiek uzstādīts Mosquitto [5] MQTT brokeris un MongoDB [6] datubāze. Ar Mosquitto [5] MQTT brokeri savienojas sensoru bāzes stacija un sensoru patērētājs (Sensor.Consumer). Bāzes stacija sūta MQTT brokerim sensoru mērījumus un citus datus, kurus uztver patērētājs, kas saglabā sensoru datus MongoDB datubāzē.

Docker konteineri

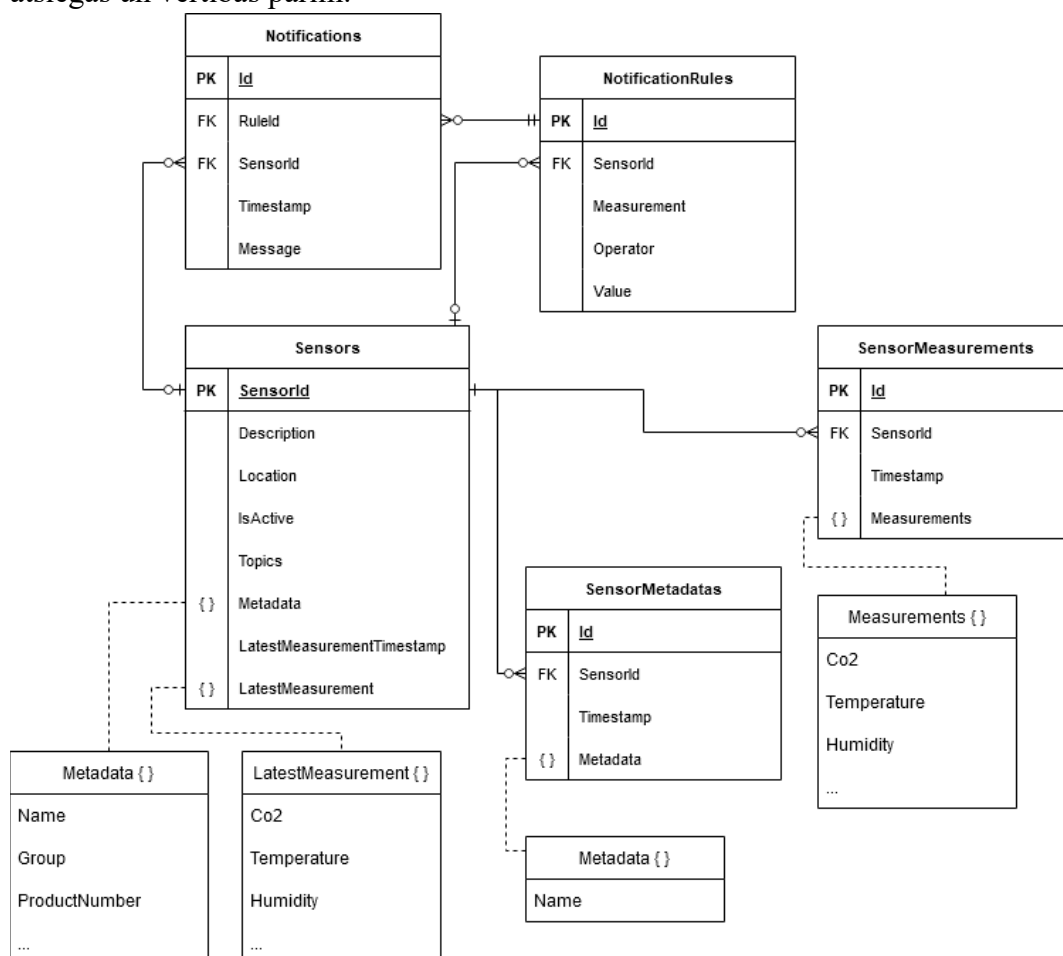
Katra projekta komponente (Att. 9) tiek darbināta atsevišķā Docker konteinerī, kur katrai daļai ir savs Docker Compose [7] fails. Konteinerizācija ar docker un compose failiem tiek izvēlēta lai nodrošinātu vieglu sistēmas palaišanu dažādās vidēs. Tā arī nodrošina ērtu sistēmas konfigurāciju lokālajai un produkcijas videi, piemēram, patērētāja MQTT un datubāzes savienojumu parametru konfigurēšana.

3.1 Datubāzes projektējums

Sistēmas datubāze ir MongoDB. Tā tiek izvēlēta, jo MongoDB ir labi optimizēts augstai caurlaidei, kas ir svarīgi sensoru sistēmām. MongoDB NoSQL pieeja ļauj elastīgi saglabāt dažāda formāta un struktūras datus, kas ir būtiski, lai uzglabātu dažādus iepriekš nedefinētus mērījumus un metadatus no sensoriem. Svarīgs iemesls MongoDB izvēlē ir izmaiņu plūsmas (“Change Streams” [8]), kas ļauj reāllaikā reaģēt uz notikumiem datubāzē.

3.1.1 Loģiskais datubāzes modelis

Datubāzes loģiskais modelis ir modificēta vārnu kāju diagramma (Att. 10), lai parādītu ne tikai attiecības starp objektiem (kolekcijām), bet arī parādītu iegultos dokumentus, kas ir specifiski MongoDB. Lauks ar simbolu “{ }” reprezentē iegulto dokumentu, kura saturu norāda raustītā līnija. Iegulto dokumentu “Metadata { }”, “LatestMeasurement { }”, “Measurements { }” lauku nosaukumi ir informatīvi – to būtība ir būt dinamiskiem atslēgu un vērtību pāriem. Tajos pašos objektos ar “...” ir apzīmēts ka tie var saturēt dinamisku skaitu atslēgu un vērtību pārus. Kolekcijas “SensorMetadataas” iegultajam objektam “metadata” ir paredzēts būt tikai vienam atslēgas un vērtības pārim.



Att. 10 Datubāzes loģiskais modelis

3.1.2 Fiziskais datubāzes modelis

Fiziskais datubāzes modelis attēlo datubāzes saturošo kolekciju dokumentu laukus un to datu tipus. Datubāze sastāv no kolekcijām “sensor”, “sensorMeasurements”, “sensorMetadata”, “notificationRules”, “notifications”, “notificationRule”.

Datu tips “Object” šajā diagrammā apzīmē iegultu dokumentu. Kolekcijas “sensor” dokumenti satur iegultos dokumentus (apakšdokumentus) “metadata” un “latestMeasurement”, kas satur atslēgu un vērtību pārus. Arī kolekcijas “sensorMeasurements” dokumenti satur iegultu dokumentu – “measurements”. Iegultais dokuments “measurements” satur sensora ziņojuma mērījumus kā atslēgu un vērtību pārus. Kolekcija “sensorMetadata” mērķis ir izsekot metadatu izmaiņas sensoriem.

sensors	
_id	String
description	String
location	String
isActive	Bool
topics	Array<String>
metadata	Object
latestMeasurementTimestamp	Date
latestMeasurement	Object

sensorMeasurements	
_id	ObjectId
sensorId	String
timestamp	Date
measurements	Object

sensorMetadata	
_id	ObjectId
sensorId	String
timestamp	Date
metadata	Object

notificationRules	
_id	ObjectId
sensorId	String
measurement	String
operator	String
value	Double

notifications	
_id	ObjectId
ruleId	ObjectId
sensorId	String
timestamp	Date
message	String

Att. 11 Fiziskais datubāzes modelis

3.1.3 Kolekcija “sensors”

Kolekcija “sensor” satur dokumentus, kas raksturo sensorus. Sensoru identificē “sensorId”, kā avots ir pats sensors un tiek iegūts no MQTT tēmas. Piemēram, tēmā “Aranet/123/sensors/SNS12/json/measurements” sensora identifikators ir “SNS12”. Dokumenti arī satur divus iegultos dokumentus “metadata” un “latestMeasurement”, satur vairākus atslēgu un vērtību pārus.

Kolekcija “sensor”		
Lauks	Datu tips	Apraksts
_id	String, obligāts	Sensora identifikators, kas ir iegūts no MQTT tēmas, parsējot tā struktūru pēc iepriekš definētas shēmas.
description	String, neobligāts	Lietotāja dots apraksts sensoram.
location	String, neobligāts	Lietotāja dotā sensora atrašanās vieta.
isActive	Bool, neobligāts	Pasaka sistēmai vai ignorēt šī sensora MQTT ziņojumus. Definē lietotājs.
topics	Array<String>, neobligāts	Masīvs, kas satur visas MQTT tēmas ar kurām ir saņemti sensora dati.
metadata	Object, neobligāts	Dokuments, kas satur no sensora iegūtos metadatus. Piemēram {“name”: “nosaukums”, “group”: “2”}.
latestMeasurement Timestamp	Date, neobligāts	Pēdējā saņemtā mērījuma laiks.
latestMeasurement	Object, neobligāts	Dokuments, kas satur pēdējo sensora mērījumu.

3.1.4 Kolekcija “sensorMeasurements”

Viens MQTT ziņojums par sensora mērījumu atbilst vienam kolekcijas “sensorMeasurements” dokumentam. Lauks “measurements” atbilst MQTT ziņojumā esošajiem mērījumiem, kas ir atslēgu un vērtību pāri, piemēram {“co2”: “200”, “temperature”: “20”}.

Indeksētie lauki: timestamp, sensorId.

Kolekcija “sensorMeasurements”		
Lauks	Datu tips	Apraksts
_id	ObjectId, obligāts	Konkrētā dokumenta identifikators, automātiski ģenerēts.
timestamp	Date, obligāts	Mērījuma laika marka, kas iegūts no MQTT ziņojuma lauka “time”.
sensorId	String, obligāts	No MQTT ziņojuma iegūtais sensora identifikators. Atbilst kolekcijas “sensors” laukam “_id”.
measurements	Object, obligāts	Iegūtais dokuments, kas satur MQTT ziņojuma mērījumus.

3.1.5 Kolekcija “sensorMetadata”

Nodrošina kolekcijas “sensor” lauku “metadata”, “isActive”, “location” izmaiņu vēsturi.

Indeksētie lauki: timestamp, sensorId.

Kolekcija “sensorMetadata”		
Lauks	Datu tips	Apraksts
_id	ObjectId, obligāts	Konkrētā dokumenta identifikators, automātiski ģenerēts.
timestamp	Date, obligāts	Mērījuma laika marka, kas ģenerēta izmaiņu brīdī.
sensorId	String, obligāts	Atbilst kolekcijas “sensors” laukam “_id”.
metadata	Object, obligāts	Iegūtais dokuments, kas satur izmainītā lauka nosaukumu un vērtību, piemēram, {“name”: “nosaukums1”}.

3.1.6 Kolekcija “notificationRules”

Paziņojumu noteikumu definē kādiem novērotajiem mērījumiem kolekcijā “sensorMeasurements” tiek izveidoti paziņojumi.

Lauki “measurements”, “operator” un “value” kopā definē noteikuma aktivizācijas loģiku.

Indeksētie lauki: sensorId, measurement

Kolekcija “notificationRules”		
Lauks	Datu tips	Apraksts
_id	ObjectId, obligāts	Konkrētā dokumenta identifikators, automātiski ģenerēts.
sensorId	String, neobligāts	Noteikumam attiecošais sensors, atbilst kolekcijas “sensors” laukam “_id”.
measurement	String, obligāts	Mērījuma nosaukums. Atbilst kādai atslēgai kolekcijas “sensorMeasurements” iegultā objekta “measurements” atslēgu un vērtību pāros.
operator	String, obligāts	“<” vai “>”. Nosaka vai noteikumu aktivizē mazāka vai lielāka sensora mērījuma vērtība.
value	String, obligāts	Sliekšņa vērtība.

3.1.7 Kolekcija “notifications”

Paziņojumi tiek izveidoti atbilstoši novērotajiem mērījumiem “sensorMeasurements” un paziņojumu noteikumiem “notificationRules”.

Indeksētie lauki: sensorId, ruleId.

Kolekcija “notifications”		
Lauks	Datu tips	Apraksts
_id	ObjectId, obligāts	Konkrētā dokumenta identifikators, automātiski ģenerēts.
sensorId	String, obligāts	Paziņojumam atbilstošais sensors, atbilst kolekcijas “sensors” laukam “_id”.
ruleId	ObjectId, obligāts	Noteikums, kas veicināja paziņojuma izveidi, atbilst kolekcijas “notificationRules” laukam “_id”.
message	String, neobligāts	Informatīvs ziņojums, kas satur noteikuma nosaukumu un mērījumu, kas aktivizēja paziņojuma izveidi.

startTimestamp	Date, obligāts	Paziņojuma izveides datums, atbilst kādam mērījumam kolekcijā “sensorMeasurements” laukam “timestamp”.
endTimestamp	Date, neobligāts	Paziņojuma beigu datums. Tiek aizpildīts, kad tiek novērota neaktivizējoša mērījuma vērtība. Atbilst kādam mērījumam kolekcijā “sensorMeasurements” laukam “timestamp”.

3.1.8 Konfigurācijas modelis Mongo datubāzes savienojumam

Šādu modeli izmanto visas C#, .NET komponentes datubāzes savienojumam.

1. Connectionstring
 - a. Piemērs: “mongodb://localhost:27017”
 - b. Norāda izmantoto Mongo datubāzes serveri. Atbilstoši MongoDB savienojuma dokumentācijai [9].
2. DatabaseName
 - a. Piemērs: “sensor-dev”
 - b. Norāda izmantoto Mongo datubāzes nosaukumu.

Atbilstoši ‘appsettings.json’ izskatītos šādi:

```
{
  "MongoDbSettings": {
    "ConnectionString": "mongodb://localhost:27017/?directConnection=true",
    "DatabaseName": "sensor-dev"
  }
}
```

Bet vides mainīgo (environment variables) konfigurācija ar docker compose failiem izskatītos šādi:

```
environment:
  MongoDbSettings__ConnectionString: mongodb://localhost:27017/?directConnection=true
  MongoDbSettings__DatabaseName: sensor-dev
```


3.2 Komponente “Sensor.Consumer”

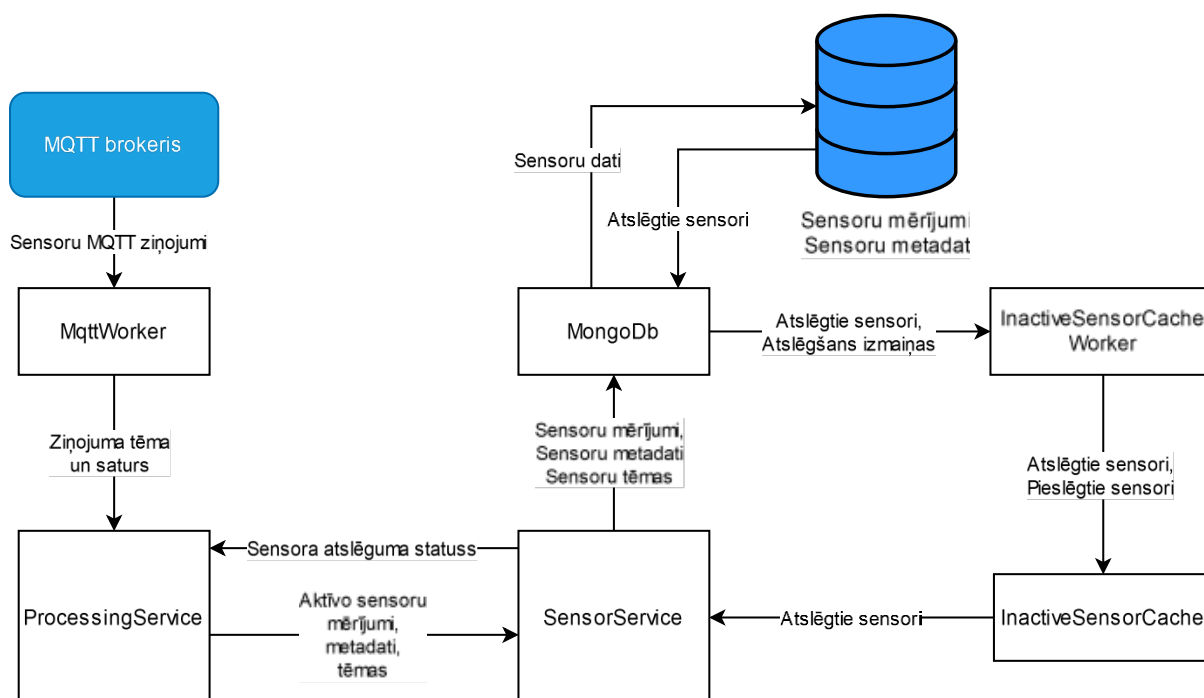
Komponente “Sensor.Consumer” sedz moduļa “Sensoru uztveršana” visas funkcionālās prasības: “Mqtt.Connect”, “Mqtt.Listen” un “Mqtt.HandleMessage”. Komponente veido un uztur savienojumu ar MQTT brokeri, klausās priekš sensoru ziņām un saglabā sensoru mērījumus un metadatus MongoDB datubāzē.

“Sensor.Consumer” klases iedalās sekojošos tipos:

1. Datu pieejas klases, kas nodrošina pieeju datiem.
 - a. Klase “MongoDb” nodrošina pieeju MongoDB datubāzes datiem.
 - b. Klase “InactiveSensorCache” nodrošina pieeju atmiņā saglabātajiem atslēgto sensoru identifikatoriem (pēc lauka “isActive”).
2. Modeļu klases, kas definē datubāzes datu modeļus.
 - a. Klase “Sensor” definē sensora datu modeli (kolekcija “sensor”).
 - b. Klase “SensorMeasurements” definē sensoru mērījumu datu modeli (kolekcija “sensorMeasurements”).
 - c. Klase “SensorMetadata” definē sensoru metadatu vēstures datu modeli (kolekcija “sensorMetadata”).
3. Servisu klases, kas nodrošina būtiskos apstrādes un datu glabāšanas procesus.
 - a. Klase “ProcessingService” nodrošina ziņojumu apstrādi.
 - i. Identificē ziņojuma tipu (mērījums vai metadati).
 - ii. Filtrē atslēgto sensoru ziņojumus.
 - iii. Izgūst no ziņojuma saglabājamās datus un saglabā tos ar klasi “SensorService”.
 - b. Klase “SensorService” nodrošina darbības ar klasēm “MongoDb” un “InactiveSensorCache”, lai nodrošinātu datu glabāšanas procesus.
 - i. Nodrošina mērījumu un metadatu glabāšanas procesus.
 - ii. Transformē datus datubāzei saglabājamā formātā atbilstoši modeļu klasēm.
4. Darbinieku (“Worker”) klases nodrošina ilgi dzīvojošos un fona procesus.
 - a. Klase “MqttWorker” kas nodrošina MQTT procesus.
 - i. Veido un uztur savienojumu ar MQTT brokeri.
 - ii. Uztver ziņojumus un katrai ziņai veido asinhronu apstrādes procesu, nododot to klasei “ProcessingService”.
5. Klase “Program” nodrošina programmas orķestrēšanu.

- a. Nodrošina sistēmas konfigurāciju.
 - b. Reģistrē sistēmas klases atkarību injekcijai (dependency injection [10]).
 - c. Palaiž galvenos procesus (darbinieku klases).
6. Konfigurācijas klases, kas nodrošina konfigurācijas datu modeļus.
- a. Klase “MongoDbSettings” nodrošina MongoDB datubāzes konfigurācijas modeli.
 - b. Klase “MqttSettings” nodrošina MQTT konfigurācijas modeli.
 - c. Klase “TopicSchema” nodrošina MQTT tēmu modeli.
 - d. Klase “MqttSettingsValidation” pārbauda klases “MqttSettings” konfigurētās tēmas un nodrošina lietojamus tēmu atšifrējumus ar klasi “TopicSchema”.

Diagrammā (Att. 12) tiek attēlotas datu plūsmas būtiskākajās komponentes klasēs.



Att. 12 Sensor.Consumer datu plūsmu diagramma

3.2.1 Atbalstītie MQTT ziņojumi

Sistēmas elastību papildina MongoDB datubāzes modeļu spēja pieņemt un saglabāt iepriekš nedefinētus datu laukus. Tas ļauj sistēmai uztvert dažādus ziņojumus un glabāt dažāda veida sensoru mērījumus un metadatus, atļaujot izmantot sistēmu ne tikai Aranet sensoriem kamēr tiek ievēroti sekojošie ierobežojumi.

Ierobežojumi:

1. Sensora identifikatoram jābūt tēmā.

2. Mērījuma ziņojumiem:
 - a. Datiem jābūt JSON formātā.
 - b. Jāsatur laika lauks ar nosaukumu “time”, kura vērtībā ir Unix laiks sekundēs.
3. Metadatu ziņojumiem:
 - a. Metadatu tēmā ir jābūt laukam, kas identificē metadatu nosaukumu.
 - b. Ziņojums ir konkrētā metadatu vērtība kā simbolu virkne.

3.2.1.1 MQTT tēmas

Sistēma saņem ziņojumus no divu veidu MQTT tēmām:

1. Mērījumu tēmas:
 - a. Konfigurētās tēmas piemērs: “Aranet/<sensorId>/json/measurements”.
 - b. Abonētā tēma: “Aranet/+ /json/measurements”.
 - c. Ienākoša ziņojuma tēmas piemērs: “Aranet/SNS12/json/measurements”.
2. Metadatu tēmas:
 - a. Identificē metadatus pēc tēmas struktūras un īpašā lauka “<metadataName>”.
 - b. Konfigurētās tēmas piemērs: “Aranet/<sensorId>/<metadataName>”.
 - c. Abonētā tēma: “Aranet/+ /+”.
 - d. Ienākoša ziņojuma tēmas piemērs: “Aranet/SNS12/name”.

Šādi tiek nodrošināta elastība tajā kādas tēmas var abonēt, kamēr tās:

1. Satur sensora identifikatora norādi tēmā.
2. Atsevišķas tēmas mērījumiem un metadatiem, kur metadatu tēma satur atribūta nosaukumu.

3.2.2 Konfigurācijas modelis MQTT savienojumam

Lai uztvertu sensoru datus, ir nepieciešams izveidot savienojumu ar MQTT brokeri. Tālāk tiek aprakstīts izmantotais konfigurācijas modelis MQTT savienojuma veidošanai. Šī modeļa parametri ir konfigurējami pirms programmas palaišanas failā “appsettings.json” vai ar sistēmas vides mainīgajiem (environment variables). Modeli izmanto klase “MqttWorker” priekš MQTT savienojuma.

1. Broker
 - Piemērs: “broker.hivemq.com”.
 - Norāda MQTT brokera adresi, ar kuru savienoties.
2. ProtocolVersion -
 - Piemērs: “V311”
 - Norāda izmantoto MQTT protokola versiju (piemēram, 3.1 vai 5).

3. Topics

- Piemērs:
“Aranet/<sensorId>/json/measurements,Aranet/<sensorId>/<metadataName>”
- MQTT tēmu saraksts, kuras jāabonē. Norādes (<sensorId>, <metadataName>) tiek izmantotas, lai identificētu sensorus un metadatu nosaukumu.

4. UseTls

- Piemērs: *true* vai *false*
- Norāda, vai izmantot transporta slāņa drošību (TLS).

5. SslProtocol

- Piemērs: “Tls12”
- Ja tiek izmantots TLS, šis parametrs norāda izmantoto SSL/TLS protokola versiju.

6. Username

- Lietotājvārds savienojumam ar nodrošinātu MQTT brokeri.

7. Password

- Parole savienojumam ar nodrošinātu MQTT brokeri.

8. PathToPem – piemēram, “C:\\certs\\ca.pem”.

- Ceļš uz PEM formāta sertifikāta failu, kas tiek izmantots drošam savienojumam.

3.2.2.1 Datu modelis “TopicSchema”

Klase “TopicSchema” ir datu modelis tēmas atšifrējumam un viens no MQTT konfigurācijas modeļa atribūtiem. Tas tiek izmantots klasē “MqttWorker”, lai abonētu tēmas. Papildus, modeli izmanto klase “ProcessingService”, lai noskaidrotu ienākošā ziņojuma tipu – mērījums vai metadati.

Sekojošajā tabulā redzams klases “TopicSchema” modelis un vērtību piemērs, ja konfigurētā tēma ir “Aranet/+/sensors/<sensorId>/<metadataName>”.

Datu modelis “TopicSchema”			
Nosaukums	Datu tips	Apraksts	Piemērs
TopicFilter	String	Abonētā tēma.	Aranet/+/sensors/+/+
TopicType	enum TopicType (Measurement vai Metadata)	Tēmas tips – mērījums vai metadati.	Metadata
SensorIdPosition	Int	Sensora identifikatora vieta tēmā, atbilstoši <sensorId>.	3
MetadataNamePosition	Int	Metadatu nosaukuma vieta tēmā, atbilstoši <metadataName>.	4

3.2.3 Klase "MqttWorker"

Nodrošina prasības: Mqtt.Connect, Mqtt.Listen, un daļēji Mqtt.HandleMessage.

Klase "MqttWorker" nodrošina MQTT savienojuma veidošanu, uzturēšanu un MQTT ziņu saņemšanu. "MqttWorker" sastāv no metodēm:

- Dzīvescikla vadībai: "StartAsync", "ExecuteAsync", "StopAsync".
- Ziņu apstrādei: "HandleReceivedMessage".
- Savienojuma pārvaldībai: "Connect", "SubscribeToTopics", "ConfigureOptions".

Klase manto īpašības no bibliotēkās "Microsoft.Extensions.Hosting" klases "BackgroundService" [11], kas nodrošina ilgi dzīvojoša procesa funkcionalitāti, piemēram, procesu korektu apturēšanu programmas apstādināšanas gadījumā. MQTT zema līmeņa funkcionalitāti nodrošina "IMqttClient" no bibliotēkas "MQTTnet" [12].

Galvenā funkcionalitāte:

1. Izveido savienojumu ar MQTT brokeri atbilstoši MQTT konfigurācijas modelim.
2. Uztur un atjauno savienojumu, pārbaudot ik pa 5 sekundēm vai savienojums joprojām ir dzīvs.
3. Uztur saņemto ziņu skaitu.
4. Izveido žurnālēšanas loku ar "ILogger.BeginScope", nodrošinot, ka katram ziņas apstrādes žurnālierakstam tiek piešķirts ziņas kārtas numurs.
 - a. "Message number {messageNumber}, Topic: {topic}"
5. Saņem un nodod ziņojumus asinhronai apstrādei "ProcessingService.ProcessMessageAsync".

Ziņas apstrāde notiek asinhroni, izmantojot Task.Run. Tas nodrošina vienlaicīgu vairāku ziņu apstrādi, kas ir būtiski lielas slodzes apstākļos.

Žurnālierakstu loks ar ziņas kārtas numuru ļauj atklūdošanas laikā viegli izsekot konkrētas ziņas apstrādes procesu un saistītos notikumus.

Žurnālēšana:

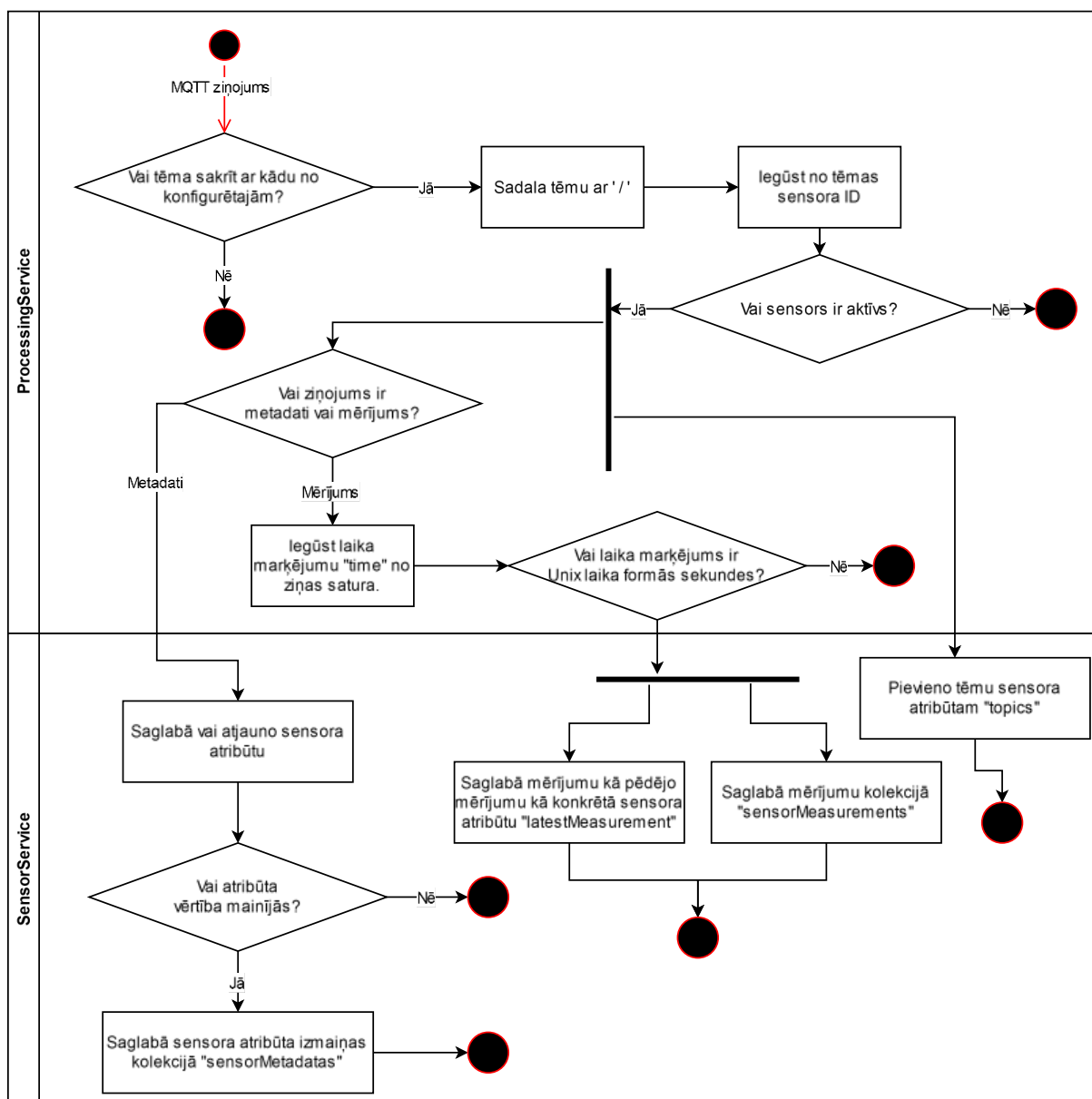
- Veic "LogLevel.Error" žurnālierakstus par:
 - Kļūdu savienojuma pārbaudē:
 - "An error occurred on ping attempt."
 - Ziņojumu apstrādē nenotikšajām kļūdām:
 - "Error processing MQTT message.", Exception

- Veic “LogLevel.Warning” žurnālierakstus par:
 - Neveiksmīgu savienojumu ar MQTT brokeri:
 - “An error occurred on connect attempt {retryCount}.”, Exception.
 - Pazuđētu savienojumu ar MQTT brokeri:
 - “MQTT connection is not alive. Attempting to reconnect.”
 - Trūkstošām abonementa tēmām:
 - “No topics to subscribe to.”
- Veic “LogLevel.Information” žurnālierakstus par:
 - Veiksmīgi izveidotu savienojumu ar MQTT brokeri, norādot brokeri.
 - “Connected to MQTT broker: {broker}.”
 - MQTT ziņojuma saņemšanu:
 - “Received message. Message count: {messageNumber}”
 - Veiksmīgu ziņas apstrādi:
 - “Processed message number {messageNumber}.”
 - Veiksmīgi abonētām tēmām:
 - “Subscribed to topic: {topic}”
- Veic “LogLevel.Debug” žurnālierakstus par:
 - Saņemtā MQTT ziņojuma saturu.
 - “ Message: {message};Payload: {payload}”

3.2.4 Ziņojumu plūsma klasēs “ProcessingService” un “SensorService”

Klases “ProcessingService” un “SensorService” nodrošina prasību Mqtt.HandleMessage.

Peldceliņu diagrammā (Att. 13) ir attēlots kā ziņas apstrādes uzdevumi tiek sadalīti starp klasēm. Klases “ProcessingService” uzdevums ir identificēt ziņojuma veidu (mērījums vai metadati), izgūt saglabājamās datus un nodot tos datiem klasei “SensorService”, kas veic datu saglabāšanu datubāzē.



Att. 13 Ziņojuma apstrādes peldceliņu diagramma

Ja nākotnē ir nepieciešams paplašināt MQTT ziņojumu apstrādi ar specifisku apstrādi konkrētām tēmām, tad soli pēc “Vai tēma sakrīt ar kādu no konfigurētajām?” var papildināt ar procesu kas identificē tēmu un nodod to citai apstrādei. Papildus, potenciālie papildinājumiem tiek nodrošinātas datu saglabāšanas funkcijas ar “SensorService”.

3.2.5 Klase "InactiveSensorCache"

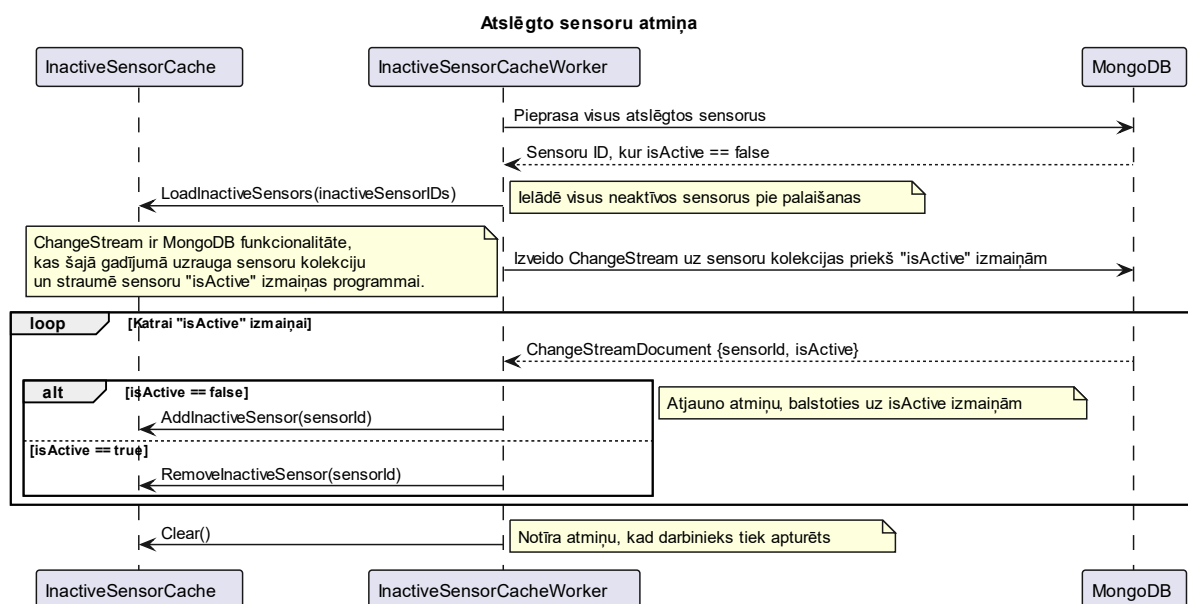
Klases "InactiveSensorCache" mērķis ir uzturēt atslēgto sensoru identifikatorus. Atslēgts sensors ir tāds, kura atribūts "isActive" ir "false". Pati klase nodrošina tikai vietu un pieeju atmiņai, izmantojot "ConcurrentDictionary<string, byte>", kas nodrošina drošu vārdnīcu vairāku procesu vienlaicīgai pieejai. Tiek izmantots "ConcurrentDictionary<string, byte>", kā alternatīva "HashSet<string>", jo "HashSet" nenodrošina drošu vienlaicīgu pieeju.

Klase nodrošina vienlaicīgi drošas atmiņas pieejas un modificēšanas metodes kā "AddInactiveSensor" un "RemoveInactiveSensor" identifikatoru pievienošanai un izņemšanai, "LoadInactiveSensors" masveida identifikatoru ielādēšanai un "IsSensorActive", lai pārbaudītu vai konkrēts sensors ir atslēgts.

3.2.6 Klase "InactiveSensorCacheWorker"

Klases mērķis ir pārvaldīt "InactiveSensorCache" modificējot sensoru identifikatoru atmiņu atbilstoši sensoru "isActive" atribūta izmaiņām. Lai nodrošinātu šādu funkcionalitāti tiek izmantots MongoDB "Change Streams" [8], kas nodrošina iespēju reaģēt uz "isActive" izmaiņām.

Diagrammā (Att. 14) tiek attēlots "InactiveSensorCache" dzīvescikls. Tas programmai uzsākoties aizpilda atmiņu ar atslēgto sensoru identifikatoriem. Pēc tam tiek izveidots ChangeStream uz kolekcijas "sensors" atribūtu "isActive" izmaiņām, kas informēs programmu katru reizi kad kāda sensora atribūts "isActive" tiek izmainīts. Atbilstoši izmaiņām tiek atjaunota atslēgto sensoru atmiņa. Tas kas netika parādīts diagrammā ir ka klase



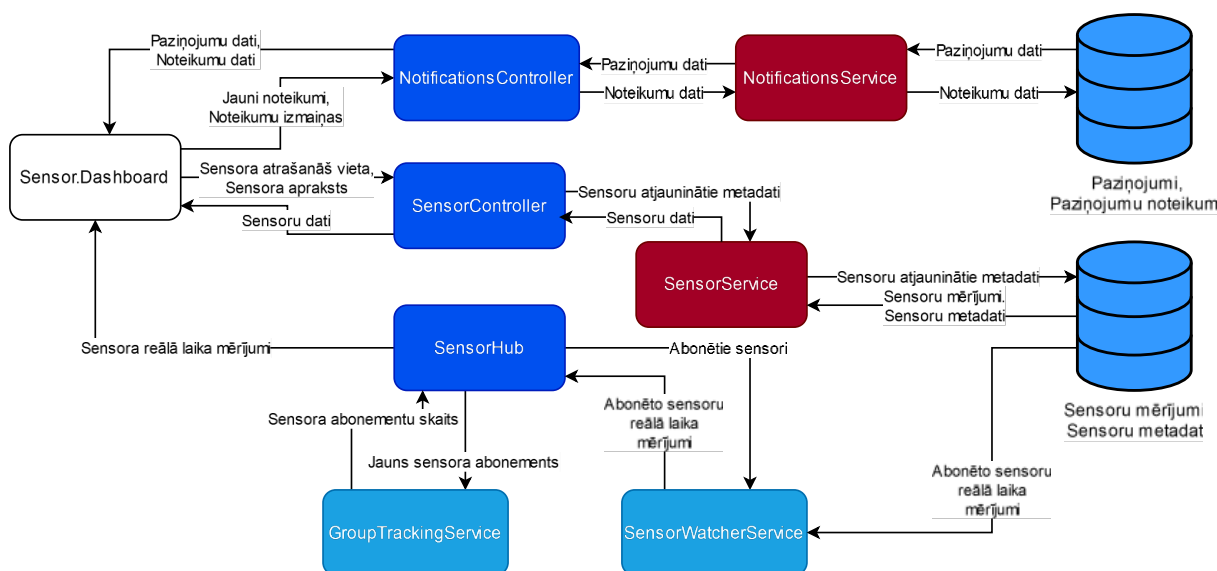
Att. 14 Atslēgto sensoru atmiņas sekvenču diagramma

“SensorService” jebkūrā brīdī var veikt vaicājumu “InactiveSensorCache” par to vai konkrēts sensors ir atslēgts ar metodi “IsSensorInactive”.

3.3 Komponente “Sensor.WebApi”

Komponente “Sensor.WebApi” nodrošina aizmugursistēmas daļu mājaslapai. Tā definē galapunktus ar kuriem iegūt datus par sensoriem un modificēt to metadatus, un galapunktus paziņojumu pārvaldībai.

Komponentes “Sensor.WebApi” datu plūsmu diagrammā (Att. 15) ir redzamas datu plūsmas no tīmekļa lapas vienīga lietotāja “Administrators” līdz datubāzei cauri komponentes klasēm.



Att. 15 "Sensor.WebApi" datu plūsmu diagramma

Zilā krāsā iezīmētie ir kontrolieri “NotificationsController”, “SensorController” un SignalR hubs “SensorHub”, kas definē pieejamos galapunktus. Sarkanajā krāsā ir iezīmētas servisu klases “NotificationsService” un “SensorService”, kas definē kā dati tiek saņemti un saglabāti. Gaiši zilā krāsā ir iekrāsotas “SensorHub” palīgklases “GroupTrackingService” un “SensorWatcherService”, kas kopā nodrošina sensora abonēšanu un mērījumu monitoringu.

3.3.1 Sensoru galapunkti “SensorController”

Sensoru galapunkti nodrošina moduļu “Sensoru panelis”, un “Sensoru pārvaldība” prasības.

GET /api/sensor	
Atbilst prasībai	Sensor.List
Apraksts	Iegūst visu sensoru sarakstu.
Atbilde	200 - Kolekcijas “sensors” dokumenti.
Veiksmīgas atbildes formāts	
<pre>[{ "id": "string", "topics": ["string"], "isActive": true, "location": "string", "metadata": { "additionalProp1": "string", "additionalProp2": "string", "additionalProp3": "string" }, "latestMeasurementTimestamp": "2024-12-27T17:12:22.523Z", "latestMeasurements": { "additionalProp1": "string", "additionalProp2": "string", "additionalProp3": "string" }, "description": "string" }]</pre>	

GET /api/sensor/{sensorId}	
Atbilst prasībai	Sensor.View
Apraksts	Iegūst konkrētā sensora informāciju pēc tā identifikatora.
Parametri	sensorId (String) - Sensora identifikators.
Atbilde	200 - Konkrētā sensora dokuments kolekcijā “sensors”.
Kļūdas	404 – Sensors netika atrasts.
Veiksmīgas atbildes formāts	
<pre>{ "id": "string", "topics": ["string"], "isActive": true, "location": "string", "metadata": { "additionalProp1": "string", "additionalProp2": "string", "additionalProp3": "string" }, "latestMeasurementTimestamp": "2024-12-27T17:28:49.620Z", "latestMeasurements": { "additionalProp1": "string", "additionalProp2": "string", </pre>	

<pre> "additionalProp3": "string" }, "description": "string" } </pre>

GET /api/sensor/{sensorId}/measurements/today	
Atbilst prasībai	Sensor.View
Apraksts	Iegūst konkrētā sensora šodienas mērījumus.
Parametri	sensorId (String) - Sensora identifikators.
Atbilde	Šodienas sensora mērījumi no kolekcijas “sensorMeasurements”.
Kļūdas	404 – Sensors netika atrasts.
Apstrāde	
1. Pārbauda vai sensors eksistē kolekcijā “sensors”, ja tas neeksistē atgriež kļūda 404 – sensors netika atrasts. 2. Pieprasa datubāzei un atgriež šodienas sensora mērījumus.	
Veiksmīgas atbildes formāts	
<pre> [{ "id": "string", "timestamp": "2024-12-27T17:34:39.130Z", "sensorId": "string", "measurements": { "additionalProp1": "string", "additionalProp2": "string", "additionalProp3": "string" } }] </pre>	

PATCH /api/sensor	
Atbilst prasībām	Sensor.Edit, Sensor.Disable
Apraksts	Atjaunina sensora atrašanās vietu un vai ir pieslēgts.
Pieprasījuma formāts	id (String, obligāts) – Sensora identifikators, tāds kāds tas ir kolekcijā “sensors”. location (String, neobligāts) – Sensora jaunā atrašanās vieta. isActive (Bool, neobligāts) – Vai sensors ir pieslēgts.
Atbilde	200 – Atjaunina sensora datus.
Kļūdas	404 – Sensors netika atrasts. 400 – Nepareizs pieprasījums.
Apstrāde	
1. Pārbauda vai sensors eksistē kolekcijā “sensors”, iegūstot pašreizējos sensora datus.	

2. Ja tas neeksistē tad atgriež kļūda 404 – sensors netika atrasts.
3. Salīdzina vai sensora atribūti “isActive” un “location” atšķiras no to jaunajām vērtībām.
4. Atribūti, kas nav ar tādu pašu veco vērtību:
 - a. Tiek atjaunināti.
 - b. Tiek saglabāta izmaiņu vēsture kolekcijā “sensorMetadatas”, kur
 - i. sensorId – patreizējais sensora identifikators.
 - ii. timestamp – atjauninājuma brīdis ģenerēts ar DateTime.Now.
 - iii. metadata – satur atjaunināto lauka nosaukumu un vērtību (piemēram, {“location”: “322.telpa”}).

3.3.2 Paziņojumu galapunkti “NotificationsController”

GET /api/notifications	
Atbilst prasībai	Notifications.List
Apraksts	Iegūst visu paziņojumu sarakstu.
Atbilde	200 - Kolekcijas “notifications” dokumenti.
Veiksmīgas atbildes formāts	
<pre>[{ "id": "string", "sensorId": "string", "message": "string", "startTimestamp": "2024-12-28T00:39:42.520Z", "endTimestamp": "2024-12-28T00:39:42.520Z", "ruleId": "string" }]</pre>	

GET /api/notifications/rules	
Atbilst prasībai	Notifications.Rules.List
Apraksts	Iegūst visu noteikumu sarakstu.
Atbilde	200 - Kolekcijas “notificationRules” dokumenti.
Veiksmīgas atbildes formāts	
<pre>[{ "id": "string", "name": "string", "sensorId": "string", "measurement": "string", "operator": "string", "value": 0 }]</pre>	

```
}
]
```

GET /api/notifications/rules/{ruleId}	
Atbilst prasībām	Notifications.Rules.View
Apraksts	Iegūst konkrēto noteikumu pēc tā identifikatora.
Parametri	ruleId (String) – Paziņojuma noteikuma identifikators.
Atbilde	200 – Konkrētā noteikuma dokuments kolekcijā “notificationsRules”, papildināts ar atribūtu “Notifications”, kas satur saistītos paziņojumus no kolekcijas “notifications”.
Kļūdas	404 – Paziņojumu noteikums netika atrasts.
Apstrāde	
1. Pieprasa datubāzei noteikumu paziņojumu ar doto identifikatoru. 2. Ja tāds netika atrasts, tad atgriež kļūda 404 – paziņojumu noteikums netika atrasts. 3. Pieprasa datubāzei visus paziņojumus saistītus ar konkrēto noteikumu. Tas ir paziņojumus, kuriem atribūts “ruleId” sakrīt ar noteikuma identifikatoru. 4. Atgriež noteikumu ar papildinātu atribūtu “notifications”, kas satur saistītos paziņojumus.	
Veiksmīgas atbildes formāts	
<pre>{ "id": "string", "name": "string", "sensorId": "string", "measurement": "string", "operator": "string", "value": 0, "notifications": [{ "id": "string", "sensorId": "string", "message": "string", "startTimestamp": "2024-12-28T01:05:02.405Z", "endTimestamp": "2024-12-28T01:05:02.405Z", "ruleId": "string" }] }</pre>	

POST /api/notifications/rules/new	
Atbilst prasībām	Notifications.Rule.CUD
Apraksts	Atjauno paziņojumu noteikumu.
Pieprasījuma formāts	name (String) – Noteikuma nosaukums. sensorId (String, neobligāts) – Ar noteikumu saistītais sensors. measurement (String) – Kritērija izteiksmes mērvienība. operator (String, (“>” vai “<”)) – Kritērija izteiksmes operators. value (String) – Kritērija sliekšņa vērtība.
Atbilde	200 – Paziņojuma noteikums veiksmīgi izveidots.

PUT /api/notifications/rules	
Atbilst prasībām	Notifications.Rule.CUD
Apraksts	Atjaunina paziņojumu noteikumu, ja tas pastāv.
Pieprasījuma formāts	id (String) – Noteikuma identifikators. name (String) – Noteikuma nosaukums. sensorId (String, neobligāts) – Ar noteikumu saistītais sensors. measurement (String) – Kritērija izteiksmes mērvienība. operator (String, (“>” vai “<”)) – Kritērija izteiksmes operators. value (String) – Kritērija sliekšņa vērtība.
Atbilde	200 – Noteikums veiksmīgi atjaunināts.
Kļūda	404 – Noteikums netika atrasts.

DELETE /api/notifications/rules	
Atbilst prasībām	Notifications.Rule.CUD
Apraksts	Dzēš paziņojumu noteikumu.
Parametri	ruleId (String) – Paziņojuma noteikuma identifikators.
Atbilde	200 – Noteikums dzēsts.

3.3.3 Sensoru mērījumu monitorings ar “SensorHub”

Klase “SensorHub” kopā ar klasēm “SensorWatcherService” un “GroupTrackingService” realizē prasību “Sensor.Monitor”, nodrošinot iespēju iegūt sensora mērījumus reālā laikā. Tiek pielietots SignlarR [13], kas nodrošina reālā laika komunikācijas funkcionalitāti.

3.3.4 Klase "SensorHub"

Piezīme par izmantotajām tehnoloģijām:

- MongoDB "Change Streams" tiek izmantots, lai reāllaikā uztvertu, kad tiek saglabāts konkrēta sensora jauns mērījums.
- SignalR grupas nodrošina mehānismu reāllaika komunikācijai un ziņojumu nosūtīšanai noteiktas grupas dalībniekiem.

Klase "SensorHub" nodrošina SignalR [13] galapunktu "/sensorhub", kas atļauj abonēt konkrēta sensora reāllaika mērījumu saņemšanu. SignalR Hub metodes tiek uztvertas kā API darbības, kuras tiek precizētas veicot pieprasījumus no klienta puses koda.

- Metode "SubscribeToSensor":
 - Izsauk klienta puses kods, lai abonētu reālā laika mērījumus konkrētam sensoram.
 - Pievieno lietotāju konkrētā sensora grupai (SignalR grupas [14] nodrošina ziņojumu apraidi grupas abonentiem).
 - Pievieno sensoru kā skatīto sensoru klasei "SensorWatcherService", kas veido un uztur jaunu mērījumu vērošanu datubāzē ar "Change Streams".
 - Uztur konkrētās grupas abonementu skaitu ar klasi "GroupTrackingService". Tas ir svarīgi lai varētu izsekot, kad grupa ir tukša. Tukšas grupas gadījumā "SensorWatcherService" beidz vērot konkrēta sensora mērījumus.
- Metode "UnsubscribeFromSensor":
 - Izsauk klienta puses kods, lai atceltu reālā laika mērījumu saņemšanu.
 - Izņem lietotāju no konkrētās sensoru grupas.
 - Atjaunina abonementu skaitu sensoram.
 - Ja sensora grupai abonementu skaits sasniedz nulli, tad tiek nodots signāls "SensorWatcherService" pārtraukt konkrētā sensora mērījumu vērošanu.

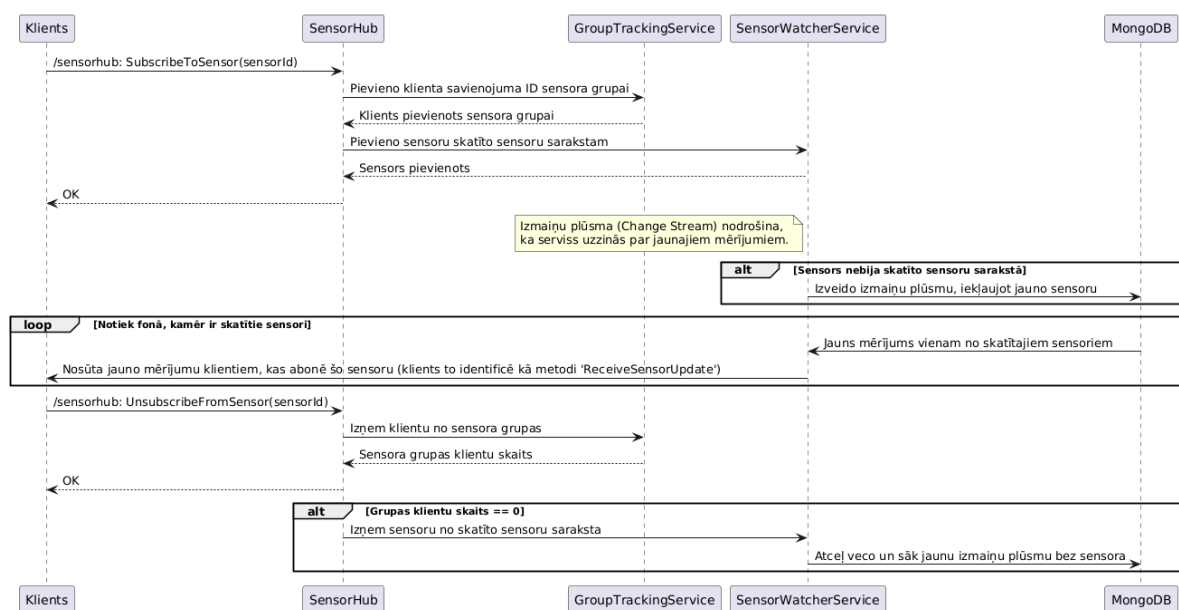
Sadarbība ar klasi "SensorWatcherService":

- Izmanto MongoDB "Change Streams", lai vērotu konkrētu sensoru jaunus mērījumus.
- Nosūta jaunus mērījumus attiecīgajai sensoru abonementu grupai,

Sadarbība ar klasi "GroupTrackingService":

- Palīdz izsekot cik lietotāji ir esošajā brīdī abonējuši konkrētu sensoru.
- Nepieciešams, jo SignalR grupas [14] nenodrošina iespēju uzzināt abonentu skaitu.

Sekvenču diagrammā (Att. 16) tiek attēlota sadarbība starp minētajām klasēm sensoru mērījumu abonēšanā. Tajā tiek parādīts process no klienta abonēšanas līdz abonementa atcelšanai. Klase “SensorWatcherService” ir aktīva nēpārtraukti fonā un sūta jaunos mērījumus sensoru abonementu grupām.



Att. 16 Sensoru reālā laika mērījumu abonēšana

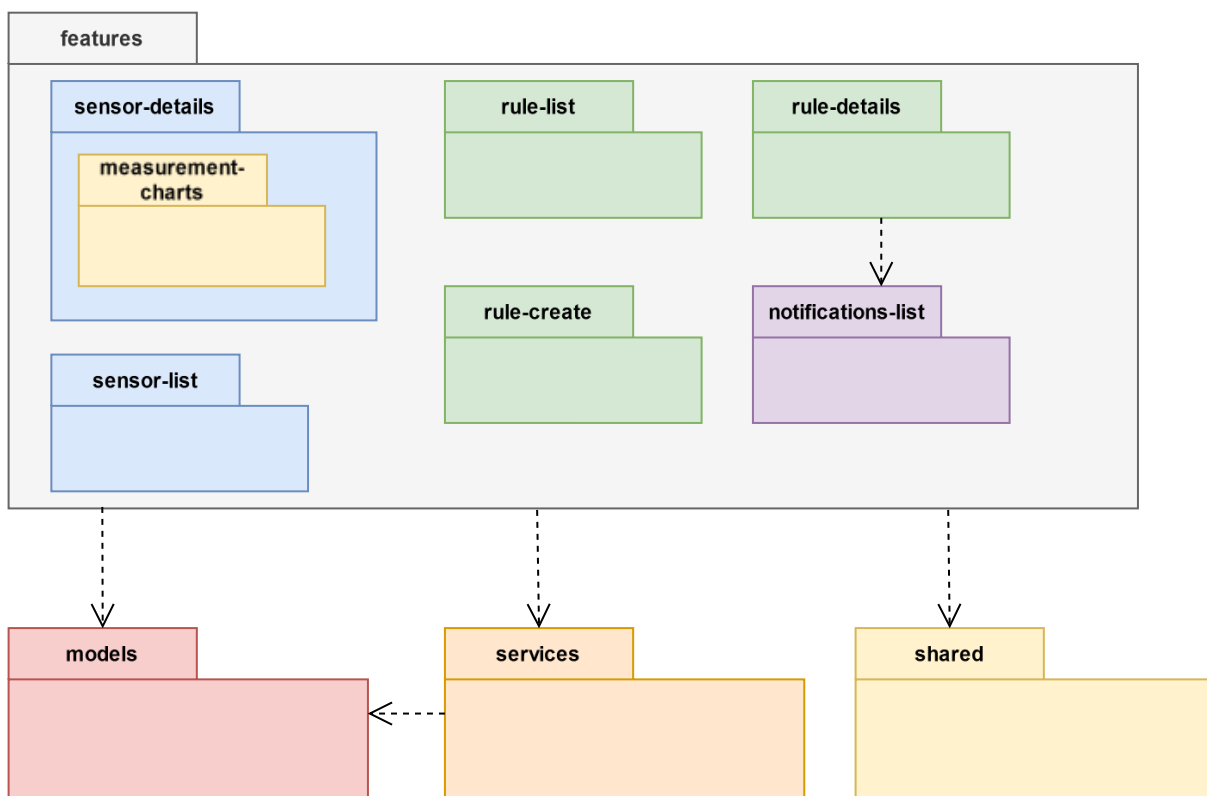
3.4 Komponente “Sensor.Dashboard”

Komponente “Sensor.Dashboard” ir tīmekļa lapas priekšgalsistēma, un tā nodrošina moduļu “Sensoru panelis” un “Paziņojumi” lietotajā saskarnes prasības. Tā tiek izstrādāta TypeScript un Angular.

“Sensor.Dashboard” tiek sadalīts funkcionalitātēs (features), servisos (services), modeļos (models) un kopīgajās komponentēs (shared) kā to attēlo Att. 17 “Sensor.Dashboard” sadalījums UML pakotņu diagramm.

- Katra funkcionalitāte (feature) nodrošina vismaz vienu skatu un vismaz vienu funkcionālo prasību (3.4.1. Tabula “Sensor.Dashboard” funkcionālo prasību trasējamības tabula).
- Servisi nodrošina pieeju “Sensor.WebApi” galapunktiem, izpildot API pieprasījumus.

- Modeļi definē TypeScript saskarnes (interfaces [15]), kas nodrošina datu struktūras saskarē ar “Sensor.WebApi” galapunktiem.
- Kopīgās komponentes (shared) ir atkārtojami izmantojamas komponentes, kuras izmanto vairākas citas komponentes, piemēram, saraksta attēlošanas komponente “TableComponent”.



Att. 17 "Sensor.Dashboard" sadalījums UML pakotņu diagramma

3.4.1. Tabula "Sensor.Dashboard" funkcionālo prasību trasējāmības tabula

Funkcionālā prasība	Atbilstošās funkcionalitātes (feature)
Sensor.List	sensor-list
Sensor.View	sensor-details
Sensor.Monitor	measurement-charts
Sensor.Edit	sensor-details
Sensor.Disable	sensor-details
Notifications.Rule.CUD	rule-details, rule-create
Notifications.Rules.List	rule-list
Notifications.Rules.View	rule-details
Notifications.List	notifications-list

3.4.1 Komponente “TableComponent”

Kopīgā (shared) komponente “TableComponent” nodrošina sarakstu loģiku un attēlošanu lietotnē.

Ievaddati:

- columns - Kolonnas:
 - key (string) - Datu atslēgas lauks.
 - label (string) – Kolonas virsraksts.
 - format – Datu formatēšanas funkcija (neobligāta).
- data (any[]) – Attēlojamie dati.
- actions (string[]) – Darbību saraksts.
- showActions (boolean) – Vai rādīt darbību sarakstu.

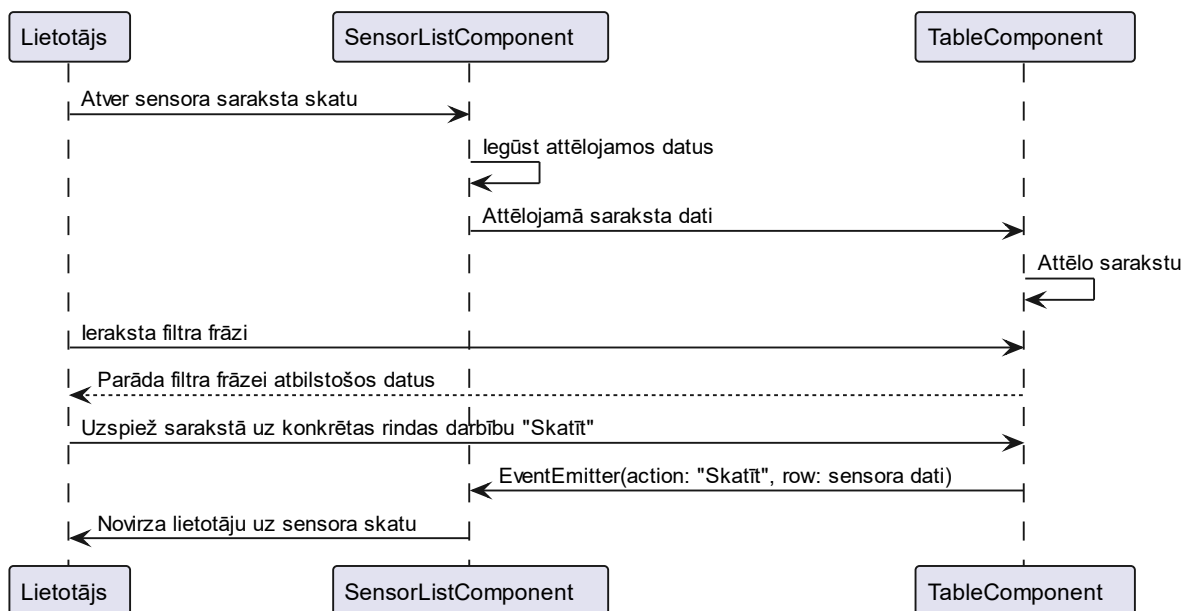
Izvaddati:

- actionClick (EventEmitter [16]<{action: string, row:any}>) - uzspiestās darbība ar tās rindas datiem.

Darbības:

1. Izveido saraksta tabulu ar kolonnu virsrakstiem no “columns”.
 - a. Ja “showActions == true”, tad papildina ar darbību kolonnu “Darbības”, kas katrai rindai nodrošinās darbības no darbību saraksta “actions”.
2. Aizpilda tabulu ar attēlojamiem datiem “data”. Katrai rindai:
 - a. Formatē datus, ja kolonnai ir norādīta “format” funkcija.
 - b. Attēlo masīva datus kā HTML sarakstu “”.
 - c. Pievieno darbības funkcijas atbilstoši “actions”, kas izvadīs darbības nosaukumu un rindas datus ar “actionClick”, ja tā tiks uzspiesta.
3. Nodrošina, ka uzspiežot rindas darbību tiks izvadīts “EventEmitter” [16] ar darbības nosaukumu un rindas datiem.
4. Saraksta filtrēšana, uzrādot tikai tās rindas, kuras datus ir filtra frāze.

Diagrammā (Att. 18) ir attēlota “TableComponent” pielietojums citā komponentē (šajā gadījumā “SensorListComponent”, kas ir daļa no “sensor-list” funkcionalitātes).



Att. 18 "TableComponent" pielietojums citā komponentē

3.4.2 Funkcionalitāte “sensor-list”

Atbilst prasībai: Sensor.List

Galvenā komponente: SensorListComponent

- Relatīvā adrese: /sensors
- Nodrošina skatu: “Sensoru saraksta skats”

Mērķis:

- Iegūt visu sensoru sarakstu no galapunkta “/api/sensor” ar servisa klasi “SensorService”.
- Attēlot sensoru sarakstu, izmantojot komponenti “TableComponent”.
- Nodrošina navigāciju uz konkrēta sensora skatu “/sensor/:id” (funkcionalitāte “sensor-details”).

Darbības:

a. Sensoru saraksta iegūšana:

1. Inicializācijā izsauc un sagaida atbildi no “SensorService.getSensors()”, lai iegūtu no galapunkta “/api/sensor” visu sensoru sarakstu.

2. Pārvērš katra sensora “latestMeasurements” par String masīvu, kas katru mērījuma atslēgas un vērtību pāri transformē par vienu String virkni. Tas tiek darīts, lai atvieglotu saraksta vizualizāciju ar “TableComponent”. To pašu izdara ar “metadata”.
- b. Saraksta komponentes “TableComponent” iestatīšana:
1. Norāda saraksta kolonas un atbilstošos laukus:
 - ID: id
 - Atrašanās vieta: location
 - Aktīvs: isActive
 1. Norāda formatējumu: true/false -> Jā/Nē.
 - Metadati: metadata
 - Pēdējie mērījumi: latestMeasurements
 2. Norāda komponentei izmantotos datus (sensoru saraksts).
 3. Precizē saraksta rindas darbību “Skatīt”, kas ar Router [17] atvērš sensora skatu, kuru nodrošina funkcionalitāte “sensor-details”.

3.4.3 Funkcionalitāte “sensor-details”

Atbilst prasībām: Sensor.View, Sensor.Edit, Sensor.Disable, Sensor.Monitor

Galvenā komponente: SensorDetailsComponent

- Relatīvā adrese: /sensor/:id
- Nodrošina skatu: “Sensora skats”

Mērķis:

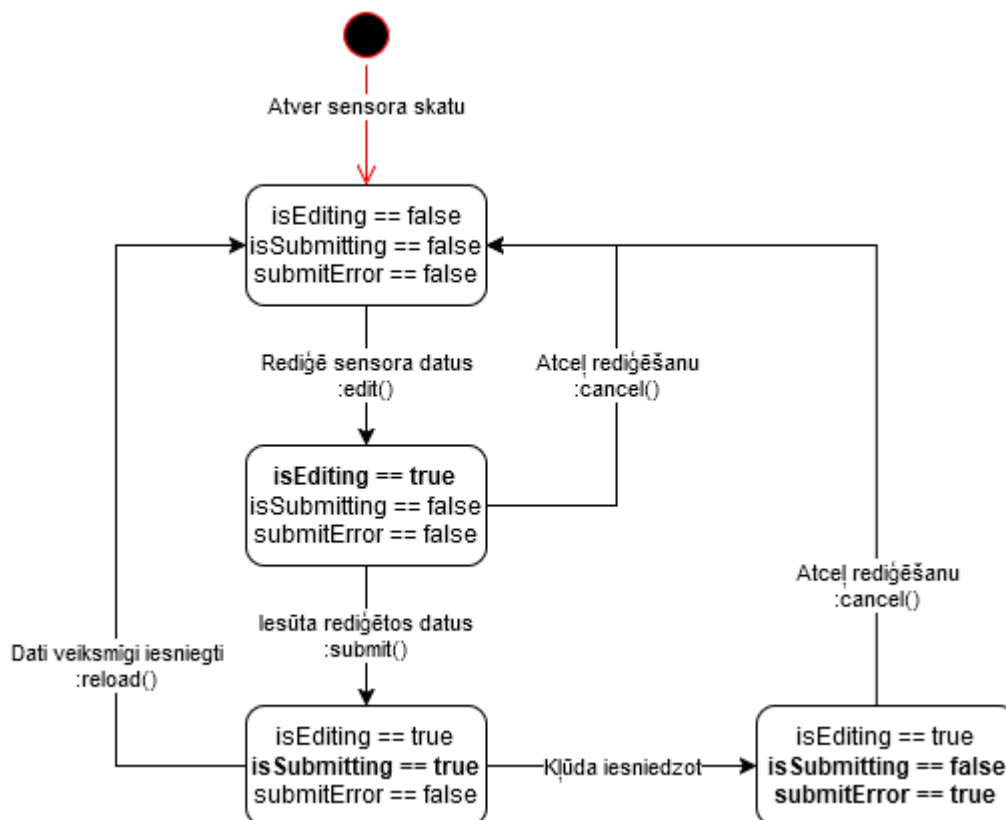
- Attēlot sensora datus un mērījumus un nodrošināt mērījumu diagrammas ar komponenti “MeasurementCharts”.
- Iegūt konkrētā sensora datus ar “SensorService.getSensoryById(sensorId)” no galapunkta “GET /api/sensor/{sensorId}”.
- Iespējot sensora atrašanās vietas “location” un pieslēguma “isActive” statusa rediģēšanu ar “SensorService.updateSensor(SensorUpdateModel)”, kas izmanto galapunktu “PATCH /api/sensor”

Darbības:

- a. Sensora datu iegūšana:
 1. Inicializācijā nolasa sensora ID no URL maršruta parametra “:id”.

2. Izsauc API galapunktu “GET /api/sensor/{sensorId}”, izmantojot “SensorService.getSensorById()”, lai iegūtu sensora datus.
 3. Pārveido sensora “metadata” lauku par String masīvu, kur katrs “metadata” atslēgas un vērtības pāris ir pārvērsts par String virkni, lai atvieglotu tā attēlošanu.
- b. Rediģēšana:
1. Kad lietotājs nospiež rediģēšanas pogu:
 - Sensora dati tiek aizpildīti rediģēšanas modelī “SensorUpdateDto”
 - Tiek iestatīts “isEditing = false”, lai tiktu parādīta rediģēšanas forma.
 2. “SensorUpdateDto” definē rediģējamos laukus:
 - location – Atrašanās vieta.
 - isActive – Sensors ir pieslēgts vai atslēgts.
- c. Rediģēšanas iesniegšana, kad lietotājs nospiež iesniegšanas pogu:
1. Iestata “isSubmitting = true”, lai ar Bootstrap Spinner [18] tiktu attēlots darbības process.
 2. Izsauc API galapunktu “PATCH /api/sensor”, izmantojot “SensorService.updateSensor(SensorUpdateDto)”.
 3. Ja API atgriež kļūdu, tad iestata “submitError = true”, lai tiktu attēlota iesnieguma kļūda.
 4. Procesā beigās iestata “isSubmitting = false”, lai apstādinātu Bootstrap Spinner.
- d. Mērījumu attēlošana:
1. Ja sensors ir aktīvs “isActive == true” tiek attēlota komponente “MeasurementChartsComponent”, kas attēlo sensora mērījumus diagrammās izmantojot ngx-charts [19] un nodrošina reālā laika mērījumus ar SignalR [20].

Diagrammā (Att. 19) ir attēlota “isEditing”, “isSubmitting”, “submitError” stāvokļu diagramma. Šie stāvokļi nosaka attēlotu skatu. Piemēram, “isEditing == true” attēlo sensora rediģēšanas skatu. “isSubmitting == true” ir patiess tikai tad, kad dati ir iesniegti un tiek gaidīta atbilde no servera, un skats attēlo gaidīšanas procesu ar Bootstrap Spinner.



Att. 19 Sensora skata būtisko mainīgo stāvokļu diagramma

3.4.3.1 Komponente “MeasurementChartsComponent”

Atbilst prasībām: Sensor.View, Sensor.Monitor

Mērķis:

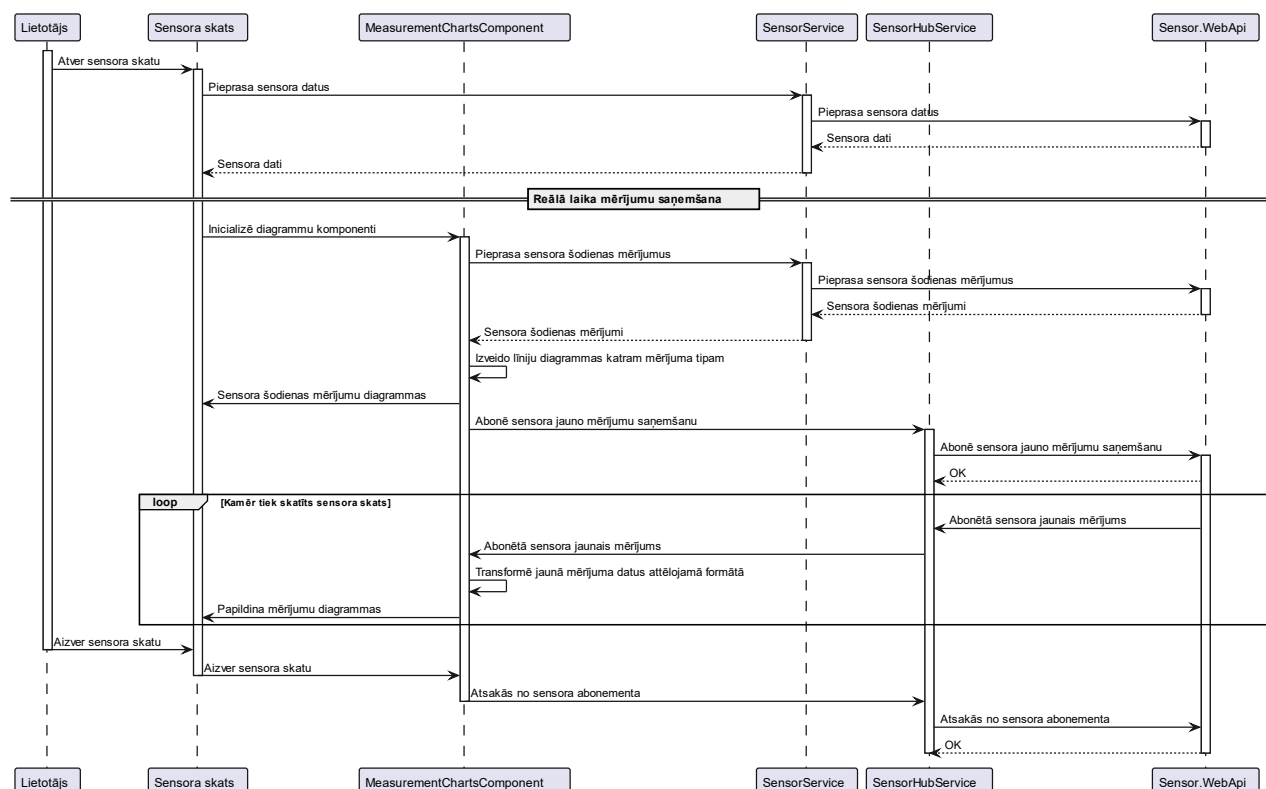
- Attēlot sensora šodienas mērījumus līniju diagrammās, izmantojot “ngx-charts” [19].
- Nodrošināt diagrammu atjaunošanu ar reālā laika mērījumiem, izmantojot “SensorHubService”.

Darbības:

- a. Diagrammu sākotnējā attēlošana:

1. Iegūst sensora šodienas mērījumus no galapunkta “GET /api/sensor/measurements/today”, izmantojot “SensorService.getTodaysSensorMeasurements(sensorId)”.
 2. Katram mērījuma tipam izveido diagrammu.
- b. Datu pārveidošana:
1. Pārveido datus formātā, kuru var izmantot “ngx-charts” un saglabā tos objektā “sensorData (TransformedMeasurement[])”.
 2. Datu tipa TransformedMeasurement struktūra:
 - name (string) – Mērījuma tipa nosaukums.
 - series – Masīvs kas satur konkrētā mērījuma tipa mērījumus: “name” kā mērījuma “timestamp” un “value” kā mērījuma vērtību.
- c. Reālā laika mērījumi:
1. Ar servisa klasi “SensorHubService” abonē “Sensor.WebApi” SignalR galapunktu “/api/sensorhub”.
 2. Saņemtos mērījumus pārveido un pievieno “sensorData” objektam, papildinot diagrammas.

Sekvenču diagrammā (Att. 20) tiek attēlots “Sensors skats” atvēršanas process, ieskaitot reālā laika mērījumu nodrošināšanu.



Att. 20 Sensora skata atvēršana un reālā laika mērījumu sekvenču diagramma

3.4.4 Funkcionalitāte “rule-create”

Atbilst prasībai: Notifications.Rule.CUD

Galvenā komponente: RuleCreateComponent

- Relatīvā adrese: /notifications/rules/new
- Saistītais skats: “Jauna paziņojumu noteikuma skats”

Mērķis:

- Nodrošināt jauna paziņojumu noteikuma izveidi, izmantojot galapunktu “POST /api/notification/rules/new” ar “NotificationService.createNotificationRule()”.
- Pēc veiksmīgas noteikuma izveides novirzīt lietotāju uz jaunizveidotā noteikuma skatu (funkcionalitāte “rule-details”).

Darbības:

a. Formas izveide:

1. Noteikums “rule” tiek inicializēts ar tukšām un ar noklusējuma vērtībām:
 - name: tukšs
 - sensorId: tukšs
 - measurement: co2
 - operator: >
 - value: 900
 - ruleString: co2 > 900
2. Ar “ruleString” tiek nodrošināta ērtāka noteikuma definēšana kā viena simbolu virkne.

b. Datu iesniegšana, kad lietotājs nospiež iesniegšanas pogu:

1. Tiek iestatīts “isSubmitting = true” un attēlots iesniegšanas process ar Bootstrap Spinner [18].
2. Nosacījuma virkne “ruleString” tiek sadalīta trīs daļās (“measurement”, “operator”, “value”).
3. Tiek pārbaudīts vai “operator” ir derīgs (“>” vai “<”).
 - Ja saskarās ar kļūdu tad iestata “submitError = true” un tiek informēts lietotājs, ka iesniegšana ir bijusi neveiksmīga un attēlots pareiza iesnieguma piemērs.

4. Noteikums tiek saglabāts ar “NotificationService.CreateNotificationRule(rule)”, nosūtot to uz galapunktu “POST /api/notification/rules/new”.

3.4.5 Funkcionalitāte “rule-details”

Atbilst prasībai: Notifications.Rules.View

Galvenā komponente: RuleDetailsComponent

- Relatīvā adrese: /notifications/rules/:id
- Saistītais skats: “Paziņojumu noteikuma skats”

Mērķis:

- Iegūt konkrētā sensora datus ar “NotificationService.getNotificationRule(ruleId)” no galapunkta “GET /api/notifications/rules/{ruleId}”.
- Attēlot saistīto paziņojumu sarakstu ar “TableComponent”.
- Iespējot noteikuma rediģēšanu un dzēšanu.

Darbības:

- a. Noteikuma datu iegūšana:
 1. Inicializācijā nolasa noteikuma ID no URL maršruta parametra “:id”.
 2. Izsauc API galapunktu “GET /api/notifications/rules/{ruleId}”, izmantojot “NotificationService.getNotificationRule(ruleId)”, lai iegūtu noteikuma datus un saistītos paziņojumus.
- b. Saistīto paziņojumu saraksta attēlošana:
 1. Pielieto funkcionalitātes “rule-list” komponenti “RuleListComponent”, kas realizē paziņojumu saraksta attēlošanu (līdzīgi funkcionalitātei “sensor-list”).
 2. Norāda paziņojumu saraksta attēlošanas komponentei “RuleListComponent”:
 - Saistīto paziņojumu datus.
 - “@Input showActions” vērtību kā “false”, lai netiktu parādīta darbību josla, kas parāda “Skatīt noteikumu” darbību, jo skats jau atrodas uz saistītā noteikuma.
- c. Rediģēšana:
 1. Rediģēšanas process ir līdzīgs funkcionalitātei “sensor-details”, bet atšķiras ar rediģējamiem laukiem un izmantoto galapunktu.

2. Rediģējamie lauki un validācija ir līdzīga kā aprakstītā loģika iekš funkcionalitātes “rule-create”.
 3. Saglabāšanai izmanto “NotificationService.updateNotificationRule(notificationRule)”, kas saglabā ar galapunktu “PUT /api/notifications/rules”.
- d. Dzēšana, kad lietotājs nospiež dzēšanas pogu:
1. Parāda uznirstošu logu, kas pieprasa lietotājam apstiprināt dzēšanu.
 2. Noteikums tiek dzēsts ar “NotificationService.deleteNotificationRule(ruleId)”, izmantojot galapunktu “DELETE /api/notifications/rules”.
 3. Pēc veiksmīgas dzēšanas atver skatu “Paziņojumu noteikumu saraksta skats”.

3.5 Komponente “Sensor.Courier”

Komponente “Sensor.Courier” nodrošina sensoru datu eksportu uz SQL datubāzi, atbilstoši “Datu eksporta moduļa” prasībām.

Tiek atbalstīts datu eksports uz divu tipa SQL datubāzēm – SqlServer un MySql. Tiek nodrošināta iespēja izvēlēties datubāzes tipu, norādot to konfigurācijā.

Komponente veidota tā, lai ja nākotnē rodas nepieciešamība paplašināt atbalstu citām SQL datubāzēm, piemēram, PostgreSQL, tad to pievienošana nebūtu pārāk sarežģīta. Tas tiek sasniegts izmantojot atsevišķus datubāzes migrāciju failus katram SQL datubāzes tipam. Lai nodrošinātu sistēmas spēju darboties ar dažādām datubāzēm tiek pielietota Entity Framework iespēja konfigurēt DbContext [21] klasi konkrētajai datubāzei sistēmas darbības laikā.

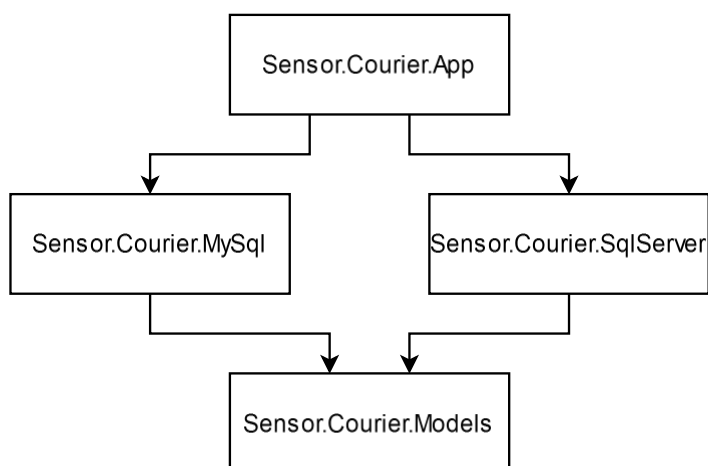
Lai pievienotu atbalstu, piemēram, PostgreSQL datubāzei, tad būtu nepieciešams ar Entity Framework komandām [22] ģenerēt migrācijas failus PostgreSQL datubāzei un papildināt programmu ar konfigurācijas atbalstu priekš PostgreSQL.

Komponentes arhitektūras izstrādei tiek izmantota Khalid Abuhakmeh rakstā "Entity Framework Core and Multiple Database Providers" [23] aprakstītā pieeja kā iedvesmas avots. Rakstā aplūkota metode, kā veidot .NET risinājumu, kas atbalsta vairākus datubāžu tipus un ļauj pārslēgties starp tiem, balstoties uz konfigurācijas iestatījumiem. Šī pieeja kalpoja par pamatu “Sensor.Courier” komponentes vairāku datubāžu atbalsta izveidei, nodrošinot gan elastīgumu, gan iespēju nākotnē viegli pievienot jaunas datubāzes.

“Sensor.Courier” sastāv no sekojošajiem .NET projektiem:

1. App – galvenais darbības pirmkods.
2. Models – SQL datubāzes modeļi un DbContext.
3. Migrāciju projekti:
 - a. MySql – satur migrācijas MySql datubāzei.
 - b. SqlServer – satur migrācijas SqlServer datubāzei.

Diagrammā ir redzams, ka projekts “App” ir atkarīgs no projekta “MySQL” un “SqlServer”, kas ir atkarīgi no projekta “Models”.



Att. 21 "Sensor.Courier" projektu atkarības

3.5.1 SQL datubāzes projektējums

SQL datubāze ir aktuāla tikai “Sensor.Courier” komponentei. Datubāzes modelis (Att. 22) ir ļoti vienkāršs, jo primārais mērķis ir nogādāt datus. Datubāze sastāv no sensoru mērījumu tabulas “SensorMeasurements”, sensoru metadatu tabulas “SensorMetadata” un parametru tabulas “Parameters”.

SensorMeasurements	
PK	<u>Id</u>
	Timestamp
	SensorId
	MetricKey
	MetricValue

SensorMetadata	
PK	<u>Id</u>
	Timestamp
	SensorId
	MetaKey
	MetaValue

Parameters	
PK	<u>Id</u>
	Name
	Value

Att. 22 SQL datubāzes loģiskais modelis datu eksportam

3.5.1.1 Tabula “SensorMeasurements”

Tabulas “SensorMeasurements” mērķis ir glabāt sensoru mērījumus no MongoDB datubāzes kolekcijas “sensorMeasurements”.

Tabula "SensorMeasurements"			
Lauks	Datu tips		Apraksts
	MySql	SqlServer	
Id (PK)	Int	Int	Automātiski ģenerēts identifikators.
Timestamp	Datetime(6)	Datetime2	Mērījuma laika marka.
SensorId	Varchar(100)	Nvarchar(100)	Sensora identifikators.
MetricKey	Varchar(100)	Nvarchar(100)	Mērījuma nosaukums.
MetricValue	Longtext	Nvarchar(max)	Mērījuma vērtība.

3.5.1.2 Tabula "SensorMetadata"

Tabulas "SensorMetadata" mērķis ir glabāt sensoru mērījumus no MongoDB datubāzes kolekcijas "sensorMetadata".

Tabula "SensorMetadata"			
Lauks	Datu tips		Apraksts
	MySql	SqlServer	
Id (PK)	Int	Int	Automātiski ģenerēts identifikators.
Timestamp	Datetime(6)	Datetime2	Mērījuma laika marka.
SensorId	Varchar(100)	Nvarchar(100)	Sensora identifikators.
MetaKey	Varchar(100)	Nvarchar(100)	Sensora atribūta nosaukums.
MetaValue	Longtext	Nvarchar(max)	Sensora atribūta vērtība.

3.5.1.3 Tabula "Parameters"

Tabulas "Parameters" mērķis ir glabāt eksporta parametrus – partijas lielums (BatchSize) un aizkave (BatchDelaySeconds).

Tabula "Parameters"			
Lauks	Datu tips		Apraksts
	MySql	SqlServer	
Id (PK)	Int	Int	Automātiski ģenerēts identifikators.
Name	Varchar(100)	Nvarchar(100)	Parametra nosaukums.
Value	Varchar(255)	Nvarchar(255)	Parametra vērtība.

3.5.2 Konfigurācijas modelis datu ekstrakcijai

Konfigurācijas modelis datu ekstrakcijai norāda uz kuru datubāzi tiks eksportēti sistēmas Mongo datubāzes sensoru dati. Šie parametri ir konfigurējami failā “appsettings.json” un caur vides mainīgajiem (environment variables).

1. Provider

- “MySql” vai “SqlServer”
- Norāda izmantotās datubāzes tipu.

2. ConnectionStrings

- MySql
 - i. Norāda izmantoto savienojuma virkni priekš MySql datubāzes.
- SqlServer
 - i. Norāda izmantoto savienojuma virkni priekš SqlServer datubāzes.

3.5.3 Darbības konfigurācija

Tiek nodrošināta iespēja konfigurēt eksporta partijas lielumu un aizkavi starp partijām. Šie parametri ir konfigurējami SQL datubāzes tabulā “Parameters” kā “BatchSize” un “BatchDelaySeconds”. Parametri tiek nolasīti pirms katras eksporta partijas. Šo procesu izskaidro tabulās: “3.5.3.1 tabula Inicializēt datubāzi” un “

3.5.3.2 tabula Izpildīt ekstrakcijas procesu”.

Mērķis
Identificēt konfigurēto datubāzi un sagatavot sistēmu darbam ar to.
Ievaddati
1. Datubāzes tips (MySQL vai SqlServer). 2. Datubāzes savienojuma virkne.
Apstrāde
1. Identificē konfigurēto datubāzi un konfigurē DbContext: a. Konfigurē DbContext, ka tas izmantos SqlServer vai MySQL. b. Norāda datubāzei atbilstošās migrācijas. 2. Pārbauda vai datubāze ir sasniedzama. a. Ja datubāze nav sasniedzama tiek veikts fakta žurnālieraksts. 3. Inicializē datubāzi: a. Pārbauda vai visas migrācijas ir pielietotas un piemēro nepielietotās. b. Ja datubāze tiek pirmo reizi inicializēta, tad aizpilda parametru tabulu ar pamatvērtībām: i. BatchDelaySeconds = 60 ii. BatchSize = 10
Žurnālēšana
• Veic "LogLevel.Information" žurnālierakstus: ○ Par centieni savienoties ar datubāzi. ▪ "Attempting to connect to the database." ○ Par veiksmīgi inicializētu datubāzi. ▪ "Database is usable." ○ Par jau piemērotajām migrācijām. ▪ "Applied migrations: {Migrations}." ○ Par migrācijām, kas tiks piemērotas. ▪ "Applying migrations: {Migrations}." ○ Par to ka datubāze tiks iesēta. ▪ "Seeding database." • Veic "LogLevel.Information" žurnālierakstus: ○ Par kļūdu datubāzes inicializācijā. ▪ "An error occurred while connecting to the database.", iekļaujot kļūdu.

Mērķis
Pārvaldīt datu ekstrakcijas procesu, nolasot darbības parametrus un uzsākot ekstrakcijas un ielādes procesus.
Apstrāde
1. Iegūst parametrus “BatchSize” un “BatchDelaySeconds” no tabulas “Parameters” ar “ParameterService.GetAppSettings()”. 2. Iegūst pēdējā ielādētā mērījuma laiku no tabulas “SensorMeasurements” ar “ParameterService.GetLastMeasurementTimeStamp()”. a. Tas ir ieraksts ar vislielāko “Timestamp” vērtību. 3. Iegūst pēdējā ielādētā metadatu laiku no tabulas “SensorMetadata” ar “ParameterService.GetLastMetadataTimeStamp()”. 4. Uzsāk asinhronus ekstrakcijas un ielādes procesus: a. Uzsāk sensoru mērījumu ekstrakcijas un ielādes procesu ar “ETLService.ExtractAndLoadMeasurements()”, nododot tam pēdējā mērījuma laiku un eksporta partijas izmēru. b. Uzsāk sensoru metadatu ekstrakcijas un ielādes procesu ar “ETLService.ExtractAndLoadMetadata ()”, nododot tam pēdējā ielādētā metadatu laiku un partijas izmēru. 5. Sagaida procesu pabeigšanu ar “await Task.WhenAll()”. 6. Gaida “BatchDelaySeconds” sekundes un atkārtoti sākot ar pirmo soli.
Žurnālieraksti
• Veic “LogLevel.Information” žurnālierakstus: ○ Par procesa sākšanu pēc parametru iegūšanas. ▪ “Starting batch of size {batchSize} processing at {time}.” ○ Par veiksmīgu procesa izpildi. ▪ “Data processed successfully.” ○ Par nākamā procesa izpildes laiku. ▪ “Next batch in {batchDelay} seconds at {time}.” • Veic “LogLevel.Error” žurnālierakstus: ○ Katrai kļūdai procesā. ▪ “An error occurred while processing data.”, iekļaujot kļūdas “Exception”.

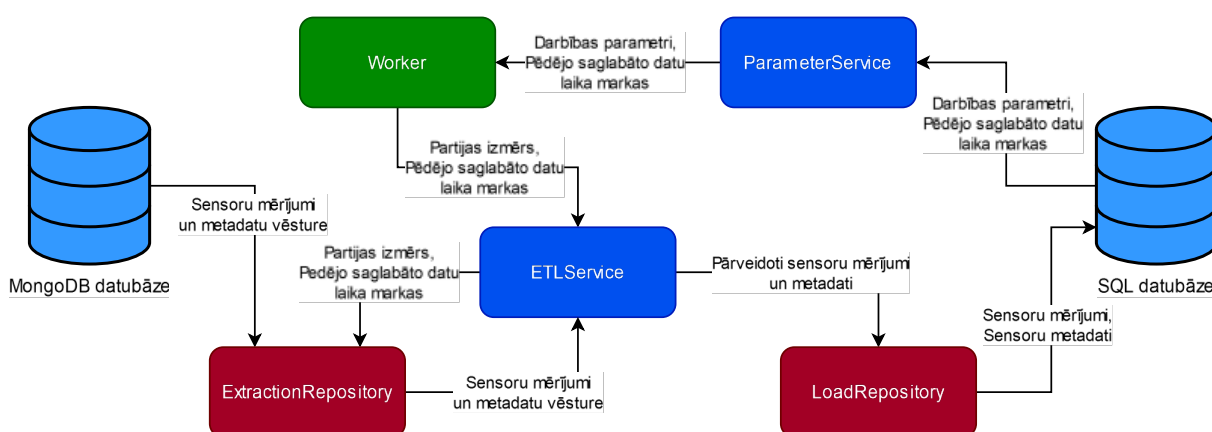
3.5.4 Sensoru datu ekstrakcija, transformācija un ielāde

Komponentes galvenā funkcija ir nodrošināt datu ekstrakciju no MongoDB datubāzes kolekcijām “sensorMeasurements” un “sensorMetadata” uz SQL datubāzes tabulām “SensorMeasurements” un “SensorMetadata”. Būtiskākā datu transformācija ir, ka katram “sensorMeasurements” dokumentam tiek izveidoti vairāki SQL tabulas “SensorMeasurements” ieraksti (3.5.4.1 tabula Mērījumu transformācija).

Diagrammā (Att. 23) ir redzama datu plūsma no MongoDB datubāzes līdz SQL datubāzei cauri komponentes klasēm.

Funkcijas nodrošina sekojošās klases:

- Klase “ETLService”
 - Nodrošina vienas partijas ekstrakcijas, transformācijas un ielādes metodes.
 - Darbojas kā transformācijas slānis, izsaucot datu ekstrakcijas metodes no klases “ExtractionRepository” un pārveidojot datus klases “LoadRepository” datu ielādes metodēm.
- Klase “ExtractionRepository”
 - Nodrošina datu ekstrakcijas metodes.
- Klase “LoadRepository”
 - Nodrošina datu ielādes metodes.



Att. 23 Komponentes “SensorCourier” datu plūsmu diagramma

Klase “ParameterService” nodrošina konfigurēto parametru “BatchSize” un “BatchDelaySeconds” iegūšanu kā arī pēdējo saglabātu datu laika markas.

Klase “Worker” nodrošina procesu vadību – parametru iegūšanu, aizkavi starp partijām un ekstrakcijas un ielādes procesa uzsākšanu.

Mērķis
Pārveidot MongoDB kolekcijas “sensorMeasurements” dokumentu saglabājumu SQL tabulā “SensorMeasurements”. Parādīt ka no viena MongoDB kolekcijas “sensorMeasurements” dokumentu tiek izveidoti vairāki ieraksti SQL datubāzes tabulā “SensorMeasurements”.
Ievaddati
Kolekcijas “sensorMeasurements” dokuments: 1. _id (ObjectId) 2. sensorId (String) 3. timestamp (Date) 4. measurements (Object) – iegultais dokuments, kur katrs lauks ir mērījums.
Apstrāde
1. Priekš katra mērījuma iegultajā dokumentā “measurements” izveido objektu “SensorMeasurement”: a. Timestamp = timestamp b. SensorId = sensorId c. MetricKey = konkrētā mērījuma nosaukums d. MetricValue = konkrētā mērījuma vērtība
Izvaddati
• Vairāki “SensorMeasurement” objekti.

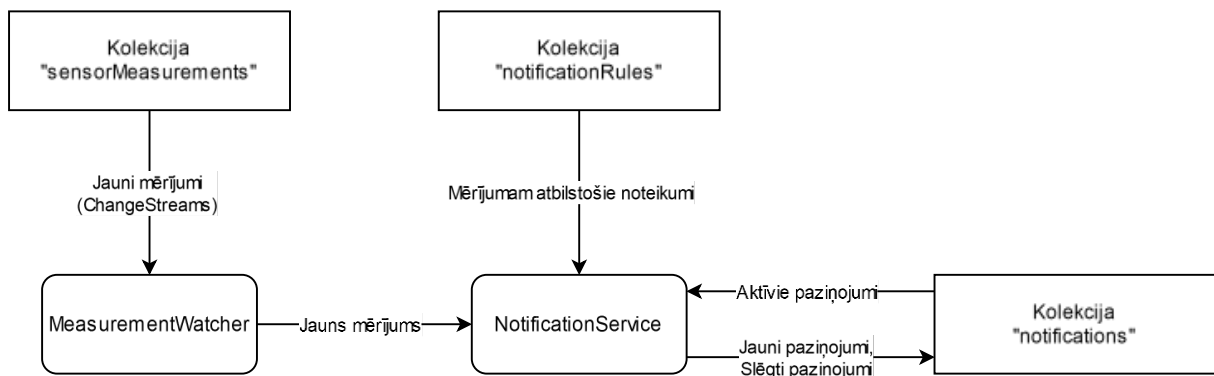
3.6 Komponente “Sensor.Notifications”

Nodrošina prasību Notifications.Generate.

Galvenā funkcionalitāte:

1. Analizēt ienākošos sensoru mērījumus (sensorMeasurements), salīdzinot ar paziņojumu noteikumiem (notificationRules).
2. Ģenerēt jaunu paziņojumu, ja mērījuma vērtība pirmoreiz pārsniedz noteikuma sliekšni.
3. Slēgt (atzīmēt ar “endTimeStamp”) paziņojumu, ja vērtība atgriežas normā.
4. Izvairīties no atkārotiem paziņojumiem par to pašu notikumu.

Komponente “Sensor.Notifications” darbojas kā atsevišķs izpildāms serviss, kas novēro datubāzes kolekciju “sensorMeasurements” ar MongoDB “Change Streams”. Kad tiek saglabāti jauni mērījumi, komponente salīdzina tos ar “notificationRules” un nepieciešamības gadījumā izveido vai slēdz paziņojumus.



Att. 24 “Sensor.Notifications” datu plūsma starp klasēm un kolekcijām

3.6.1 Klase “MeasurementWatcher”

Klases mērķis uzraudzīt kolekciju “sensorMeasurements” priekš jauniem ievietot mērījumiem.

Atbildības:

1. Sāk “Change Stream” [8] uz kolekcijas “sensorMeasurements”, lai uzraudzītu ievietotos dokumentus (OperationType.Insert).
2. Katru ievietoto dokumentu nodod “NotificationService.HandleMeasurements(SensorMeasurements)”.

Žurnalēšana:

- Veic “LogLevel.Error” žurnālierakstus par:
 - Procesā radušām kļūdām.
 - “Error watching for measurements.”, iekļaujot kļūdas “Exception”.
- Veic “LogLevel.Information” žurnālierakstus par:
 - ChangeStream izveidi.
 - “Watching for measurements.”
 - Procesa apturēšanu.
 - “Measurement watcher stopped.”
- Veic “LogLevel.Debug” žurnālierakstus par:
 - Katru novēroto ievietoto “sensorMeasurements” dokumentu.
 - “Measurement insertion detected: {Change}.”

Kļūdu apstrāde:

- “Change Stream” pārtrūkšanas gadījumā, veic “LogLevel.Error” žurnālierakstu ar kļūdu un cenšas atkārtoti izveidot “Change Stream”.

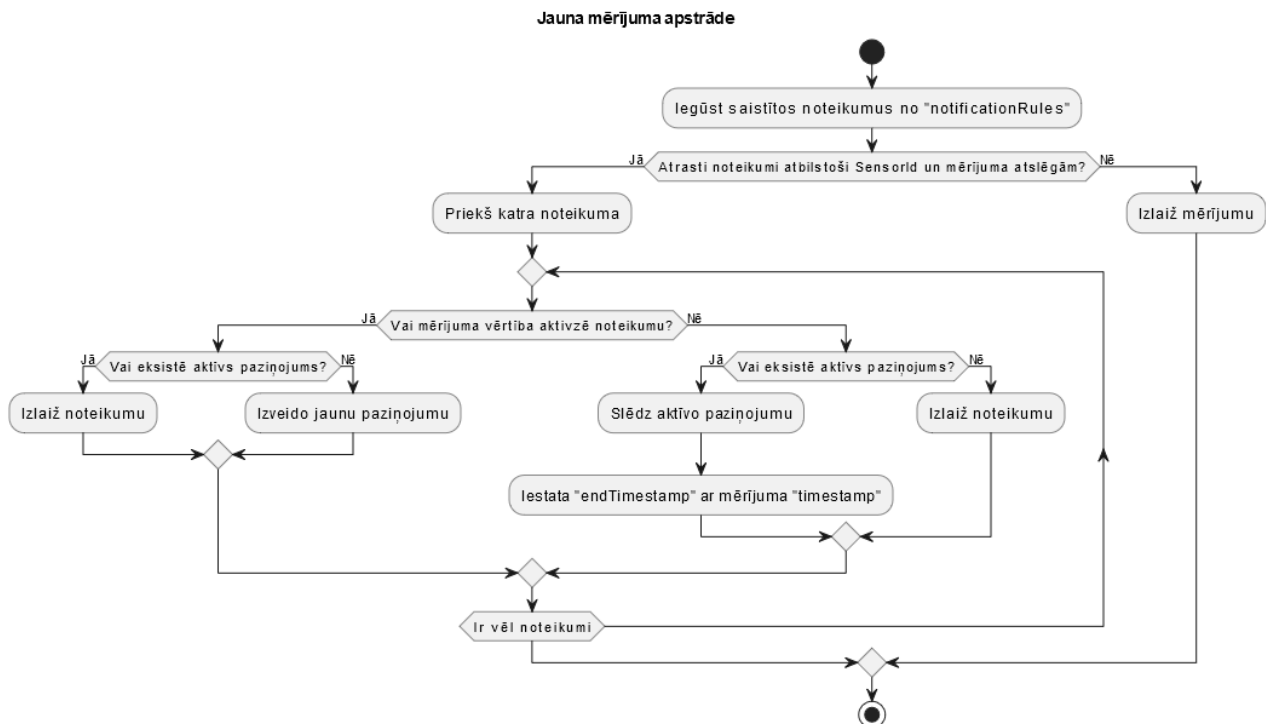
3.6.2 Klase “NotificationService”

Klase pārbauda, vai jaunie mērījumi aktivizē kādu paziņojumu noteikumu un izveido vai slēdz paziņojumus.

Process:

1. **Atrod atbilstošos noteikumus:** Iegūst noteikumu sarakstu no kolekcijas “notificationRules”, filtrējot pēc “sensorId” un mērījuma atslēgām.
2. Katram izfiltrētajam noteikumam:
 - a. **Pārbauda vai mērījums aktivizē noteikumu:**
 - i. Aktivizē, ja noteikuma operators ir “>” un mērījuma vērtība ir lielāka par noteikuma vērtību.
 - ii. Aktivizē, ja noteikuma operators ir “<” un mērījuma vērtība ir mazāka par noteikuma vērtību.
 - b. Ja aktivzē, tad:
 - i. Pārbauda vai nav jau aktīvs paziņojums kolekcijā “notifications”. Aktīvs paziņojums ir tāds, kuram sakrīt “sensorId” un “ruleId”, un “endTimeStamp” == null.

- ii. Ja nav aktīva paziņojuma, izveido jaunu paziņojumu.
- c. Ja neaktivizē, tad:
 - i. Pārbauda vai ir jau aktīvs paziņojums kolekcijā “notifications”.
 - ii. Ja atrod aktīvu paziņojumu, tad slēdz paziņojumu, iestatot paziņojuma “endTimestamp” ar mērījuma laika marku “timestamp”.



Att. 25 Jauna mērījuma apstrādes UML aktivitāšu diagramma

Žurnalēšana:

- Veic “LogLevel.Warning” žurnālierakstus:
 - Ja mērījuma vērtība nav parsējama kā “double”.
 - “Measurement not a number: {Measurement}.”
 - Ja tika atrasti vairāki aktīvi paziņojumi.
 - “Unexpected number of notifications: {Count}.”
- Veic “LogLevel.Information” žurnālierakstus:
 - Par katru izveidoto paziņojumu.
 - “Notification created: {Notification}.”
 - Par katru slēgto paziņojumu.

- “Notification updated: {Notification}.”
- Veic “LogLevel.Debug” žurnālierakstus:
 - Par aktivizēto noteikumu, norādot noteikumu un mērījumu.
 - “Rule triggered: {Rule}, Measurement: {Measurement}.”
 - Ja mērījums neaktivizē noteikumu, un nav atrodams aktīvs paziņojums.
 - “No notifications to clear found.”

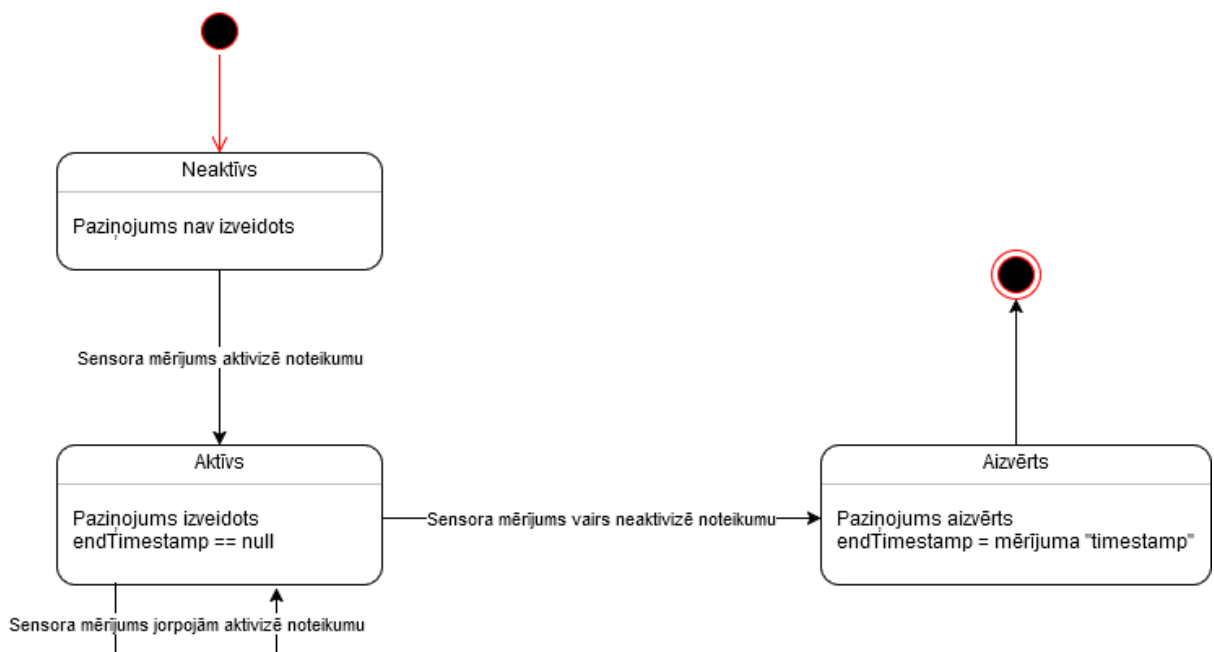
3.6.3 Dublētu (atkārtotu) paziņojumu novēršana

Atbilstoši prasības punktam “3. Nodrošināt, ka netiek sūtīti atkārtoti paziņojumi par to pašu notikumu...”.

Risinājums:

1. Kad pirmoreiz sensora mērījuma vērtība pārsniedz noteikuma sliekšni, tiek izveidots paziņojums ar “endTimeStamp” vērtību “null”.
2. Kamēr paziņojums ir aktīvs (endTimeStamp == null), netiek veidots jauns paziņojums, pat ja vērtība turpina augt (piemēram, no co2 = 900 līdz co2 = 1200)
3. Tiklīdz sensora mērījuma vērtība atgriežas normā (zem sliekšņa) un parādās jauns mērījums, paziņojums tiek slēgts iestatot “endTimeStamp” ar mērījuma laika marķu “timestamp”. Tikai pēc slēgšanas nākamais pārsniegšanas gadījums atkal rada jaunu paziņojumu.

Stāvokļu diagrammā (Att. 26) attēlo paziņojuma objekta stāvokļu pārmaiņas.



Att. 26 Paziņojuma stāvokļu diagramma

3.6.4 Saistītie datu modeļi

Sensor.Notifications svarīgākie datu modeļi:

1. Notification

- Atbilst kolekcijai “notifications”
- Struktūra:
 - Id (ObjectId)
 - SensorId (String) – norāda paziņojuma sensoru.
 - RuleId (ObjectId) – norāda aktivizēto noteikumu.
 - Message (String) – satur ziņu, ko izveido “NotificationRule.GetMessage(...)”.
 - StartTimestamp (DateTime) – kad aktivizēts.
 - EndTimestamp (DateTime?) – kad slēgts; null ja vēl aktīvs.

2. SensorMeasurements

- Atbilst kolekcijai “sensorMeasurements”
- Struktūra
 - Timestamp (DateTime)
 - SensorId (String)
 - Measurements (Dictionary<string, object>) – atslēgas ir, piemēram, “co2”, “temperature”.

3. NotificationRule

- Atbilst kolekcijai “notificationRules”
- Struktūra
 - Id (String)
 - Name (String)
 - SensorId (String) – ja tukšs, tad noteikums attiecināms uz visiem sensoriem
 - Measurement (String) – piemēram, “co2”, “temperature” u.c.
 - Operator (String) – “>” vai “<”
 - Value (Double) – sliekšņa vērtība.
- Metodes
 - Triggers(double value)
 - Pārbauda vai dotā vērtība aktivizē noteikumu.

- GetMessage(SensorMeasurements)
 - Izveido paziņojuma ziņu, izmantojot noteikuma nosaukumu, mērījumu, operatoru, vērtību un sensora identifikatoru.

3.7 Nefunkcionālo prasību projektējums

1. Sistēma spēj apstrādāt Raiņa bulvāri 19 izvietotos sensorus (~50).

Komponente “Sensor.Consumer” ir veidota horizontāli mērogojama, atļaujot vairāku vienlaicīgu patērētāju darbību, lai segtu augstāku slodzi, ja tas ir nepieciešams. To nodrošina MQTT piektā protokola versija, kas atļauj veidot grupas abonementus [24]. Sistēma veic žurnālierakstu rakstīšanu uz konsoli, kuros tiek ziņots par veiksmīgu un neveiksmīgu ziņojumu apstrādi.

Pārbaude:

- a) Simulēt 100 sensoru darbību pārbaudīt vai ir saņemti un saglabāti sagaidāmie dati.
- b) Pieslēgt sistēmu Aranet publiskajam HiveMQ brokerim “broker.hivemq.com” [25] un pārbaudīt vai sistēma izvada kļūdas.
- c) Pieslēgt sistēmu Raiņa bulvāri 19 MQTT brokerim un pārbaudīt vai sistēma izvada kļūdas.

2. Sensoru panelī apskatāmā sensora mērījumi tiek papildināti līdz ko jauns mērījums tiek uztverts (ne ilgāk kā ~1 sekunde).

Komponente “Sensor.Consumer” apstrādā saņemtos ziņojumus asinhroni, lai netiktu kavēta ziņojumu apstrāde. Reālā laika mērījumu monitorings tiek realizēts izmantojot MongoDB “Change Streams” [8], kas atļauj sistēmai nekavējoties uzzināt par jauniem mērījumiem datubāzē. Savukārt, lai lietotājs saņemtu mērījumus tiek izmantots SignalR [13]. SignalR nodrošina reālā laika komunikāciju ar WebSockets, tādā veidā ļaujot tīmekļa lapas aizmugursistēmai “Sensor.WebApi” izsūtīt jaunus datus lietotāja pārlūkprogrammai tiklīdz tie ir pieejami, un to paveikt bez nepieciešamības pēc atkārtotiem pieprasījumiem.

Šī datu plūsma darbojas šādi:

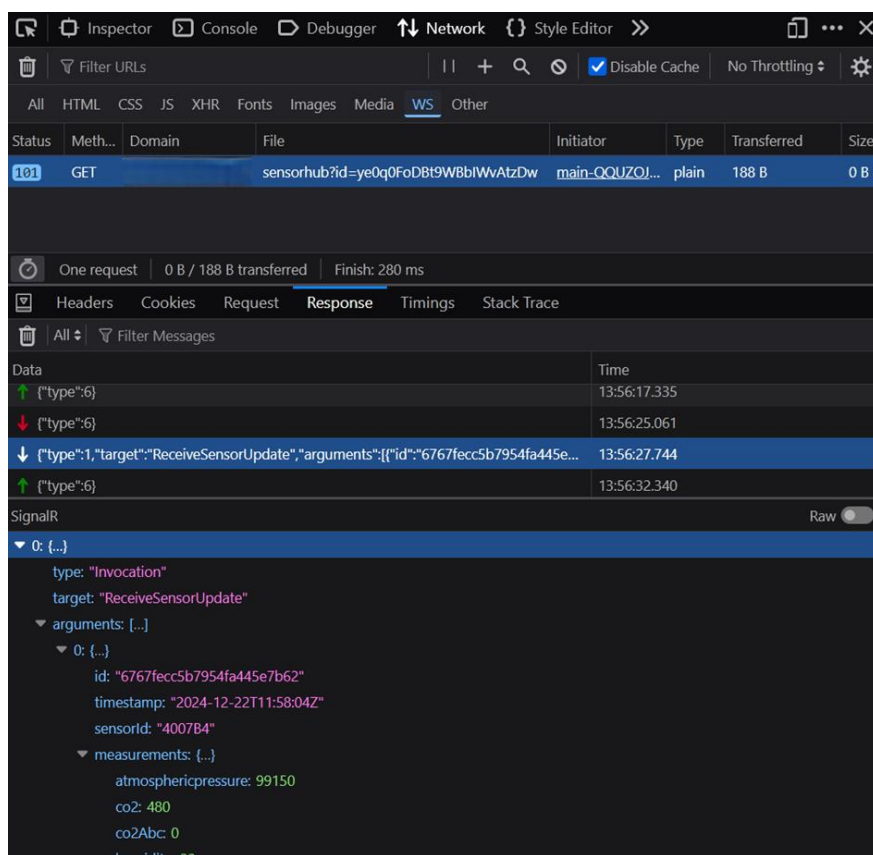
1. Lietotājs abonē konkrēta sensora jaunu mērījumu saņemšanu ar SignalR.
2. Sistēma izmanto MongoDB “Change Streams”, lai uzzinātu par jauniem mērījumiem, tiklīdz tie tiek saglabāti datubāzē.
3. Sistēma ar SignalR nosūta ar MongoDB “Change Streams” identificētos jaunus mērījumus.

Asinhronā ziņojumu apstrāde, MongoDB Change Streams un SignalR nodrošina ka sensoru paneļa diagrammas tiek papildinātas ar reālā laika mērījumiem. Aizkaves šajā procesā ir atkarīgas no pieejamiem resursiem datubāzei, komponentei “Sensor.Consumer” un tīkla ātrums starp lietotāja ierīci un serveri.

Pārbaude:

- a) “Sensor.Consumer” veic žurnālierakstu uz konsoli ziņojuma saņemšanas momentā. Šī žurnālieraksta laiks tiek salīdzināts ar laiku, kad klients saņem jauno mērījumu. Laiku, kad lietotājs saņem ziņojumu, var apskatīt pārlūkprogrammas iebūvētajā tīkla satiksmes rīkā.

Attēlā (Att. 27) ir redzama WebSocket satiksmes apskate Firefox DevTools. Saņemtos mērījumus identificē pēc “target: “ReceiveSensorUpdate””.



Att. 27 WebSocket satiksmes apskate

3.8 Tīmekļa lietotnes saskarņu apraksts

3.8.1 Sensoru saraksta skats

Relatīvā adrese: /sensors

Saistītā prasība: Sensor.List

Skats nodrošina visu sensoru sarakstu ar iespēju meklēt sensorus.

☰ Sākums 📄 Sensoru saraksts 🔍 Pazinojumi ⚙️ Pazinojumu noteikumi

co2					
Id	Atrašanās vieta	Aktīvs	Metadati	Pēdējie mērījumi	Darbības
402DAC		Jā	group: HVAC & Building Management Systems groupid: 1 productNumber: TDSPC003 name: Classroom 2	co2: 794 atmosphericpressure: 99770 battery: 0.39 temperature: 21.450 humidity: 29.0 rssi: -69.000000	Skatīt
10118A		Jā	group: HVAC & Building Management Systems groupid: 1 productNumber: TD5PT801 name: Sensor CO2 Sala 1	battery: 0.92 temperature: 21.300 humidity: 33.0 rssi: -71.000000	Skatīt
405B00		Jā	productNumber: TDSPC003 name: 405B00	co2: 833 atmosphericpressure: 99690 battery: 0.89 temperature: 21.550 humidity: 34.0 rssi: -78.000000	Skatīt
3002FA		Jā	productNumber: TD5PC005 group: HVAC & Building Management Systems groupid: 1 name: CO2 and Temperature IP67 sensor	co2: 707 rssi: -67.000000 battery: 0.35 temperature: 20.100 atmosphericpressure: 99750	Skatīt

Att. 28 Visu sensoru skats

3.8.2 Sensora skats

Relatīvā adrese: /sensor/:id

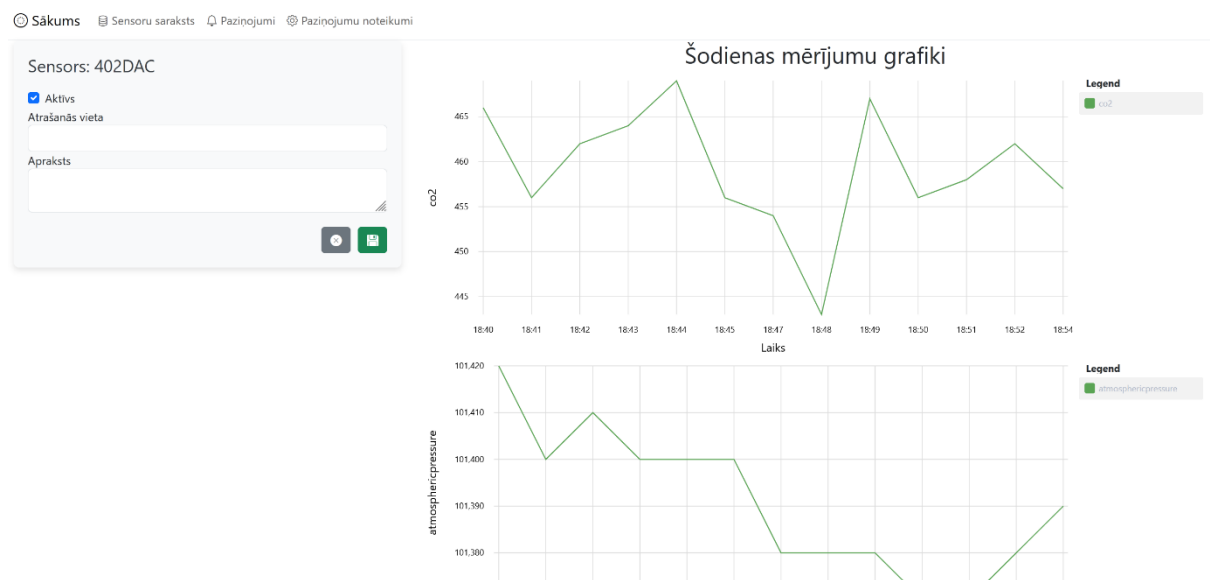
Saistītās prasības: Sensor.View, Sensor.Monitor, Sensor.Edit, Sensor.Disable.

Skats nodrošina konkrētā sensora metadatu un mērījumu apskati. Mērījumi tiek attēloti diagrammās, kur katram mērījumam ir sava diagramma. Diagrammas tiek papildinātas ar sensora reālā laika mērījumiem.



Att. 29 Konkrēta sensora skats

Papildus, skats nodrošina sensora rediģēšanu – atrašanās vieta, pieslēguma statuss.



Att. 30 Konkrēta sensora skats - rediģēšana

3.8.3 Paziņojumu saraksta skats

Relatīvā adrese: /notifications

Saistītā prasība: Notifications.List

Skats nodrošina visu paziņojumu sarakstu, kur jaunākie paziņojumi ir saraksta augšgalā. Papildus, tiek nodrošināta filtrēšanas iespēja ar filtra frāzi.

SākumsSensoru sarakstsPaziņojumiPaziņojumu noteikumi

Search...

Id	Sensors	Iemesls	Izveidots	Beidzies	Darbības
67718285425f6b3522300c7	402DAC	Parak augsts co2.Sensors 402DAC: co2 > 460	2024-12-29 19:12:12		Skatīt noteikumu
6771821d5425f6b3522300c6	405B00	Parak augsts co2.Sensors 405B00: co2 > 460	2024-12-29 19:10:28		Skatīt noteikumu
6771820b5425f6b3522300c5	402DAC	Parak augsts co2.Sensors 402DAC: co2 > 460	2024-12-29 19:10:11	2024-12-29 19:11:11	Skatīt noteikumu
6771812d5425f6b3522300c4	405B00	Parak augsts co2.Sensors 405B00: co2 > 460	2024-12-29 19:06:28	2024-12-29 19:08:28	Skatīt noteikumu
6771811b5425f6b3522300c3	402DAC	Parak augsts co2.Sensors 402DAC: co2 > 460	2024-12-29 19:06:11	2024-12-29 19:08:12	Skatīt noteikumu
67718058425f6b3522300c2	60325D	Zema baterija.Sensors 60325D: battery < 0,4	2024-12-29 19:02:56		Skatīt noteikumu
677180215425f6b3522300c1	3002FA	Zema baterija.Sensors 3002FA: battery < 0,4	2024-12-29 19:02:01		Skatīt noteikumu
6771801a5425f6b3522300c0	202843	Zema baterija.Sensors 202843: battery < 0,4	2024-12-29 19:01:54		Skatīt noteikumu
677180175425f6b3522300bf	6000FF	Zema baterija.Sensors 6000FF: battery < 0,4	2024-12-29 19:01:51		Skatīt noteikumu
677180155425f6b3522300be	500BSF	Zema baterija.Sensors 500BSF: battery < 0,4	2024-12-29 19:01:48		Skatīt noteikumu
6771800e5425f6b3522300bd	1022E0	Zema baterija.Sensors 1022E0: battery < 0,4	2024-12-29 19:01:41		Skatīt noteikumu
6771800a5425f6b3522300bc	500517	Zema baterija.Sensors 500517: battery < 0,4	2024-12-29 19:01:38		Skatīt noteikumu
677180005425f6b3522300bb	603781	Zema baterija.Sensors 603781: battery < 0,4	2024-12-29 19:01:35		

Att. 31 Paziņojumu saraksta skats

3.8.4 Paziņojumu noteikumu saraksta skats

Relatīvā adrese: /notifications/rules

Saistītās prasības: Notifications.Rules.List, Notifications.Rule.CUD

Šis skats nodrošina visu noteikumu sarakstu ar iespēju apskatīt vai dzēst konkrētu noteikumu. Noklikšķinot uz dzēšanas pogu atveras uznirstošais logs, kas prasa apstiprināt dzēšanu.

SākumsSensoru sarakstsPaziņojumiPaziņojumu noteikumi

Pievienot jaunu noteikumu

Search...

Id	Nosaukums	Sensora Id	Mērijums	Operators	Vērtība	Darbības
67717eba7e729edc5d1b136e	Parak augsts co2		co2	>	460	Skatīt Dzēst
67717fe47e729edc5d1b136f	Zema baterija		battery	<	0.4	Skatīt Dzēst

Att. 32 Paziņojumu noteikumu saraksta skats

3.8.5 Paziņojumu noteikuma skats

Relatīvā adrese: /notifications/rule/:id

Saistītās prasība: Notifications.Rules.View, Notifications.Rule.CUD

Skatā var apskatīt konkrētu paziņojumu noteikumu un rediģēt tā datus. Papildus tiek attēloti ar noteikumu saistītie paziņojumi sākot ar jaunāko.

SākumsSensoru sarakstsPaziņojumiPaziņojumu noteikumi

Noteikums: 67717eba7e729edc5d1b136e

Nosaukums

Parak augsts co2

Saistītais sensors

-

Nosacījums

co2 > 460

Paziņojumi

Search...

Id	Sensors	Iemesls	Izveidots	Beidzies
6771811b5425f6b3522300c3	402DAC	Parak augsts co2.Sensors 402DAC: co2 > 460	2024-12-29 19:06:11	2024-12-29 19:08:12
6771812d5425f6b3522300c4	405B00	Parak augsts co2.Sensors 405B00: co2 > 460	2024-12-29 19:06:28	2024-12-29 19:08:28
6771820b5425f6b3522300c5	402DAC	Parak augsts co2.Sensors 402DAC: co2 > 460	2024-12-29 19:10:11	2024-12-29 19:11:11
6771821d5425f6b3522300c6	405B00	Parak augsts co2.Sensors 405B00: co2 > 460	2024-12-29 19:10:28	

Att. 33 Paziņojumu noteikuma skats

3.8.6 Jauna paziņojumu noteikuma skats

Relatīvā adrese: /notifications/rule/new

Saistītā prasība: Notifications.Rule.CUD

Skatā var izveidot jaunu paziņojumu noteikumu. Tiek dots piemērs pareizam noteikumam. Pēc izveides novirza uz noteikuma skatu.

SākumsSensoru sarakstsPaziņojumiPaziņojumu noteikumi

Jauna noteikuma izveide

Nosaukums

Saistītais sensors (neobligāts)

Nosacījums

co2 > 900

Piemēri: "temperature > 20", "battery < 0.1"

Att. 34 Jauna noteikuma izveides skats

4 TESTĒŠANAS DOKUMENTĀCIJA

Aprakstītie testi pārbauda, vai izstrādātā sistēma realizē prasību specifikācijā noteiktās funkcionālās prasības. Testēšana tiek veikta manuāli.

Lokālās vides testēšanai tiek izmantoti katras komponentes Docker konteineri, kurus pārvalda ar Docker Compose failiem. Lokālās vides testēšanai tiek izmantoti lokāli MongoDB datubāzes, Mosquitto Docker konteineri. Būtiskākais ir nodrošināt, ka MongoDB datubāze izmanto “replica set” [26], kas nodrošina “Change Streams” funkcionalitāti.

4.1 Funkcijas “Pieslēgties MQTT brokerim” testēšana

Funkcijas identifikators: Mqtt.Connect

1. Pārbaudīt, vai tiek izveidots savienojums ar MQTT brokeri.	
Soļi	
1. Konfigurē “Sensor.Consumer” priekš savienojuma ar zināmu MQTT brokeri. 2. Palaist “Sensor.Consumer”.	
Sagaidāmais rezultāts	
Žurnālieraksts par veiksmīgi izveidotu savienojumu.	
Reālais rezultāts	
2024-12-26 20:05:32 info: SensorConsumer.Workers.MqttWorker[0] 2024-12-26 20:05:32 Connecting to MQTT broker: sensor-mqtt-local 2024-12-26 20:05:32 info: SensorConsumer.Workers.MqttWorker[0] 2024-12-26 20:05:32 Connected to MQTT broker 2024-12-26 20:05:32 info: SensorConsumer.Workers.MqttWorker[0] 2024-12-26 20:05:32 Subscribed to topic: Aranetest/+/sensors/+/json/measurements 2024-12-26 20:05:32 info: SensorConsumer.Workers.MqttWorker[0] 2024-12-26 20:05:32 Subscribed to topic: Aranetest/+/sensors/+/+ 2024-12-26 20:05:32 info: SensorConsumer.Workers.MqttWorker[0] 2024-12-26 20:05:32 Subscribed to topics 2024-12-26 20:05:32 info: SensorConsumer.Workers.MqttWorker[0] 2024-12-26 20:05:32 MQTT connection is alive.	
Vai tests izpildīts veiksmīgi?	Jā (2024.12.26)

2. Pārbaudīt, vai savienojumus tiek atjaunots, ja tas ir pārtrūcis.	
Soļi	
1. Palaist “Sensor.Consumer”. 2. Izslēgt MQTT brokeri. 3. Sagaidīt žurnālierakstu par pārtrūkušu savienojumu.	

4. Ieslēgt MQTT brokeri	
Sagaidāmais rezultāts	
1. Žurnālieraksts par pārtrūkušu savienojumu. 2. Žurnālieraksts par atjaunotu savienojumu.	
Reālais rezultāts	
1. 2024-12-26 20:10:16 warn: SensorConsumer.Workers.MqttWorker[0] 2024-12-26 20:10:16 MQTT connection is not alive. Attempting to reconnect. 2024-12-26 20:10:19 fail: SensorConsumer.Workers.MqttWorker[0] 2024-12-26 20:10:19 An error occurred on connect attempt 0. 2. 2024-12-26 20:10:31 info: SensorConsumer.Workers.MqttWorker[0] 2024-12-26 20:10:31 Connecting to MQTT broker: sensor-mqtt-local 2024-12-26 20:10:31 info: SensorConsumer.Workers.MqttWorker[0] 2024-12-26 20:10:31 Connected to MQTT broker	
Vai tests izpildīts veiksmīgi?	Jā (2024.12.26)

4.2 Funkcijas “Klausīties sensorus” testēšana

Funkcijas identifikators: Mqtt.Listen

1. Pārbaudīt, vai tiek žurnālēts ziņojuma saņemšanas fakts un dati tiek saglabāti.
Soļi
1. Konfigurē MQTT abonementa tēmu: a. Test/<sensorId>/<metadataName> 2. Palaiž “Sensor.Consumer”. 3. Nosūta MQTT brokerim ziņojumu uz tēmu “Test/SNS12/name” ar saturu “Sensors nosaukums”.
Sagaidāmais rezultāts
1. Žurnālieraksts par saņemtu ziņojumu. 2. Datubāzē sensors ar identifikatoru “SNS12” un atribūtu “name” kā “Sensors nosaukums”.
Reālais rezultāts
1. 2024-12-26 21:04:33 info: SensorConsumer.Workers.MqttWorker[0] 2024-12-26 21:04:33 => Message number 0, Topic: Test/SNS12/name 2024-12-26 21:04:33 Received message. Message count: 0 2. Kolekcijā “sensors”:

<pre>{ "_id": "SNS12", "topics": ["Test/SNS12/name"], "metadata": { "name": "Sensors nosaukums" } }</pre>	
Vai tests izpildīts veiksmīgi?	Jā (2024.12.26)

2. Pārbaudīt, vai tiek izlaisti atslēgto sensoru ziņojumi.	
Soļi	
- Konfigurē MQTT abonementa tēmu: - Test/<sensorId>/<metadataName> - Palaiž "Sensor.Consumer". - Nosūta MQTT brokerim ziņojumu uz tēmu "Test/SNS12/name" ar saturu "Sensors nosaukums". - Datubāzē iestata konkrētajam sensoram atribūtu "isActive" kā false. - Nosūta MQTT brokerim ziņojumu uz tēmu "Test/SNS12/name" ar saturu "Cits nosaukums".	
Sagaidāmais rezultāts	
Datubāzē sensors ar identifikatoru "SNS12" un atribūtu "name" kā "Sensors nosaukums".	
Reālais rezultāts	
Kolekcijā "sensors": <pre>{ "_id": "SNS12", "topics": ["Test/SNS12/name"], "metadata": { "name": "Sensors nosaukums" }, "isActive": false }</pre> Saistītie žurnālieraksti: <pre>2024-12-26 22:19:35 info: SensorConsumer.Data.InactiveSensorCache[0] 2024-12-26 22:19:35 Sensor SNS12 is now inactive. 2024-12-26 22:22:49 info: SensorConsumer.Services.ProcessingService[0] 2024-12-26 22:22:49 => Message number 1, Topic: Test/SNS12/name 2024-12-26 22:22:49 Sensor with ID 'SNS12' is inactive. Skipping message processing.</pre>	
Vai tests izpildīts veiksmīgi?	Jā (2024.12.26)

4.3 Funkcijas “Apstrādāt ziņojumu” testēšana

Funkcijas identifikators: Mqtt.HandleMessage

1. Pārbaudīt, vai tiek saglabāti sensoru metadati un mērījumi.
Soļi
1. Palaiž “Sensor.Consumer” ar konfigurētu MQTT abonementa tēmu: a. Test/<sensorId>/measurements b. Test/<sensorId>/<metadataName> 2. Nosūta MQTT brokerim metadatu ziņojumus: a. Uz tēmu “Test/SNS413/color” ar saturu “Rozā”. b. Uz tēmu “Test/SNS413/room” ar saturu “315.t”. 3. Nosūta MQTT brokerim mērījumu ziņojumus uz tēmu “Test/SNS413/measurements” ar saturu: a. {"co2": "200", "cars": "20", "time": "1735247181"} b. {"co2": "300", "cars": "25", "time": "1735247191"} c. {"co2": "200", "cars": "15", "time": "1735247281"}
Sagaidāmais rezultāts
1. Datubāzes kolekcijā “sensors” sensors ar identifikatoru “SNS413” satur atribūtus: a. “color” ar vērtību “Rozā”. b. “room” ar vērtību “315.t”. 2. Kolekcijā “sensorMeasurements” satur 3 dokumentus ar: a. Sensora identifikatoru “SNS413”. b. “timestamp” vērtībām atbilstoši iesūtītajām. c. “measurements” saturu atbilstoši iesūtītajam.
Reālais rezultāts
1. Kolekcijā “sensors”<pre>{ "_id": "SNS413", "topics": ["Test/SNS413/color", "Test/SNS413/room", "Test/SNS413/measurements"], "metadata": { "color": "Rozā", "room": "315.t" }, "latestMeasurementTimestamp": { "\$date": "2024-12-26T21:08:01.000Z" }, }</pre>

<pre> "latestMeasurements": { "co2": "200", "cars": "15" } } </pre>	
2. Kolekcijā “sensorMeasurements”	
1.	
<pre> { "_id": { "\$oid": "677589a2ef794b88361a8084" }, "timestamp": { "\$date": "2024-12-26T21:08:01.000Z" }, "sensorId": "SNS413", "measurements": { "co2": "200", "cars": "15" } } </pre>	
2.	
<pre> { "_id": { "\$oid": "67758978ef794b88361a8083" }, "timestamp": { "\$date": "2024-12-26T21:06:31.000Z" }, "sensorId": "SNS413", "measurements": { "co2": "300", "cars": "25" } } </pre>	
3.	
<pre> { "_id": { "\$oid": "6775893cef794b88361a8082" }, "timestamp": { "\$date": "2024-12-26T21:06:21.000Z" }, "sensorId": "SNS413", "measurements": { "co2": "200", "cars": "20" } } </pre>	
Vai tests izpildīts veiksmīgi?	Jā (2024.12.26)

2. Pārbaudīt, vai tiek saglabāta metadatu izmaiņu vēsture.
Soļi
1. Palaiž “Sensor.Consumer” ar konfigurētu MQTT abonementa tēmu: Test/<sensorId>/<metadataName>

2. Nosūta MQTT brokerim metadatu ziņojumus: a. Uz tēmu “Test/SNS413/name” ar saturu “Nosaukums1”. b. Uz tēmu “Test/SNS413/name” ar saturu “Labāks nosaukums2”.	
Sagaidāmais rezultāts	
1. Kolekcijā “sensorMetadata” ir dokuments: a. Sensora identifikators “SNS413” b. “metadata” satur atslēgu “name” un vērtību “Nosaukums1”. 2. Kolekcijā “sensors” sensors ar identifikatoru “SNS413” satur atribūtu: a. “name” ar vērtību “Labāks nosaukums2”.	
Reālais rezultāts	
1. Kolekcijā “sensorMetadata” <pre> 1. { "_id": { "\$oid": "67758b2def794b88361a8085" }, "timestamp": { "\$date": "2025-01-01T18:36:29.581Z" }, "sensorId": "SNS413", "metadata": { "name": "Nosaukums1" } } 2. { "_id": { "\$oid": "67758b34ef794b88361a8086" }, "timestamp": { "\$date": "2025-01-01T18:36:36.988Z" }, "sensorId": "SNS413", "metadata": { "name": "Labāks nosaukums2" } } </pre> 2. Kolekcijā “sensors” <pre> { "_id": "SNS413", "topics": ["Test/SNS413/name"], "metadata": { "name": "Labāks nosaukums2" } } </pre>	
Vai tests izpildīts veiksmīgi?	Jā (2025.01.01)

4.4 Funkcijas “Iegūt sensoru sarakstu” testēšana

Funkcijas identifikators: Sensor.List

1. Pārbaudīt, vai sensoru saraksta skats parāda pilnu sensoru sarakstu.	
Soļi	
1. Atver skatu “Sensoru saraksta skats”.	
Sagaidāmais rezultāts	
1. Tiek parādīti visi sistēmā saglabātie sensori. 2. Sensoru saraksts satur funkcijas “Sensor.List” izvaddatus kā saraksta laukus.	
Vai tests izpildīts veiksmīgi?	Jā (2025.01.01)

2. Pārbaudīt, vai ievadot filtra frāzi, tiek atgriezti tikai tie sensori, kuru datos ir minēta frāze.	
Soļi	
1. Atver skatu “Sensoru saraksta skats”. 2. Filtra laukā ievada frāzi. Pamēģina vairākas dažādas.	
Sagaidāmais rezultāts	
1. Tiek parādīts saīsināts sensoru saraksts, kur katram sensoram datu rindā ir atrodama ievadītā filtra frāze.	
Vai tests izpildīts veiksmīgi?	Jā (2025.01.01)

4.5 Funkcijas “Apskatīt sensoru” testēšana

Funkcijas identifikators: Sensor.View

3. Pārbaudīt, vai sistēma rāda konkrēta sensora metadatus un šīs dienas mērījumus.	
Soļi	
1. Atver skatu “Sensora skats”. 2. Apskata konkrētā sensora datus kolekcijā “sensors”, lai salīdzinātu.	
Sagaidāmais rezultāts	
1. Tiek parādīti prasībai atbilstošie sensora dati: a. Sensor identifikators b. Atrašanās vieta	

c. Vai ir atslēgts. d. Metadati e. Šodienas mērījumi, kur katra metrika ir savā līnijdiagrammā.	
2. Vai ir poga, kas nodrošina iespēju atvērt sensora rediģēšanu.	
Vai tests izpildīts veiksmīgi?	Jā (2025.01.01)

4.6 Funkcijas “Sensoru mērījumu monitorings” testēšana

Funkcijas identifikators: Sensor.Monitor

1. Pārbaudīt, vai mērījumu diagrammas tiek papildinātas ar reālā laika mērījumiem.	
Soļi	
1. Palaiž “Sensor.Consumer” ar konfigurētu MQTT abonementa tēmu: a. Test/<sensorId>/measurements 2. Nosūta MQTT brokerim mērījumu ziņojumus uz tēmu “Test/SNS413/measurements” ar saturu: a. {“co2”: “200”, “temp”: “20”, “time”: “šī brīža laiks Unix laika formātā”} 3. Atver skatu “Sensora skats” sensoram “SNS413”. 4. Nosūta MQTT brokerim mērījumu ziņojumus uz tēmu “Test/SNS413/measurements” ar saturu: a. {"co2": "400", "temp": "30", "time": "<šī brīža laiks Unix laika formātā>"} b. {"co2": "300", "temp": "25", "time": "<šī brīža laiks Unix laika formātā>"} c. {"co2": "200", "temp": "15", "time": "<šī brīža laiks Unix laika formātā>"} 5. Starp mērījumu ziņojumu nosūtījumiem novēro vai atjauninās mērījumu diagrammas.	
Sagaidāmais rezultāts	
1. Mērījumu diagrammas tiek atjauninātas bez pilnas skata atsvaidzināšanas ar ziņojumos norādītajiem mērījumiem. a. “co2” diagrammā ir 3 jauni mērījumi 400, 300 un 200. b. “temp” diagrammā ir 3 jauni mērījumi 30, 25, 15.	
Vai tests izpildīts veiksmīgi?	Jā (2025.01.01)

4.7 Funkcijas “Rediģēt sensora atrašanās vietu” testēšana

Funkcijas identifikators: Sensor.Edit

1. Pārbaudīt, vai tiek saglabāta sensora jaunā atrašanās vieta un vecā saglabāta vēsturē.	
Soļi	
1. Atver skatu “Sensora skats” un atver rediģēšanu. 2. Saglabā jaunu sensora atrašanās vietu “Vecā vieta1”. 3. Saglabā jaunu sensora atrašanās vietu “Jaunā vieta1”.	
Sagaidāmais rezultāts	
1. Sensora skatā ir redzama jaunā atrašanās vieta “Jaunā vieta1”. 2. Kolekcijā “sensors” konkrētajam sensoram lauks “location” ir ar vērtību “Jaunā vieta1”. 3. Kolekcijā “sensorMetadata” ir ieraksts ar konkrētā sensora ID un lauku “metadata” kā “{“location”: “Vecā vieta1”}”.	
Vai tests izpildīts veiksmīgi?	Jā (2025.01.01)

4.8 Funkcijas “Atslēgt/pieslēgt sensora uztveršanu” testēšana

Funkcijas identifikators: Sensor.Disable

1. Pārbaudīt, vai tiek mainot pieslēguma statusu sensors vairs netiek uztverts.	
Soļi	
1. Palaiž “Sensor.Consumer” ar konfigurētu MQTT abonementa tēmu: a. Test/<sensorId>/measurements 2. Nosūta MQTT brokerim pirmo ziņojumu uz tēmu “Test/SNS413/measurements” ar saturu: {"co2": "200", "temp": "20", "time": "<šī brīža laiks Unix laika formātā>"} 3. Pārbauda kolekcijā “sensorMeasurements”, ka ziņojums tika saņemts un mērījums saglabāts. 4. Atver skatu “Sensora skats” un atver rediģēšanu. 5. Nomaina pieslēguma statusu uz atslēgts. 6. Nosūta MQTT brokerim otro ziņojumu uz tēmu “Test/SNS413/measurements” ar saturu: {"co2": "300", "temp": "30", "time": "<šī brīža laiks Unix laika formātā>"} 7. Pārbauda kolekcijā “sensorMeasurements”, ka mērījums netika saglabāts.	

8. Atver skatu “Sensors skats” un atver rediģēšanu. 9. Nomaina pieslēguma statusu uz pieslēgts. 10. Nosūta MQTT brokerim trešo ziņojumu uz tēmu “Test/SNS413/measurements” ar saturu: {"co2": "500", "temp": "15", "time": "<ši brīža laiks Unix laika formātā>"} 11. Pārbauda, ka mērījums tika saglabāts kolekcijā “sensorMeasurements”.	
Sagaidāmais rezultāts	
1. Kolekcijā “sensorMeasurements” ir divi ieraksti ar “sensorId” kā “SNS413” ar lauka “measurements” vērtībām no pirmā un trešā ziņojuma.	
Vai tests izpildīts veiksmīgi?	Jā (2025.01.01)

4.9 Funkcijas “Eksportēt mērījumus” testēšana

Funkcijas identifikators: Export.Measurements

1. Pārbaudīt, vai tiek eksportēti mērījumi.
Soļi
1. Palaiž “Sensor.Consumer” ar konfigurētu MQTT abonementa tēmu: a. Test/<sensorId>/measurements 2. Nosūta MQTT brokerim ziņojumu uz tēmu “Test/SNS413/measurements” ar saturu: {"co2": "200", "temp": "20", "time": "1735596762"} 3. Sagaida eksporta partijas izpildi. 4. Sagaida nākamo eksporta partijas izpildi. 5. Nosūta MQTT brokerim ziņojumu uz tēmu “Test/SNS222/measurements” ar saturu: {"co2": "400", "temp": "25", "time": "1735596776"}
Sagaidāmais rezultāts
1. Pēc pirmās partijas SQL datubāzē ir divi mērījumu tabulā “SensorMeasurements” ar vērtībām: a. SensorId = SNS413 b. Pirmajam ierakstam: i. MetricKey = co2 ii. MetricValue = 200 iii. Timestamp = 1735596762 c. Otrajam ierakstam i. MetricKey = temp

ii. MetricValue = 20 iii. Timestamp = 1735596762	
2. Pēc otrās partijas nav neviens ieraksts saglabāts un programma turpina veiksmīgi strādāt.	
3. Pēc trešās partijas SQL datubāzē ir vēl divi mērījumu tabulā “SensorMeasurements” ar vērtībām:	
a. SensorId = SNS222	
b. Pirmajam ierakstam:	
i. MetricKey = co2	
ii. MetricValue = 400	
iii. Timestamp = 1735596776	
c. Otrajam ierakstam	
i. MetricKey = temp	
ii. MetricValue = 25	
iii. Timestamp = 1735596776	
Vai tests izpildīts veiksmīgi?	Jā (2025.01.01)

4.10 Funkcijas “Eksportēt metadatus” testēšana

Funkcijas identifikators: Export.Metadata

1. Pārbaudīt, vai tiek eksportēti metadati.
Soļi
1. Palaiž “Sensor.Consumer” ar konfigurētu MQTT abonementa tēmu: a. Test/<sensorId>/<metadataName> 2. Nosūta MQTT brokerim ziņojumu uz tēmu “Test/SNS413/name” ar saturu: “Nosaukums1”. 3. Sagaida eksporta partijas izpildi. 4. Sagaida nākamo eksporta partijas izpildi. 5. Nosūta MQTT brokerim ziņojumu uz tēmu “Test/SNS222/name” ar saturu: “Jauns nosaukums2”.
Sagaidāmais rezultāts
1. Pēc pirmās partijas SQL datubāzē ir ieraksts tabulā “SensorMetadata” ar vērtībām: a. SensorId = SNS413

b. MetaKey = Name c. MetaValue = Nosaukums1	
2. Pēc otrās partijas nav neviens ieraksts saglabāts un programma turpina veiksmīgi strādāt.	
3. Pēc trešās partijas SQL datubāzē ir vēl viens ieraksts tabulā “SensorMetadata” ar vērtībām:	
a. SensorId = SNS413 b. MetaKey = Name c. MetaValue = Jauns nosaukums2	
Vai tests izpildīts veiksmīgi?	Jā (2025.01.01)

4.11 Funkcijas “Atjaunot parametrus” testēšana

Funkcijas identifikators: Export.Parameters

2. Pārbaudīt, vai tiek izmantoti izmainīti eksporta parametri.
Soļi
1. Kolekcijās “sensorMeasurements” un “sensorMetadata” katrā ir vismaz 14 ieraksti. 2. Iestata eksporta parametrus: a. BatchSize = 10 b. BatchDelaySeconds = 60 3. Sagaida vienas eksporta partijas izpildi. 4. Pārbauda vai attiecīgo 10 dokumentu saturs no katras kolekcijas ir saglabāts SQL datubāzē. 5. Iestata eksporta parametrus: a. BatchSize = 2 b. BatchDelaySeconds = 30 6. Sagaida vienas eksporta partijas izpildi un pārlicinās vai pagāja 60 sekundes. 7. Pārbauda vai attiecīgo nākamo 2 dokumentu saturs no katras kolekcijas ir saglabāts SQL datubāzē. 8. Sagaida nākamās eksporta partijas izpildi un pārlicinās vai pagāja 30 sekundes. 9. Pārbauda vai attiecīgo nākamo 2 dokumentu saturs no katras kolekcijas ir saglabāts SQL datubāzē.
Sagaidāmais rezultāts

1. Pēc pirmās partijas SQL datubāzē ir 10 MongoDB datubāzes dokumentu saturs katrā tabulā “SensorMeasurements” un “SensorMetadata”. 2. Pēc pirmās partijas tiek gaidīts 60 sekundes. 3. Pēc otrās partijas SQL datubāzē ir nākamo 2 dokumentu saturs katrā tabulā. 4. Pēc otrās partijas tiek gaidīts 30 sekundes. 5. Pēc trešās partijas SQL datubāzē ir nākamo 2 dokumentu saturs katrā tabulā.	
Vai tests izpildīts veiksmīgi?	Jā (2025.01.01)

4.12 Funkcijas “Pārvaldīt noteikumus” testēšana

Funkcijas identifikators: Notifications.Rule.CUD

1. Pārbaudīt, vai tiek izveidots paziņojuma noteikums.	
Soļi	
1. Atver skatu “Jauna paziņojumu noteikuma skats”. 2. Izveido paziņojumu.	
Sagaidāmais rezultāts	
1. Novirzīts uz paziņojuma skatu ar izveidoto paziņojumu. 2. Kolekcijā “notificationRules” ir jaunais paziņojums.	
Vai tests izpildīts veiksmīgi?	Jā (2025.01.01)

2. Pārbaudīt, vai tiek rediģēts paziņojuma noteikums.	
Soļi	
1. Palaiž “Sensor.Consumer” ar konfigurētu MQTT abonementa tēmu: a. Test/<sensorId>/measurements 2. Izveido noteikumu ar nosacījumu: “co2 > 900”. 3. Atver skatu “Paziņojumu noteikuma skats” un atver rediģēšanu. 4. Maina nosacījumu uz “co2 < 900”. 5. Iesniedz noteikuma izmaiņas. 6. Nosūta MQTT brokerim ziņojumus uz tēmu “Test/SNS413/measurements” ar saturu: a. {"co2": "1100", "temp": "20", "time": "1735596762"} b. {"co2": "800", "temp": "20", "time": "1735596862"}	
Sagaidāmais rezultāts	

1. Novirzīts uz noteikuma skatu ar rediģēto noteikumu un noteikumam ir jaunās vērtības. 2. Kolekcijā “notificationRules” noteikumam ir jaunās rediģēšanas rezultātā iesūtītās vērtības. 3. Ir izveidots paziņojums, kas liecina, ka ir aktivizēts nosacījums “co2 < 900” un nevis “co2 > 900”.	
Vai tests izpildīts veiksmīgi?	Jā (2025.01.01)

3. Pārbaudīt, vai tiek dzēsts paziņojuma noteikums.	
Soļi	
1. Atver skatu “Paziņojumu noteikuma skats”. 2. Spiež dzēšanas pogu.	
Sagaidāmais rezultāts	
1. Noteikums nav vairs atrodams visu noteikumu saraksta skatā. 2. Noteikums nav vairs atrodams kolekcijā “notificationRules”.	
Vai tests izpildīts veiksmīgi?	Jā (2025.01.01)

4.13 Funkcijas “Ģenerēt paziņojumus” testēšana

Funkcijas identifikators: Notifications.Generate

1. Pārbaudīt, vai netiek lieki izveidots atkārtots paziņojums.	
Soļi	
1. Nodrošina, ka eksistē noteikums ar nosacījumu “co2 > 900”. 2. Palaiž “Sensor.Consumer” ar konfigurētu MQTT abonementa tēmu: a. Test/<sensorId>/measurements 3. Nosūta MQTT brokerim ziņojumus uz tēmu “Test/SNS413/measurements” ar saturu: a. Pirmais ziņojums: {“co2”: “700”, “temp”: “20”, “time”: “1735596762”} b. Otrais ziņojums: {“co2”: “1000”, “temp”: “20”, “time”: “1735596862”} c. Trešais ziņojums: {“co2”: “1200”, “temp”: “20”, “time”: “1735596962”} d. Ceturtais ziņojums: {“co2”: “700”, “temp”: “20”, “time”: “1735597762”} e. Piektais ziņojums: {“co2”: “1000”, “temp”: “20”, “time”: “1735598762”} f. Sestais ziņojums: {“co2”: “1300”, “temp”: “20”, “time”: “1735598762”}	

Sagaidāmais rezultāts	
1. Kolekcijā “notifications” vai paziņojumu sarakstā ir 2 paziņojumi. 2. Pirmais paziņojums: a. Lauks “startTimestamp” atbilst Unix laikam “1735596862”. b. Lauks “endTimestamp” atbilst Unix laikam “1735597762”. 3. Otrais paziņojums: a. Lauks “startTimestamp” atbilst Unix laikam “1735598762”. b. Lauks “endTimestamp” ir “null”.	
Vai tests izpildīts veiksmīgi?	Jā (2025.01.01)

2. Pārbaudīt, vai paziņojums tiek izveidots atbilstoši tikai noteikuma sensoram.	
Soļi	
1. Nodrošina, ka eksistē noteikums ar nosacījumu “co2 > 900” un “sensorId” kā “SNS413”. 2. Palaiž “Sensor.Consumer” ar konfigurētu MQTT abonementa tēmu: a. Test/<sensorId>/measurements 3. Nosūta MQTT brokerim ziņojumus uz tēmu “Test/SNS413/measurements” ar saturu: {“co2”: “1000”, “temp”: “20”, “time”: “1735596862”}. 4. Nosūta MQTT brokerim ziņojumus uz tēmu “Test/SNS222/measurements” ar saturu: {“co2”: “1200”, “temp”: “20”, “time”: “1735596962”}.	
Sagaidāmais rezultāts	
1. Kolekcijā “notifications” vai paziņojumu sarakstā ir viens paziņojums, kura lauks “sensorId” atbilst “SNS413”.	
Vai tests izpildīts veiksmīgi?	Jā (2025.01.01)

5 PROJEKTA ORGANIZĀCIJA

Sistēma tika izstrādāta iteratīvi un inkrementāli, kas nozīmē, ka prasības tika pakāpeniski precizētas un papildinātas ar katru izstrādāto versiju. Projekts sākās ar sākotnēju ideju par sistēmu, kurā būtu divi galvenie komponenti: patērētāju, kas uztver sensoru datus, un mājaslapu, kas nodrošina reāllaika mērījumu vizualizāciju, kā arī vienu mazāk svarīgu komponentu – datu eksportu uz SQL datubāzi. Primārais mērķis izveidot pēc iespējas ātrāk versiju priekš Aranet sensoru mērījumu uztveršanas Raiņa bulvārī 19 un to datu saglabāšanu.

Pirmā versija tika veidota, balstoties uz publiski pieejamajiem Aranet sensoriem MQTT brokerī “broker.hivemq.com”, un sākotnēji izmantoja SQL datubāzi. Īsi pēc šīs realizācijas tika pieņemts lēmums pāriet uz Mongo datubāzi, ņemot vērā tās elastīgumu un piemērotību sensoru datu plūsmām.

Pēc sākotnējās versijas izstrādes (v1.0.0), kas fokusējās uz minimālās dzīvotspējīgās sistēmas izveidi Aranet sensoru mērījumu uztveršanai un to datu saglabāšanai, sistēmas prasības tika paplašinātas. Otrajā versijā tika izstrādāts modulis “Sensoru panelis”, kas nodrošināja reāllaika mērījumu vizualizāciju mājaslapā. Pēc šīs versijas tika uzstādīts Mosquitto brokeris Raiņa bulvārī 19, kas savienojās ar ēkā esošajiem Aranet sensoriem, un tika uzstādīta minimālā versija, kas uztver un saglabā sensoru datus.

Nākamajā posmā tika ieviests “Shēmu modulis”, kura mērķis bija nodrošināt sistēmai spēju uztvert un apstrādāt dažādu sensoru datus neatkarīgi no datu formāta un MQTT tēmas uzbūves. Tomēr, sākoties darbam pie “Datu eksporta moduļa” un sensoru pārvaldības funkcionalitātes, kļuva skaidrs, ka “Shēmu modulis” ievieš pārmērīgu sarežģītību. Tas sadalīja sensoru datus vairākās kolekcijās atkarībā no sensoru tipa, kas apgrūtināja datu pārvaldību un jaunu funkcionalitāšu izstrādi. Turklāt modulim nebija tieša pielietojuma, jo bija pieejami tikai viena veida sensori (Aranet), līdz ar to netika izstrādāts precīzi.

Lai mazinātu sarežģītību un veicinātu sistēmas attīstību, tika pieņemts lēmums atteikties no “Shēmu moduļa” pilnveides un labošanas. Vēlāk tika pielietots “Shēmu moduļa” izstrādē iegūtais, lai nodrošinātu minimālu elastību, kas ir aprakstīts “Atbalstītie MQTT ziņojumi”. Turpmākā izstrāde tika veikta, atgriežoties pie sistēmas versijas pirms “Shēmu moduļa” ieviešanas, prioritizējot datu eksportu un sensoru pārvaldības iespējas.

Nākamais paplašinājums (v2.0.0) fokusējās uz “Datu eksporta moduļa” izstrādi, kas nodrošināja datu sinhronizāciju ar dažādām SQL datubāzēm, un sensoru pārvaldības

funkcionalitātes pilnveidi. Visbeidzot tika izstrādāta versija (v2.1.0) ar moduli “Paziņojumi”, kas nodrošina paziņojumu veidošanu pēc lietotāja definētiem noteikumiem.

Testēšana izstrādes laikā bija manuāla un mērķēta. Piemēram, lai pārbaudītu vai sistēma spēj uztvert sensorus un saglabāt to datus, tika testēts sūtīt datus uz savienoto brokeri un pārbaudot vai sagaidāmie dati parādās datubāzē.

5.1 Konfigurācijas pārvaldība

Tika izmantota versiju kontroles sistēma Git un GitHub platforma. Katrai projektējuma aprakstītā komponente tika glabāta savā repozitorijā un ir publiski pieejama [27] [28] [29] [30] [31].

.NET projekta nosaukumi satur prefiksu “Sensor”, piemēram “SensorConsumer”. Ja projekts satur apakšprojektus, tad to nosaukums satur projekta nosaukumu un apakšprojekta nosaukumu atdalītu ar punktu, piemēram “SensorCourier.MySql”.

GitHub repozitoriju nosaukumi un Docker konteineru nosaukumi seko shēmai “sensor-
<projekta nosaukums>”, piemēram “sensor-consumer”. Izstrādes vides konteineri satur paplašinājumu “...-dev”.

Tika izveidoti Docker compose faili, kas nodrošināja izstrādes (dev) vidi, gatavošanās (staging) vidi un produkcijas vidi. Katrai videi tika izveidoti atsevišķas datubāzes arī ar Docker compose. Izstrādes vides compose faili nodrošināja konteineru izveidi uz esošo kodu, kas ir apskatāmi komponentu GitHub repozitorijos mapē “.build”. “Staging” vide bija pēc iespējas tuva produkcijas videi, izmantojot tos pašu parametrus un pieslēgumu tam pašam MQTT brokerim. “Staging” vidē tika veikti manuālie testi pirms versija tika uzstādīta produkcijas vidē. Piegādē uz produkcijas serveri tika izveidoti konteineru “.taz” faili, kas ar SSH savienojumu tika nogādāti uz produkcijas serveri kopā ar “compose.yaml” failiem, kur tie tika palaisti manuāli izmantojot Docker compose komandas.

Sākot ar “Paziņojumu moduli” tika ievērots stils veidot paplašinājumus atsevišķos zaros, piemēram “feature/notifications”, kas ir vienīgais šāds atzars.

Versiju kontrolē bija centieni izmantot semantiskās verzionēšanas ieteikumus [32]. Tika pielietots verzionēšanas modelis <x>.<y>.<z>, kur x versija apzīmē būtiskas izmaiņas visā sistēmā, y versija apzīmē būtiskas izmaiņas komponentē nepieprasot izmaiņas citās

komponentēs, un z versija apzīmē mazas izmaiņas komponentē. Vienīgais izņēmums y versijai bija tīmekļa lapas komponentes “Sensor.WebApi”, “Sensor.Dashboard”, kuras tika verzionētas kopā.

Versiju kontrole tika papildināta ar “CHANGELOG.md” [33] failiem, kas tika ieviesti komponentēm pēc tās pirmās versijas. Tie palīdz izsekot, kas katrā versijā tika ieviests un kādas ir neizlaistās versijas izmaiņas.

5.2 Kvalitātes nodrošināšana

Programmatūras izstrādē tika ievērots vienas atbildības princips, pēc kura kods tika dalīts tā, lai katrai klasei, metodei vai koda kopumam būtu viena atbildība. Piemēram, viena klase uztver MQTT ziņojumus, otrā klase apstrādā ziņojumus, trešā klase nodrošina datu saglabāšanu. Trešajai klasei nav nekādas informācijas par datu izcelsmi, kad padara kodu vieglāk maināmu un skaidrāku.

Lai standartizētu koda stilu un formatējumu, tika izmantots “.editorconfig” fails [29]. Tas ļauj definēt, piemēram, atstarpju izmēru, mainīgo nosaukumu standartus un citus ar koda rakstīšanas saistītos stila noteikumus.

Kodā tiek bieži izmantota žurnālēšana, lai nodrošinātu vieglāku diagnosticēšanu problēmu gadījumā. Žurnālēšana arī papildina komentārus, jo tie arī sniedz informāciju par programmas darbību, palīdzot saprast kodu.

Koda izstrādes procesā tika izmantota šāda dokumentācija un vadlīnijas:

- Microsoft .NET dokumentācija [35] un stila vadlīnijas.
- Angular dokumentācija [36] un stila vadlīnijas.
- MongoDB dokumentācija [37] datubāzes izveidē un ar datubāzi saistīta koda rakstīšanu.

Dokumenta kvalitātes nodrošināšanai tika izmantoti LVS 68:1996 un LVS 72:1996 standarti.

5.3 Darbietilpības novērtējums

Darbietilpības novērtējums veikts atsaucoties uz QSM (Quantitative Software Management) resursiem. Etalon-tabula “Business Systems Implementation Unit (New and Modified IU) Benchmarks” [38] pirmajā kvartilē ir projekti, kas vidēji ilguši 3,2 mēnešus un

tos izstrādājuši vidēji 1,57 cilvēki. Šāda lieluma projektiem mediāna ir 1889 programmas koda loģiskās rindīņas.

Programmas loģisko rindīņu noteikšanai tika izmantots “Visual Studio Code” paplašinājums “VS Code Counter” [39]. Tiek skaitītas koda rindīņas, neieskaitot komentētās un tukšās rindīņas.

Kopumā izstrādātas aptuveni 4000 koda rindīņas, kas pārsniedz 3 personmēnešu darba izstrādes mediānu.

5.3.1 tabula **Koda rindīņu skaits**

Komponente	Rindīņu skaits
Sensor.Consumer	732
Sensor.WebApi	790
Sensor.Notifications	336
Sensor.Courier	760
Sensor.Dashboard	1476
Kopā	4094

SECINĀJUMI

Kvalifikācijas darba ietvaros tika izstrādāta “Sensoru monitoringa sistēma” un tās dokumentācija, kas tika veidota pēc darba autora iniciatīvas.

Darba izstrādes procesā tika uzlabotas esošās zināšanas un iegūtas jaunas prasmes vairākās jomās. Tika padziļināta izpratne par programmēšanu C# un .NET, kā arī par tīmekļa lietotņu veidošanu, izmantojot TypeScript, Angular. Turklāt tika iepazīts un apgūts MongoDB. Zināšanas arī tika paplašinātas konteinerizācijas jomā, darbojoties ar Docker un Docker Compose. Kā arī tika iegūtas zināšanas darbā ar MQTT un Mosquitto brokera uzstādīšanā un iemaņas Nginx uzstādīšanā priekš tīmekļa lietotnes priekšgalsistēmas. Papildus, tika nostiprinātas iemaņas Linux operētājsistēmas izmantošanā kā produkcijas serveris.

Rezultātā tika realizētas dokumentā aprakstītās prasības un izveidota sistēma, kas uzrauga Raiņa bulvāra 19 sensorus, nodrošinot datu saglabāšanu, datu eksporta iespēju uz SQL datubāzi, kā arī administratora tīmekļa lietotne sensoru un paziņojumu pārvaldībai un mērījumu reālā laika apskatei.

Sistēmas izstrāde ir bijusi vērtīga pieredze, kas lika iedziļināties programmatūras izstrādes procesā.

IZMANTOTĀ LITERATŪRA UN AVOTI

- [1] «Aranet» Pieejams: <https://aranet.com/en/home>. [Piekļūts 29.12.2024].
- [2] «MQTT - The Standard for IoT Messaging» Pieejams: <https://mqtt.org/>. [Piekļūts 29.12.2024].
- [3] «Aranet MQTT functionality and integration with Amazon AWS» Pieejams: <https://management.saftehnika.com/pub/Aranet%20MQTT%20functionality%20and%20integration%20with%20Amazon%20AWS%20v2.0.pdf>. [Piekļūts 29.12.2024].
- [4] «nginx» Pieejams: <https://nginx.org/en/>. [Piekļūts 29.12.2024].
- [5] «Eclipse Mosquitto» Pieejams: <https://mosquitto.org/>. [Piekļūts 29.12.2024].
- [6] «MongoDB» Pieejams: <https://www.mongodb.com/>. [Piekļūts 29.12.2024].
- [7] «Docker Compose» Pieejams: <https://docs.docker.com/compose/>. [Piekļūts 29.12.2024].
- [8] «Change Streams» Pieejams: <https://www.mongodb.com/docs/manual/changeStreams/>. [Piekļūts 29.12.2024].
- [9] «Connection Strings» MongoDB, Pieejams: <https://www.mongodb.com/docs/manual/reference/connection-string/>. [Piekļūts 29.12.2024].
- [10] «.NET dependency injection» Pieejams: <https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection>. [Piekļūts 29.12.2024].
- [11] «BackgroundService Class» Pieejams: <https://learn.microsoft.com/en-us/dotnet/api/microsoft.extensions.hosting.backgroundservice>. [Piekļūts 29.12.2024].
- [12] «MQTTnet» Pieejams: <https://github.com/dotnet/MQTTnet>.
- [13] «SignalR» Pieejams: <https://learn.microsoft.com/en-us/aspnet/signalr/overview/getting-started/introduction-to-signalr>. [Piekļūts 29.12.2024].
- [14] «Working with Groups in SignalR» Pieejams: <https://learn.microsoft.com/en-us/aspnet/signalr/overview/guide-to-the-api/working-with-groups>. [Piekļūts 29.12.2024].

[15] «TypeScript Interfaces» Pieejams:

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html#interfaces>. [Piekļūts 29.12.2024].

[16] «Component Communication with @Output» Pieejams:

<https://angular.dev/tutorials/learn-angular/9-output#add-an-output-property>. [Piekļūts 29.12.2024].

[17] «Angular Router reference» Pieejams: <https://angular.dev/guide/routing/router-reference>. [Piekļūts 29.12.2024].

[18] «Bootstrap Spinners» Pieejams:

<https://getbootstrap.com/docs/4.2/components/spinners/>. [Piekļūts 29.12.2024].

[19] «ngx-charts» Pieejams: <https://swimlane.gitbook.io/ngx-charts>. [Piekļūts 29.12.2024].

[20] «@microsoft/signalr - JavaScript and TypeScript clients for SignalR for ASP.NET Core and Azure SignalR Service» Pieejams: <https://www.npmjs.com/package/@microsoft/signalr>. [Piekļūts 29.12.2024].

[21] «Entity Framework DbContext» Pieejams: <https://learn.microsoft.com/en-us/ef/core/dbcontext-configuration/>. [Piekļūts 29.12.2024].

[22] «Entity Framework Core tools» Pieejams: <https://learn.microsoft.com/en-us/ef/core/cli/powershell>. [Piekļūts 29.12.2024].

[23] «Entity Framework Core and Multiple Database Providers» Khalid Abuhakmeh, 24 August 2022. Pieejams: <https://blog.jetbrains.com/dotnet/2022/08/24/entity-framework-core-and-multiple-database-providers/>. [Piekļūts 29.12.2024].

[24] «MQTT Shared Subscriptions» Pieejams: <https://www.hivemq.com/blog/mqtt5-essentials-part7-shared-subscriptions/>. [Piekļūts 29.12.2024].

[25] «Aranet Forum: How one can see and test sample of MQTT message format published from Aranet PRO base station?» Pieejams: <https://forum.aranet.com/aranet-bases-sensors/how-one-can-see-and-test-sample-of-mqtt-message-format-published-from-aranet-pro-base-station/>. [Piekļūts 29.12.2024].

- [26] «Deploy a Self-Managed Replica Set» Pieejams:
<https://www.mongodb.com/docs/manual/tutorial/deploy-replica-set/>. [Piekļūts 29.12.2024].
- [27] «Sensor.Consumer» Pieejams: <https://github.com/Markuss-B/sensor-consumer>.
- [28] «Sensor.WebApi» Pieejams: <https://github.com/Markuss-B/sensor-webapi>.
- [29] «Sensor.Dashboard» Pieejams: <https://github.com/Markuss-B/sensor-dashboard>.
- [30] «Sensor.Notifications» Pieejams: <https://github.com/Markuss-B/sensor-notifications>.
- [31] «Sensor.Courier» Pieejams: <https://github.com/Markuss-B/sensor-courier>.
- [32] «Semantic Versioning 2.0.0» Pieejams: <https://semver.org/spec/v2.0.0.html>. [Piekļūts 29.12.2024].
- [33] «Keep a Changelog» Pieejams: <https://keepachangelog.com/en/1.1.0/>. [Piekļūts 29.12.2024].
- [34] «Code-style rule options - Example EditorConfig file» Pieejams:
<https://learn.microsoft.com/en-us/dotnet/fundamentals/code-analysis/code-style-rule-options#example-editorconfig-file>. [Piekļūts 29.12.2024].
- [35] «.NET documentation» Pieejams: <https://learn.microsoft.com/en-us/dotnet/>. [Piekļūts 29.12.2024].
- [36] «What is Angular?» Pieejams: <https://angular.dev/overview>. [Piekļūts 29.12.2024].
- [37] «MongoDB C# Driver» Pieejams:
<https://www.mongodb.com/docs/drivers/csharp/current/>. [Piekļūts 29.12.2024].
- [38] «Software Project Performance Benchmark Tables» Pieejams:
<https://www.qsm.com/resources/qsm-benchmark-tables>. [Piekļūts 29.12.2024].
- [39] «VS Code Counter» Pieejams:
<https://marketplace.visualstudio.com/items?itemName=uctakeoff.vscode-counter>. [Piekļūts 29.12.2024].
- [40] «MQTTnet» Pieejams: <https://github.com/dotnet/MQTTnet>. [Piekļūts 29.12.2024].

PIELIKUMI

1. pielikums

“Sensor.Consumer” klase “MqttWorker” – atbild par MQTT savienojuma izveidošanu, uzturēšanu un ziņojumu saņemšanu.

```
using Microsoft.Extensions.Options;
using MQTTnet;
using MQTTnet.Client;
using SensorConsumer.Configuration;
using SensorConsumer.Helpers;
using SensorConsumer.Services;
using System.Net.Security;
using System.Security.Cryptography.X509Certificates;

namespace SensorConsumer.Workers;

/// <summary>
/// Worker service that connects to an MQTT broker, subscribes to topics and routes
incoming messages to a processing service.
/// </summary>
public class MqttWorker : BackgroundService
{
    private readonly ILogger<MqttWorker> _logger;
    private readonly IMqttClient _client;

    private readonly MqttSettings _settings;
    private readonly MqttClientOptions _options;

    private readonly ProcessingService _processingService;
    private int _messageCount = 0;

    public MqttWorker(ILogger<MqttWorker> logger, IOptions<MqttSettings> options,
        ProcessingService processingService)
    {
        _logger = logger;
        _settings = options.Value;
        _options = ConfigureOptions(_settings);
        var factory = new MqttFactory();
        _client = factory.CreateMqttClient();
        _processingService = processingService;
    }

    public override async Task StartAsync(CancellationToken cancellationToken)
    {
        await Connect(cancellationToken);

        await base.StartAsync(cancellationToken);
    }

    /// <summary>
    /// Starts receiving messages and keep the connection alive by pinging the
    broker every 5 seconds.
    /// </summary>
    protected override async Task ExecuteAsync(CancellationToken
        cancellationToken)
    {
        // Handle incoming messages
    }
}
```

```

        _client.ApplicationMessageReceivedAsync
        HandleReceivedMessage(cancellationToken, ea);

        while (!cancellationToken.IsCancellationRequested)
        {
            try
            {
                if (!await _client.TryPingAsync(cancellationToken))
                {
                    _logger.LogWarning("MQTT connection is not alive. Attempting
to reconnect.");
                    await Connect(cancellationToken);
                }

                _logger.LogInformation("MQTT connection is alive.");
            }
            catch (Exception ex)
            {
                _logger.LogError(ex, "An error occurred on ping attempt.");
            }

            // Check connection every 5 seconds
            await Task.Delay(5000, cancellationToken);
        }
    }

    public override async Task StopAsync(CancellationTokens cancellationToken)
    {
        _logger.LogInformation("StopAsync called, shutting down...");
        await _client.DisconnectAsync();

        await base.StopAsync(cancellationToken);
    }

    /// <summary>
    /// Handles the received message by processing it and acknowledging it.
    /// </summary>
    private async Task HandleReceivedMessage(CancellationTokens cancellationToken,
MqttApplicationMessageReceivedEventArgs e)
    {
        e.AutoAcknowledge = false;
        int messageNumber = _messageCount++;

        // Create the processing task
        async Task ProcessAsync()
        {
            // Create a logger scope for message
            using (_logger.BeginScope("Message number {messageNumber}, Topic:
{topic}", messageNumber, e.ApplicationMessage.Topic))
            {
                try
                {
                    _logger.LogInformation("Received message. Message count:
{messageNumber}", messageNumber);

                    if (_logger.IsEnabled(LogLevel.Debug))
                        _logger.LogDebug("Message: {message};Payload: {payload}",
e.ToString(), e.ApplicationMessage.ConvertPayloadToString());

                    // Process the message
                    await
_processingService.ProcessMessageAsync(e.ApplicationMessage.Topic,
e.ApplicationMessage.ConvertPayloadToString());
                    // Acknowledge the message
                    await e.AcknowledgeAsync(cancellationToken);
                }
            }
        }
    }

```

```

        _logger.LogInformation("Processed message number
{messageNumber}.", messageNumber);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error processing MQTT message.");
    }
}

// Run the processing task
_ = Task.Run(ProcessAsync, cancellationTokens);
}

/// <summary>
/// Connect to the MQTT broker and subscribe to topics.
/// </summary>
private async Task Connect(CancellationTokens cancellationTokens)
{
    int retryCount = 0;

    while (!_client.IsConnected && !cancellationTokens.IsCancellationRequested)
    {
        try
        {
            _logger.LogInformation("Connecting to MQTT broker: {broker}.",
            _settings.Broker);

            await _client.ConnectAsync(_options, cancellationTokens);
            _logger.LogInformation("Connected to MQTT broker: {broker}.",
            _settings.Broker);

            await SubscribeToTopics();
            _logger.LogInformation("Subscribed to topics");
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "An error occurred on connect attempt
{retryCount}.", retryCount);
            retryCount++;
            // Wait for a maximum of 30 seconds before retrying
            await Task.Delay(Math.Min(1000 * retryCount, 30000),
            cancellationTokens);
        }
    }
}

/// <summary>
/// Subscribe to the topics specified in the TopicsToSubscribeTo property.
/// </summary>
private async Task SubscribeToTopics()
{
    string[] topics = _settings.TopicSchemas.Select(s =>
s.TopicFilter).ToArray();

    if (topics.Length == 0)
    {
        _logger.LogWarning("No topics to subscribe to.");
        return;
    }
    _logger.LogDebug("Subscribing to topics: {topics}.", topics);
    foreach (var topic in topics)
    {
        await _client.SubscribeAsync(topic);
    }
}

```



```

        _logger.LogInformation("Subscribed to topic: {topic}", topic);
    }
}

/// <summary>
/// Configure the options for the MQTT client based on the settings.
/// </summary>
private MqttClientOptions ConfigureOptions(MqttSettings settings)
{
    var optionsBuilder = new MqttClientOptionsBuilder()
        .WithTcpServer(settings.Broker)
        .WithProtocolVersion(settings.ProtocolVersion)
        .WithCredentials(settings.Username, settings.Password);

    // Configure TLS options if needed
    if (settings.UseTls)
    {
        X509Certificate2Collection certificates = new();
        certificates.ImportFromPemFile(settings.PathToPem);

        optionsBuilder.WithTlsOptions(new MqttClientTlsOptions
        {
            UseTls = true,
            TrustChain = certificates,
            SslProtocol = settings.SslProtocol,
            CertificateValidationHandler = CertificateValidationHandler
        });
    }

    return optionsBuilder.Build();
}

/// <summary>
/// Logs information about the certificate validation.
/// </summary>
private bool CertificateValidationHandler(MqttClientCertificateValidationEventArgs eventArgs)
{
    _logger.LogInformation("Certificate Validation Handler called.");

    _logger.LogInformation("Certificate Subject: {subject}",
eventArgs.Certificate.Subject);
    _logger.LogInformation("Certificate Expiration Date: {expirationDate}",
eventArgs.Certificate.GetExpirationDateString());
    _logger.LogInformation("Chain Revocation Mode: {revocationMode}",
eventArgs.Chain.ChainPolicy.RevocationMode);
    _logger.LogInformation("Chain Status: {chainStatus}",
eventArgs.Chain.ChainStatus);
    if (eventArgs.SslPolicyErrors != SslPolicyErrors.None)
        _logger.LogWarning("SSL Policy Errors: {sslPolicyErrors}",
eventArgs.SslPolicyErrors);

    return true;
}
}

```

2. pielikums

“Sensor.Consumer” klase “SensorService” – atbild par sensoru datu saglabāšanu.

```
using MongoDB.Bson;
using MongoDB.Driver;
using SensorConsumer.Data;
using SensorConsumer.Models;

namespace SensorConsumer.Services;

/// <summary>
/// Service to handle sensor data. Saves sensor measurements and metadata to the
/// MongoDB database.
/// </summary>
public class SensorService
{
    private readonly ILogger<SensorService> _logger;
    private readonly MongoDB _db;
    private readonly InactiveSensorCache _inactiveSensorCache;

    public SensorService(ILogger<SensorService> logger, MongoDB db,
InactiveSensorCache inactiveSensorCache)
    {
        _logger = logger;
        _db = db;
        _inactiveSensorCache = inactiveSensorCache;
    }

    /// <summary>
    /// Check if the inactive sensor cache contains the sensor.
    /// </summary>
    public bool IsSensorInactive(string sensorId)
    {
        return _inactiveSensorCache.IsSensorInactive(sensorId);
    }

    /// <summary>
    /// Save the measurements to the database:
    /// Saves the measurements to the <see cref="MongoDb.sensorMeasurements"/>
collection and
    /// the sensor document in <see cref="MongoDb.sensors"/> collection with the
latest measurements.
    /// </summary>
    public async Task SaveMeasurementsAsync(string sensorId, DateTime timestamp,
BsonDocument measurements)
    {
        // Create a SensorMeasurement object
        var sensorMeasurement = new SensorMeasurements
        {
            SensorId = sensorId,
            Timestamp = timestamp,
            Measurements = measurements
        };

        List<Task> tasks = new();
        // Insert the measurements into the sensorMeasurements collection
        tasks.Add(_db.sensorMeasurements.InsertOneAsync(sensorMeasurement));

        // Update the sensor document with the latest measurements
        var filter = Builders<Sensor>.Filter.Eq(s => s.Id, sensorId);
        var update = Builders<Sensor>.Update
```

```

        .Set(s => s.LatestMeasurements, measurements)
        .Set(s => s.LatestMeasurementTimestamp, timestamp);

tasks.Add(_db.sensors.UpdateOneAsync(filter, update));

await Task.WhenAll(tasks);

_logger.LogInformation("Saved measurements for sensor with ID '{SensorId}'
with timestamp '{Timestamp}'.", sensorId, timestamp);
}

/// <summary>
/// Update the sensor topics of the sensor.
/// </summary>
public async Task UpdateSensorTopicsAsync(string sensorId, string topic)
{
    var filter = Builders<Sensor>.Filter.Eq(s => s.Id, sensorId);
    var update = Builders<Sensor>.Update
        .AddToSet(s => s.Topics, topic);

    var updateOptions = new UpdateOptions { IsUpsert = true };

    var result = await _db.sensors.UpdateOneAsync(filter, update,
updateOptions);

    if (result.UpsertedId != null)
    {
        _logger.LogInformation("A new document was inserted with ID
'{DocumentId}' and topic '{Topic}'.", result.UpsertedId, topic);
    }
    else if (result.ModifiedCount == 0)
    {
        _logger.LogInformation("Sensor with ID '{SensorId}' already had the
topic '{Topic}'. No changes were made.", sensorId, topic);
    }
    else
    {
        _logger.LogInformation("Successfully added topic '{Topic}' to sensor
with ID '{SensorId}'.", topic, sensorId);
    }
}

/// <summary>
/// Update the sensor metadata:
/// Saves the metadata to the the sensor document in <see
cref="MongoDb.sensors"/> collection
/// and if the metadata is new, adds it to the <see
cref="MongoDb.sensorMetadatas"/> collection to keep a history of the metadata
changes.
/// </summary>
public async Task UpdateSensorMetadataAsync(string sensorId, string fieldName,
string newValue)
{
    var filter = Builders<Sensor>.Filter.Eq(s => s.Id, sensorId);
    var update = Builders<Sensor>.Update
        .Set(s => s.Metadata[fieldName], newValue);

    // Upsert so that a new document is created if the sensor does not exist
    var updateOptions = new UpdateOptions { IsUpsert = true };

    var result = await _db.sensors.UpdateOneAsync(filter, update,
updateOptions);

    if (result.UpsertedId != null)
    {

```

```

        // A new document was created
        await SaveMetadataHistory(sensorId, fieldName, newValue);
        _logger.LogInformation("A new document was inserted with ID
'{DocumentId}' and field '{FieldName}' with value '{NewValue}'.",
result.UpsertedId, fieldName, newValue);
    }
    else if (result.ModifiedCount > 0)
    {
        // An existing sensor was updated
        await SaveMetadataHistory(sensorId, fieldName, newValue);
        _logger.LogInformation("Successfully updated sensor with ID
'{SensorId}' and field '{FieldName}' to '{NewValue}'.", sensorId, fieldName,
newValue);
    }
    else
    {
        _logger.LogInformation("Sensor with ID '{SensorId}' already had the
same value for the field '{FieldName}'. No changes were made.", sensorId,
fieldName);
    }
}

private async Task SaveMetadataHistory(string sensorId, string fieldName,
string newValue)
{
    var series = new SensorMetadatas
    {
        SensorId = sensorId,
        Timestamp = DateTime.UtcNow,
        Metadata = new BsonDocument
        {
            { fieldName, newValue }
        }
    };

    await _db.sensorMetadatas.InsertOneAsync(series);

    _logger.LogInformation("Saved metadata history for sensor with ID
'{SensorId}' with field '{FieldName}' and value '{NewValue}'.", sensorId,
fieldName, newValue);
}
}

```

3. pielikums

“Sensor.WebApi” klase “SensorWatcherService” – skatās un sūta skatīto sensoru jaunus mērījumus.

```
using Microsoft.AspNetCore.SignalR;
using MongoDB.Driver;
using SensorWebApi.Data;
using SensorWebApi.Models;
using System.Collections.Concurrent;

/// <summary>
/// Service that watches for changes in the sensor measurements collection and
/// sends updates to clients managed by <see cref="SensorHub"/>.
/// </summary>
public class SensorWatcherService
{
    private readonly ILogger<SensorWatcherService> _logger;
    private readonly MongoDB _context;
    private readonly IHubContext<SensorHub> _hubContext;
    private ConcurrentDictionary<string, byte> _watchedSensors;
    private CancellationTokenSource _cancellationTokensource;
    private Task _watchTask;

    public SensorWatcherService(
        ILogger<SensorWatcherService> logger,
        MongoDB context,
        IHubContext<SensorHub> hubContext)
    {
        _logger = logger;
        _context = context;
        _hubContext = hubContext;
        _watchedSensors = new ConcurrentDictionary<string, byte>();
        _cancellationTokensource = new CancellationTokenSource();
        _watchTask = RunAsync(_cancellationTokensource.Token); // Start the
initial change stream
        _logger.LogInformation("SensorWatcherService initialized.");
    }

    /// <summary>
    /// Adds a sensorId to the watch list and restarts the change stream. If the
    /// sensorId is already being watched, this method does nothing.
    /// </summary>
    public void AddSensorId(string sensorId)
    {
        bool added = _watchedSensors.TryAdd(sensorId, 0);

        if (added)
        {
            _logger.LogInformation("Added sensorId {SensorId} to watch list.",
sensorId);
            RestartChangeStream();
        }
    }

    /// <summary>
    /// Removes a sensorId from the watch list and restarts the change stream. If
    the sensorId is not being watched, this method does nothing.
    /// </summary>
    public void RemoveSensorId(string sensorId)
    {

```

```

        bool removed = _watchedSensors.TryRemove(sensorId, out _);
        if (removed)
        {
            _logger.LogInformation("Removed sensorId {SensorId} from watch list.",
sensorId);
            RestartChangeStream();
        }
    }

    /// <summary>
    /// Restarts the change stream with the updated list of watched sensors.
    /// </summary>
    private async void RestartChangeStream()
    {
        _logger.LogInformation("Restarting change stream.");
        _cancellationTokenSource.Cancel();
        _cancellationTokenSource = new CancellationTokenSource();
        await _watchTask;
        _watchTask = RunAsync(_cancellationTokenSource.Token);
    }

    /// <summary>
    /// Watches for changes in the sensor measurements collection and sends updates
to clients.
    /// </summary>
    private async Task RunAsync(Cancellation token cancellationToken)
    {
        var col = _context.SensorMeasurements;

        while (!cancellationToken.IsCancellationRequested && _watchedSensors.Count
> 0)
        {
            try
            {
                _logger.LogInformation("Starting change stream to watch
[{WatchedSensors}].", string.Join(", ", _watchedSensors.Keys));

                // Only watch for insert operations on the watched sensors
                var pipeline = new
EmptyPipelineDefinition<ChangeStreamDocument<SensorMeasurements>>()
                .Match(change =>
                    change.OperationType == ChangeStreamOperationType.Insert
&&
                    _watchedSensors.ContainsKey(change.FullDocument.SensorId));

                // Include the full document in the change stream event
                var options = new ChangeStreamOptions
                {
                    FullDocument = ChangeStreamFullDocumentOption.UpdateLookup,
                };

                // Start the change stream
                using var cursor = await col.WatchAsync(pipeline, options,
cancellationToken);

                _logger.LogInformation("Watching for changes in sensor
measurements collection.");

                // Process change stream events
                await cursor.ForEachAsync(async change =>
                {

                    var sensorMeasurement = change.FullDocument;
                    var sensorId = sensorMeasurement.SensorId;

```

```

        _logger.LogInformation("Change stream detected for sensorId
{SensorId}.", sensorId);

        // Send update to clients subscribed to the sensor
        if (_watchedSensors.TryGetValue(sensorId, out var clients))
        {
            _logger.LogDebug("Sending update for sensorId {SensorId}
to clients.", sensorId);

            await _hubContext.Clients.Group(sensorId)
                .SendAsync("ReceiveSensorUpdate", sensorMeasurement);

            _logger.LogInformation("Update sent for sensorId
{SensorId}.", sensorId);
        }
    }, cancellationToken);
}
catch (OperationCanceledException)
{
    _logger.LogInformation("Change stream canceled.");
}
catch (Exception ex)
{
    _logger.LogError($"Error in change stream: {ex.Message}");
    await Task.Delay(1000, cancellationToken); // Brief delay before
retrying
}
}
}
}

```

4. pielikums

“Sensor.Courier” galvenā procesu pārvaldības klase.

```
using SensorCourier.App.Models;
using SensorCourier.App.Services;

namespace SensorCourier.App;

/// <summary>
/// Manages the ETL process.
/// </summary>
public class Worker : BackgroundService
{
    private readonly ILogger<Worker> _logger;
    private readonly IServiceProvider _serviceProvider;

    public Worker(ILogger<Worker> logger, IServiceProvider serviceProvider)
    {
        _logger = logger;
        _serviceProvider = serviceProvider;
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        AppSettings appSettings;
        double batchDelaySeconds = 100; // initial value incase of error
        DateTime lastMeasurementTime;
        DateTime lastMetadataTime;

        while (!stoppingToken.IsCancellationRequested)
        {
            Task[] tasks = null; // stores the ETL tasks
            try
            {
                // Create scope for ParameterService
                using (var scope = _serviceProvider.CreateScope())
                {
                    var parameterService = scope.ServiceProvider.GetRequiredService<ParameterService>();
                    // Get parameters from the database: BatchDelaySeconds,
                    // BatchSize, LastDateTime and Monitor.
                    // ParameterService throws an exception if parameters cannot
                    // be loaded.
                    appSettings = await parameterService.GetAppSettings(stoppingToken);
                    lastMeasurementTime = await parameterService.GetLastMeasurementTimeStamp(stoppingToken);
                    lastMetadataTime = await parameterService.GetLastMetadataTimeStamp(stoppingToken);
                    batchDelaySeconds = appSettings.BatchDelaySeconds;
                }

                _logger.LogInformation("Starting batch of size {batchSize} processing at {time}.", appSettings.BatchSize, DateTime.Now);

                // Create scope1 for the ETLService
                using var scope1 = _serviceProvider.CreateScope();
                var etlService1 = scope1.ServiceProvider.GetRequiredService<ETLService>();
```



```

        var etlTask1 =
etlService1.ExtractAndLoadMeasurements(lastMeasurementTime,
appSettings.BatchSize, stoppingToken);

        // Create scope2 for the ETLService
        using var scope2 = _serviceProvider.CreateScope();
        var etlService2 =
scope2.ServiceProvider.GetRequiredService<ETLService>();
        var etlTask2 =
etlService2.ExtractAndLoadMetadatas(lastMetadataTime, appSettings.BatchSize,
stoppingToken);

        tasks = new[] { etlTask1, etlTask2 };
        // Wait for both ETL tasks to complete
        await Task.WhenAll(etlTask1, etlTask2);

        _logger.LogInformation("Data processed successfully.");
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "An error occurred while processing data.");
        foreach (var task in tasks)
        {
            if (task.IsFaulted)
                _logger.LogError(task.Exception.Flatten(), "An error
occurred while processing data.");
        }
    }

    _logger.LogInformation("Next batch in {batchDelay} seconds at
{time}.",
        TimeSpan.FromSeconds(batchDelaySeconds),
        DateTime.Now +
        TimeSpan.FromSeconds(batchDelaySeconds));

    await Task.Delay(TimeSpan.FromSeconds(batchDelaySeconds),
stoppingToken);
    }
}

```

5. pielikums

“SensorDashboard” serviss “SensorHubService” – atbild par skatīto sensoru abonēšanu, lai saņemtu reālā laika mērījumus.

```
import { Injectable } from '@angular/core';
import * as signalR from '@microsoft/signalr';
import { SensorMeasurements } from '../models/sensor-measurements';
import { BehaviorSubject, Observable } from 'rxjs';
import { environment } from 'environments/environment';

@Injectable({
  providedIn: 'root'
})

/**
 * Service for subscribing to sensor updates from the SensorHub SignalR hub.
 */
export class SensorHubService {
  private hubConnection: signalR.HubConnection;
  private sensorMeasurementsSubject = new BehaviorSubject<SensorMeasurements | null>(null);
  private connectionPromise: Promise<void>;

  private hubUrl = environment.apiUrl + '/api/sensorhub';

  /**
   * Initializes the SensorHubService by creating a SignalR hub connection.
   */
  constructor() {
    this.hubConnection = new signalR.HubConnectionBuilder()
      .withUrl(this.hubUrl)
      .build();

    // Handle incoming sensor updates
    this.hubConnection.on('ReceiveSensorUpdate', (measurement:
SensorMeasurements) => {
      this.sensorMeasurementsSubject.next(measurement);
    });

    // Initialize connection promise
    this.connectionPromise = this.hubConnection.start()
      .then(() => console.log('SignalR connection established'))
      .catch(err => console.error('SignalR connection error:', err));
  }

  /**
   * Subscribes to a sensor to receive real-time updates receivable by the
   {@link getSensorUpdates} observable.

```

```

    * @param sensorId
    */
    async subscribeToSensor(sensorId: string) {
        console.log('Subscribing to sensor:', sensorId);

        await this.connectionPromise; // Ensure the connection is established
        this.hubConnection.invoke('SubscribeToSensor', sensorId)
            .catch(err => console.error('Subscription error:', err));
    }

    /**
     * Unsubscribes from a sensor to stop receiving real-time updates.
     * @param sensorId
     */
    async unsubscribeFromSensor(sensorId: string) {
        console.log('Unsubscribing from sensor:', sensorId);

        await this.connectionPromise; // Ensure the connection is established
        this.hubConnection.invoke('UnsubscribeFromSensor', sensorId)
            .catch(err => console.error('Unsubscription error:', err));
    }

    /**
     * Returns an observable that emits sensor updates.
     * @returns An observable that emits sensor updates.
     */
    getSensorUpdates(): Observable<SensorMeasurements> {
        return this.sensorMeasurementsSubject.asObservable() as
Observable<SensorMeasurements>;
    }
}

```

Kvalifikācijas darbs “**Sensoru monitoringa sistēma**” izstrādāts Latvijas Universitātes Eksakto zinātņu un tehnoloģiju fakultātē, Datorikas nodaļā.

Ar savu parakstu apliecinu, ka darbs izstrādāts patstāvīgi, izmantoti tikai tajā norādītie informācijas avoti un iesniegtā darba elektroniskā kopija atbilst izdrukai un/vai recenzentam uzrādītajai darba versijai.

Autors: **Markuss Birznieks** _____ **06.01.2025.**

Rekomendēju darbu aizstāvēšanai

Darba vadītājs/a: **Dr. dat. Guntis Arnicāns** _____ **06.01.2025.**

Recenzents: **Andris Bozis**

Darbs iesniegts **06.01.2025.**

Kvalifikācijas darbu pārbaudījumu komisijas sekretārs (elektronisks paraksts)